

Arnau Faura i Ciré

Sistema Bimodal per a Entorns Corporatius: RAG Local amb RBAC, Recuperació Documental Híbrida, Encaminament Semàntic i Flux d'Ingesta Segur.

TREBALL DE FI DE GRAU

dirigit per Jordi Massaguer Pla

Grau d'Enginyeria Informàtica



UNIVERSITAT ROVIRA I VIRGILI

Tarragona

2026

Vull agrair el suport del tutor del treball Jordi Massaguer en el desenvolupament del projecte. Gràcies per ajudar quan ho he necessitat i ha deixar prou llibertat per prendre decisions i fer canvis que considerava.

També a la meva família i parella per animar-me i sobretot haver-me donat els mitjans per poder acabar descobrint el que m'agrada fer.

Resum.

Els assistents RAG existents permeten l'accés semàntic a corpus corporatius, però comprometen la confidencialitat, ja que depenen d'*APIs cloud* externes i no apliquen control d'accés a nivell de fragment (*chunk*). Aquest projecte es divideix en dues parts principals: d'una banda, un estudi exhaustiu orientat a identificar el disseny de solució més òptim, mitjançant la comparació de diferents enfocaments i l'anàlisi de les mètriques obtingudes per generar resultats avaluable; de l'altra, el disseny d'un sistema RAG empresarial 100% *on-premise* que implementa un model Zero Trust per a cada *chunk* i que segueix els principis derivats de l'estudi inicial.

La metodologia s'estructura en tres fases d'exploració desenvolupades amb *Jupyter Notebooks* (*chunking* amb consciència de RBAC, recuperació híbrida amb RRF i encaminament semàntic mitjançant KSP), complementades per una implementació per capes en què s'apliquen i es validen les decisions clau de disseny. El resultat és un sistema desplegable mitjançant una pila de contenidors *Docker*.

Resumen.

Los asistentes RAG existentes permiten el acceso semántico a corpus corporativos, pero comprometen la confidencialidad, puesto que dependen de *APIs cloud* externas y no aplican control de acceso a nivel de fragmento (*chunk*). Este proyecto se divide en dos partes principales: por un lado, un exhaustivo estudio orientado a identificar el diseño de solución más óptimo, mediante la comparación de diferentes enfoques y el análisis de las métricas obtenidas para generar resultados evaluables; por otro, el diseño de un sistema RAG empresarial 100% *on-premise* que implementa un modelo *Zero Trust* para cada *chunk* y que sigue los principios derivados del estudio inicial.

La metodología se estructura en tres fases de exploración desarrolladas con *Jupyter Notebooks* (*chunking* con conciencia de RBAC, recuperación híbrida con RRF y encaminamiento semántico mediante KSP), complementadas por una implementación por capas en la que se aplican y validan las decisiones clave de diseño. El resultado es un sistema desplegable mediante un montón de contenedores *Docker*.

Abstract.

Existing RAG wizards allow semantic access to corporate corpora, but compromise confidentiality, as they rely on external cloud APIs and do not implement access control at the chunk level. This project is divided into two main parts: on the one hand, an exhaustive study aimed at identifying the most optimal solution design, by comparing different approaches and analyzing the metrics obtained to generate evaluable results; on the other, the design of a 100% on-premise enterprise RAG system that implements a Zero Trust model for each chunk and follows the principles derived from the initial study.

The methodology is structured in three exploration phases developed with Jupyter Notebooks (RBAC-aware chunking, hybrid retrieval with RRF and semantic routing using KSP), complemented by a layered implementation in which key design decisions are applied and validated. The result is a deployable system using a stack of Docker containers

*L'annex Declaració d'ús d'IA detalla les pràctiques i usos que se li han donat a aquesta per al desenvolupament del projecte. Veure per a més informació.

Paraules Clau i Abreviacions

RAG · Retrieval Augmented Generation

LLM · Large Language Model

RBAC · Role-Based Access Control

PII · Personally Identifiable Information

HITL · Human-In-The-Loop

KSP · Key Semantic Points

RRF · Reciprocal Rank Fusion

BM25 · Best Matching 25

IDF · Inverse Document Frequency

TF · Term Frequency

API · Application Programming Interface

ABC · Abstract Base Class

FIFO · First-In First-Out

LRU · Least Recently Used

UI · User Interface

TFG · Treball de Fi de Grau

NLP · Natural Language Processing

kNN · k-Nearest Neighbors

IBAN · International Bank Account Number

JSON · JavaScript Object Notation

YAML · YAML Ain't a Markup Language

HTTP · HyperText Transfer Protocol

CORS · Cross-Origin Resource Sharing

RF i RNF · Requisits Funcionals i Requisits No Funcionals

S.O. · Sistema Operatiu

E2E · End-to-End

Índex

1. Introducció.....	10
1.1.1. Motivació.....	10
1.1.2. Problema i Solució	10
1.1.3. Abast.....	11
2. Objectius del Projecte.....	13
3. Planificació	14
4. Requisits	15
4.1.1. Requisits Funcionals (RF)	15
4.1.2. Requisits No Funcionals (RNF)	16
5. Estat de la Tècnica	18
5.1. Anàlisi d'Alternatives – Bases de Dades Vectorials.....	18
5.1.1. FAISS.....	18
5.1.2. Milvus.....	19
5.1.3. Qdrant	19
5.1.4. ChromaDB.....	19
5.1.5. Conclusió i Justificació.....	20
5.2. Anàlisi d'Alternatives – <i>Frameworks</i> d'Integració RAG.....	22
5.2.1. Alternatives valorades	22
5.2.2. Decisió.....	22
5.3. Anàlisi d'Alternatives – Models de Llenguatge Generatiu LLMs.....	23
5.3.1. Context Teòric i Restriccions.....	23
5.3.2. Anàlisi de Maquinari i Pressupost	24
5.3.3. Comparativa de Models i Elecció de la Solució Òptima.....	24
5.3.4. Projecció a l'Entorn Real (Escalabilitat)	25
5.4. Anàlisi d'Alternatives – Models d' <i>Embedding</i>	25
5.4.1. Anàlisi d'Alternatives.....	25
5.4.2. Decisió Final.....	25
5.5. Anàlisi d'Alternatives – Motors d'Inferència Local.....	26
5.5.1. Anàlisi d'Alternatives Locals	26
5.5.2. Decisió Final.....	26
6. Disseny	27
6.1. Fase 1 – Ingesta: Estratègies de <i>Chunking</i> i Assignació de Metadades RBAC... 30	
6.1.1. Plantejament del Problema	30
6.1.2. Entorn d'Experimentació.....	30
6.1.3. Càrrega Multiformat i Metadades Globals	32
6.1.4. Estratègies de <i>Chunking</i> Avaluades.....	32

6.1.5.	Detecció de PII i Model de Clearance	34
6.1.6.	Resultats i Validació RBAC	35
6.1.7.	Limitacions i Conclusions de la Fase 1	37
6.2.	Fase 2 – Recuperació: <i>Ensemble Retriever</i> Híbrid amb Seguretat RBAC	37
6.2.1.	Problemes Heretats de la Fase 1	37
6.2.2.	Fonaments dels Motors de Cerca	38
6.2.3.	Justificació de l'Ensemble Híbrid (RRF)	39
6.2.4.	Metodologia – Pipeline de 5 passos	40
6.2.5.	Pas 1: Baseline sense Protecció	41
6.2.6.	Pas 2: Pesos RRF Dinàmics	43
6.2.7.	Pas 3: Reconstrucció Pare-Fill.....	49
6.2.8.	Pas 4: Fusió Asimètrica amb Filtratge.....	55
6.2.9.	Pas 5: Consolidació de Mètriques	59
6.2.10.	Validació a Escala: Augment del Corpus i el Rol de la Fusió Asimètrica	61
6.3.	Fase 3 – Enrutament Multiagent (Routing)	64
6.3.1.	Resolució del Deute Tècnic – Classificador Cognitiu de Metadades (Pas 1)	64
6.3.2.	Agent d'Ingesta I Router Index (Pas 2)	66
6.3.3.	Desqualificació del Baseline Estàndard (Pas 3)	68
6.3.4.	Arquitectura Final (Pas 4).....	70
7.	Implementació	72
7.1.	Estructura del Sistema	72
7.2.	Metodologia de Treball.....	73
7.2.1.	Fail Fast	73
7.2.2.	Single Source of Truth.....	74
7.3.	Capa <i>Core</i>	74
7.3.1.	Settings – Configuració Centralitzada	74
7.3.2.	Enums	75
7.3.3.	Jerarquia d'Excepcions.....	76
7.4.	Capa d'Infraestructura	77
7.4.1.	ChromaDBClient	77
7.4.2.	BM25Client	77
7.4.3.	OllamaClient.....	78
7.4.4.	Embedder.....	78
7.4.5.	JsonUserStore	79
7.4.6.	Script Verificació de Salut	79
7.5.	Capa del Domini (<i>Domain</i>)	80
7.5.1.	Chunking	80

7.5.2.	Routing	85
7.5.3.	Retrieval	89
7.6.	Tractament d'Usuaris.....	93
7.6.1.	User – Value Object.....	93
7.6.2.	Excepcions i Abstraccions	93
7.6.3.	Implementació i Validació	94
7.7.	Capa d'Aplicació (Application)	95
7.7.1.	L'App d'Ingesta	95
7.7.2.	L'App de Consulta:.....	98
7.8.	Capa de l'API (FastAPI)	99
7.8.1.	Endpoints.....	100
7.9.	Desplegament (Local & Dockerització).....	101
7.10.	Capa Frontend (Streamlit)	105
8.	Avaluació	107
9.	Legislació	110
10.	Implicacions Ètiques, d'Igualtat i Mediambientals	112
11.	Avaluació de Costos	113
11.1.	Estimació de l'Esforç i la Durada.....	113
11.2.	Estimació del Cost Personal	114
11.3.	Estimació del Cost Material	114
11.4.	Cost Llicències	114
11.5.	Cost Total Estimat.....	114
12.	Conclusions i Reptes Futurs	115
13.	Referències	116
14.	Recursos	117
15.	Annexos.....	118
15.1.	Declaració d'ús d'IA.	118
15.2.	Guia d'Instal·lació & Desplegament	119
15.2.1.	Prerequisits	119
15.2.2.	Procediment.....	119
15.2.3.	Aturar / Reiniciar / Netejar	120

Índex de Taules

Taula 1 - Relació requisits funcionals - cassos d'ús	17
Taula 2 - Descripció corpus utilitzat per proves	28
Taula 3 - Descripció Arxius pel Disseny de Chunking	31
Taula 4 - Fitxers de sortida obtinguts del estudi de Chunking	32
Taula 5 - Tipus de patrons PII beta Chunking	35
Taula 6 - Mètriques de les tècniques de chunking	35
Taula 7 - Resultats de tipus de chunking	36
Taula 8 - Descripció Arxius pel Disseny de Retrieval	41
Taula 9 - Resultats del Retriever baseline	42
Taula 10 - Categories regex beta enrutador d'intencions	45
Taula 11 - Resultats esperats per queries suit ph2-step2	46
Taula 12 - Mètriques RRF retrieval	46
Taula 13 - Joc de proves pas 3 estratègia retrieval	52
Taula 14 - Resultats productivitat LLM amb Recuperació Pare-Fill.....	54
Taula 15 - Eliminació total de violacions en fase de disseny	57
Taula 16 - Latència tests retriever asimètric	57
Taula 17 - MRR resultats tests retriever asimètric	58
Taula 18 - Evolució de la seguretat del retrieving	59
Taula 19 - Resum fitxers afegits al corpus augmentat.....	61
Taula 20 - Mètriques del nou corpus respecte l'anterior	61
Taula 21 - Millora del MRR aplicant Intent Router amb un corpus més gran	62
Taula 22 - Resultats d'assignació camps globals amb Llama3.2	65
Taula 23 - Resultats KSP vs full-text embeddings al Agent Ingesta	66
Taula 24 - Resultats MultiRetrievalQACchain vs Index-Router.....	68
Taula 25 - Resultats latència en disseny del procés de consulta.....	70
Taula 26 - α per regles regex en producció.....	85
Taula 27 - Usuaris dissenyats del sistema	94
Taula 28 - Mapeig excepcions a codis HTTP	99
Taula 29 - Endpoints de l'API.....	100
Taula 30 - Taula ports dels serveis.....	101
Taula 31 - Mount-Binds dels containers.....	104
Taula 32 - Taula de Tests e2e sobre el sistema	109
Taula 33 - Resum tests i resultats	109
Taula 34 - Mètriques observades	109
Taula 35 - Esforç de la guia docent	113
Taula 36 - Esforç real x hores	113
Taula 37 - Estimació cost personal	114
Taula 38 - Estimació cost material	114
Taula 39 - Cost llicències.....	114
Taula 40 - Cost total estimat	114

Índex de Figures

Figura 1 - Diagrama de Gantt desenvolupament projecte	14
Figura 2 - Diagrama cassos d'ús	15
Figura 3 - Adopció de BBDD vectorials en IA/RAG empresarial	21
Figura 4 - Adopció de BBDD vectorials en IA/RAG empresarial	21
Figura 5 - Diagrama d'estats del sistema	29
Figura 6 - Exemple del problema de Chunking.....	30

Figura 7 - Exemple de cerca amb clàusula where en ChromaDB	39
Figura 8 - Fórmula Retriever Híbrid - RRF	39
Figura 9 - Tests per la estratègia de disseny del pas 3 del retrieval	51
Figura 10 - PII Chunk Restoration	52
Figura 11 - Confidence of Routers & Distance Distribution per document graphics	67
Figura 12 - Latency & Scaling Projection Router-Index vs LLM-based Comparison.....	68
Figura 13 - Layer Structure Architecture.....	72
Figura 14 - Estructura enum clearance	75
Figura 15 - Estructures enum department i allowed_department.....	75
Figura 16 - Diagrama jerarquia dissenyada de les excepcions.....	76
Figura 17 - Metadades de cada chunk (producció).....	80
Figura 18 - Extensions fitxers suportades (implementació)	81
Figura 19 - Diagrama flux del chunker	84
Figura 20 - Definició funció punt d'entrada al routing	87
Figura 21 - Diagrama flux funcionament router.....	88
Figura 22 - Esquema aplicació RBAC 2D sobre consulta	89
Figura 23 - Diagrama flux del retrieval	91
Figura 24 - Estructura classe User pel domain/ i application/	93
Figura 25 - Esquema json per al LLMDocumentClassifier.....	95
Figura 26 – IngestionReport results fields.....	96
Figura 27 - Diagrama de comunicació de la fase Ingesta.....	97
Figura 28 – Diagrama de desplegament híbrid.....	101
Figura 29 – Diagrama de desplegament full-Docker	102
Figura 30 - Imatge solució web, ingestion tab.....	106
Figura 31 - Imatge solució web, query tab amb telemetria	106

1. Introducció

*El repositori amb el codi és públic i es troba a:

https://github.com/MrSkinnyKraken/corporate_rbac_rag.git

1.1.1. Motivació

Les organitzacions acumulen volums massius de coneixement intern repartit entre diferents tipus de documents i amb diferents formats com ara contractes, informes financers, manuals d'enginyeria, polítiques de RRHH, comunicacions legals i d'altres. Tota aquesta informació sol estar desorganitzada i repartida en diferents llocs, el que la fa poc accessible. Un exemple pot ser voler trobar quins protocols s'apliquen a un cas operacional. **Els assistents basats en Retrieval-Augmented Generation (RAG)** sobre models de llenguatge generatius (LLMs) **ofereixen una solució**: Tenint accés a una base de coneixement pròpia, busquen i proporcionen documents semànticament semblants al dubte inicial i els proporcionen al LLM, que genera una resposta a partir de la informació del document proporcionat.

Aquests sistemes han topat amb dues barreres principals. La primera és la dependència de proveïdors cloud externs (OpenAI, Google...) per al procés d'inferència del LLM. En un context corporatiu amb informació sensible, enviar el context recuperat a una API externa sol ser incompatible amb la normativa interna i el RGPD, i en escenaris regulats com financers o sanitari simplement no és viable. **La segona és la falta de control d'accés a la informació**: un sistema RAG ingenu que indexi tot el corpus corporatiu sense filtres exposa qualsevol document a qualsevol usuari. Filtrar a posteriori a la capa del LLM tampoc resol el problema, ja que en el moment que el *chunk* arriba al *prompt*, la informació ja s'ha mostrat al motor d'inferència.

Aquest projecte neix de la idea que un sistema RAG empresarial seriós requereix dos atributs estructurals indispensables: ha de córrer completament on-premise (cap dada surt mai del perímetre de l'organització) i ha d'aplicar un model de seguretat Zero-Trust granular per chunk, evitant que arribi al LLM qualsevol informació inaccessible per l'usuari. La combinació d'aquests dos requisits, junt amb l'interès per la seguretat i el creixent ús de la IA, ha motivat la realització d'aquest treball, **no només amb l'objectiu de construir una eina, sinó de dur a terme una investigació rigorosa prèvia per validar quines estratègies algorítmiques són les òptimes per resoldre aquests reptes abans de la seva implementació definitiva.**

1.1.2. Problema i Solució

El problema

Considerarem el següent escenari realista. Una empresa amb 7 departaments (RRHH, Finances, Enginyeria, Legal, Vendes, Operacions, Màrqueting) i s'han definit usuaris amb 4 nivells de confidencialitat (PUBLIC, INTERNAL, CONFIDENTIAL, STRICT) vol desplegar un assistent RAG sobre tot el seu corpus (informació) documental. Tots els documents tenen:

1. Material públic que tothom pot llegir
2. Material confidencial per departament
3. Material confidencial accessible per múltiples departaments.

4. PII estrictament protegits que només poden ser accessibles per usuaris amb un nivell de permisos molt alt, encara que estiguin barrejats amb text públic.

Un sistema RAG normal fracassaria en donar solució a aquest problema per una pèrdua semàntica en fer la fragmentació de la informació. Una estratègia de *chunking* de la informació bàsica seria incapaç de separar context públic del privat i acabaria barrejant PII amb material públic al mateix *chunk*. També s'ha de buscar una solució a la eficiència de la persistència de la informació que afectarà directament a la capacitat de recuperar aquesta per passar-li al motor d'inferència.

La solució

Aquest TFG dissenya, implementa i valida un sistema RAG amb arquitectura *Clean Architecture*, *Zero-Trust* per chunk i desplegament 100% *on-premise* que els problemes principals de seguretat i eficiència amb una combinació coherent de decisions arquitectòniques:

- **Model 2D de seguretat:** Cada *chunk* porta `clearance_level: 0..3` (eix sever) $\times$ `allowed_departments: list[str]` (eix scope), permetent aplicar un filtre que cobreixi aquests dos previ a accedir a la informació.
- **Custom RBAC Chunker amb aïllament PII:** Un sistema d'ingesta de documents que tracti la separació de interna en els fragments estructurals per separar informació confidencial d'altres tipus preservant el nivell de seguretat.
- **Asymmetric Hybrid Retrieval:** Solució de cerca dual (vectorial i lèxica) + mecanisme de ranking per juntar els resultats ordenant-los segons quin tipus de cerca beneficiï més a la pregunta de l'usuari.
- **Hierichal Routing (KSP Router-Index):** Creació d'una col·lecció especialitzada, on existeix un vector per document al sistema amb la informació principal de cada document que ajudarà al encaminador a decidir sobre quina col·lecció buscar informació per resoldre la pregunta.

1.1.3. Abast

El sistema implementat cobreix els següents punts:

- **Ingesta multi-format:** Implementació de parsers per a 9 extensions de fitxer (`.txt`, `.md`, `.csv`, `.pdf`, `.docx`, `.xlsx`, `.pptx`, `.html`, `.htm`) amb un sistema de *dispatch* automàtic segons l'extensió del document.
- **Pipeline d'ingesta en dues fases (HITL):** Sistema *Human-In-The-Loop* on l'LLM proposa les metadades (`department`, `clearance`, abast i resum semàntic) i l'usuari les valida o edita abans de realitzar el *commit* irreversible a les bases de dades.
- **Chunking personalitzat amb aïllament de PII:** Detecció robusta mitjançant regex de 9 patrons de PII (dades sensibles), amb un algorisme de fusió d'interval·ls encavalcats i escalada automàtica del nivell de seguretat (`clearance`).
- **Routing semàntic jeràrquic:** Ús d'un índex de rutes KSP (*Key Semantic Points*) amb un classificador d'intenció multilingüe (EN/ES/CA) per calcular el pes α dinàmic de la cerca.
- **Asymmetric Hybrid Retrieval:** Combinació de cerca vectorial (ChromaDB) i lèxica (BM25). Aplica el filtre de seguretat RBAC abans de la fusió de resultats i realitza una reconstrucció de context pare-fill.

- **Generació amb LLM local:** Ús d'Ollama amb Llama 3.2 en local. El *system prompt* força l'ús exclusiu del context, citacions *inline* obligatòries i resposta en el mateix idioma de la consulta.
- **Capa API REST (FastAPI):** Arquitectura amb 7 endpoints documentats i una gestió d'excepcions que tradueix els errors de domini en codis d'estat HTTP estructurats de forma determinista.
- **Frontend Streamlit:** Interfície d'usuari completa que inclou selector d'identitat per a proves, panell d'ingesta editable, visualització de cites, telemetria d'auditoria i avís legal d'IA.
- **Sistema d'identitat extensible:** Basat en una *fixture* JSON amb 8 usuaris demo (7 departaments × 4 nivells de `clearance`).
- **Desplegament Docker** multimodal de la solució.

2. Objectius del Projecte

L'objectiu principal d'aquest projecte és dissenyar i desenvolupar un sistema de recuperació d'informació corporativa basat en intel·ligència artificial (arquitectura RAG) que democratitzi l'accés al coneixement intern, garantint la privacitat, la sobirania de les dades i l'aplicació estricta de polítiques de control d'accés (RBAC) intentant augmentar alhora la productivitat del model de llenguatge.

A diferència d'un projecte exclusivament orientat al desenvolupament d'una aplicació final, aquest Treball de Fi de Grau posa el valor en la investigació, l'experimentació empírica i el disseny arquitectònic previ a la implementació. Es busca demostrar com la presa de decisions a baix nivell en les fases de processament de dades impacta directament en la seguretat, productivitat i viabilitat del producte final.

Per assolir aquest propòsit general, es defineixen els següents objectius específics:

1. Investigació i Disseny Algorítmic en la Ingesta de Dades:

Avaluar i dissenyar estratègies de fragmentació de text (*chunking*) que permetin identificar i aïllar informació sensible. L'objectiu és superar les limitacions dels mètodes estàndard, assegurant una granularitat de permisos a nivell de fragment sense destruir el context semàntic necessari per al model de llenguatge.

2. Disseny d'una Arquitectura de Recuperació Híbrida Segura i Eficax:

Analitzar les millors tècniques de filtratge i resoldre les vulnerabilitats inherents que es presentin. L'objectiu és dissenyar un *pipeline* de recuperació híbrid que integri de manera nativa la lògica de rols i departaments (RBAC). Això garantirà que cap dada confidencial sigui exposada a un usuari sense autorització –independentment del motor de cerca utilitzat– i que no es perdi el valor semàntic de la informació. A més, dissenyar un sistema multiagent encaminador (*router*) que esculli de manera òptima a quines col·leccions de la base de dades vectorial accedir per guardar o recuperar les dades.

3. Garantir la Sobirania de les Dades:

Projectar una arquitectura 100% *on-premise* recolzada exclusivament en tecnologies de codi obert i models d'inferència locals. Intentant eliminar la dependència d'APIs externes per mitigar completament el risc de fuites de propietat intel·lectual.

4. Transparència i Mitigació d'Al·lucinacions:

Assegurar que les respostes generades pel sistema es basin estrictament en la documentació interna recuperada. El sistema haurà de ser auditable, citant de manera transparent les fonts i fragments específics utilitzats per generar cada resposta, permetent verificar la informació a l'instant.

5. Desenvolupament d'un Prototip Funcional i Avaluable:

Implementar una plataforma de programari que orquestri eficientment tots els components dissenyats (ingesta, recuperació, agent d'enrutament i inferència LLM) darrere d'una interfície adaptable i intuïtiva per a usuaris corporatius, permetent validar els resultats teòrics en un entorn d'ús pràctic.

3. Planificació

	Tasques del projecte				Març 2026					Abril 2026					Maig 2026					Juny 2026	
					S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15		
Fase	Tasca	Inici	Fi	Dies	23/2-1/3	2-8/3	9-15/3	16-22/3	23-29/3	30/3-5/4	6-12/4	13-19/4	20-26/4	27/4-3/5	4-10/5	11-17/5	18-24/5	25-31/5	1-7/6		
0. Estudi previ	Estat de l'art (RAG, RBAC, Zero-Trust)	01/03	07/03	7																	
0. Estudi previ	Definició d'abast + requisits	05/03	10/03	6																	
0. Estudi previ	Anàlisi d'alternatives (LLM, vector DB)	09/03	14/03	6																	
0. Estudi previ	Setup d'entorn + repositori inicial	12/03	15/03	4																	
1. Fase 1 – Ingesta	Notebook 01 – càrrega multiformat	16/03	21/03	6																	
1. Fase 1 – Ingesta	Notebook 02 – estratègies de chunking	22/03	29/03	8																	
1. Fase 1 – Ingesta	Notebook 03 – PII + metadades RBAC	28/03	05/04	9																	
2. Fase 2 – Retrieval	Pas 1 – Baseline retriever	06/04	10/04	5																	
2. Fase 2	Pas 2 – RRF dinàmic	10/04	16/04	7																	
2. Fase 2	Pas 3 – Reconstrucció pare-fill	15/04	21/04	7																	
2. Fase 2	Pas 4 – Fusió asimètrica	20/04	27/04	8																	
2. Fase 2	Pas 5 – Mètriques + LLM productivity	26/04	01/05	6																	
2. Fase 2	Augment corpus + re-anàlisi	30/04	06/05	7																	
3. Fase 3 – Routing	Pas 1 – Classificador LLM metadades	04/05	08/05	5																	
3. Fase 3 – Routing	Pas 2 – Agent ingesta + KSP Router-Index	07/05	11/05	5																	
3. Fase 3 – Routing	Pas 3 – Desqualif. baseline estàndard	10/05	13/05	4																	
3. Fase 3 – Routing	Pas 4 – Arquitectura jeràrquica final	12/05	15/05	4																	
4. Implementació	Capa Core (config, exceptions, security)	13/05	15/05	3																	
4. Implementació	Infrastructure (Chroma, BM25, Ollama, ...)	15/05	19/05	5																	
4. Implementació	Domain – Chunking + cross-dept rules	17/05	20/05	4																	
4. Implementació	Domain – Retrieval (RBAC + RRF + ensemble)	19/05	22/05	4																	
4. Implementació	Domain – Routing (intent + KSP + hierarch.)	21/05	24/05	4																	
4. Implementació	Users + Application (Ingestion + Query)	23/05	26/05	4																	
4. Implementació	API FastAPI + Frontend Streamlit	25/05	28/05	4																	
4. Implementació	Dockerització (compose + 2 Dockerfile)	27/05	29/05	3																	
5. Validació	Tests unitaris de primitives	22/05	26/05	5																	
5. Validació	Tests E2E ingestió + query	26/05	29/05	4																	
5. Validació	Tests del stack Docker complet	29/05	01/06	4																	
5. Validació	Scripts d'inspecció + auditoria	01/06	02/06	2																	
6. Documentació	Redacció Disseny (Fases 1-3)	18/05	27/05	10																	
6. Documentació	Redacció Implementació + diagrames	25/05	01/06	8																	
6. Documentació	Avaluació + Conclusions + Annexos	30/05	03/06	5																	
6. Documentació	Revisió final i correccions	03/06	04/06	2																	
7. Lliurament	★ Entrega memòria final del TFG	05/06	05/06	1															★		

Figura 1 - Diagrama de Gantt desenvolupament projecte

4. Requisits

Aquest apartat exposa els requisits del sistema estructurats en dues categories principals: requisits funcionals (RF) i no funcionals (RNF). Com a punt de partida per a la seva definició i enumeració, es proporciona un diagrama de casos d'ús que mostra les interaccions fonamentals de l'aplicació i actua com a base de referència metodològica.

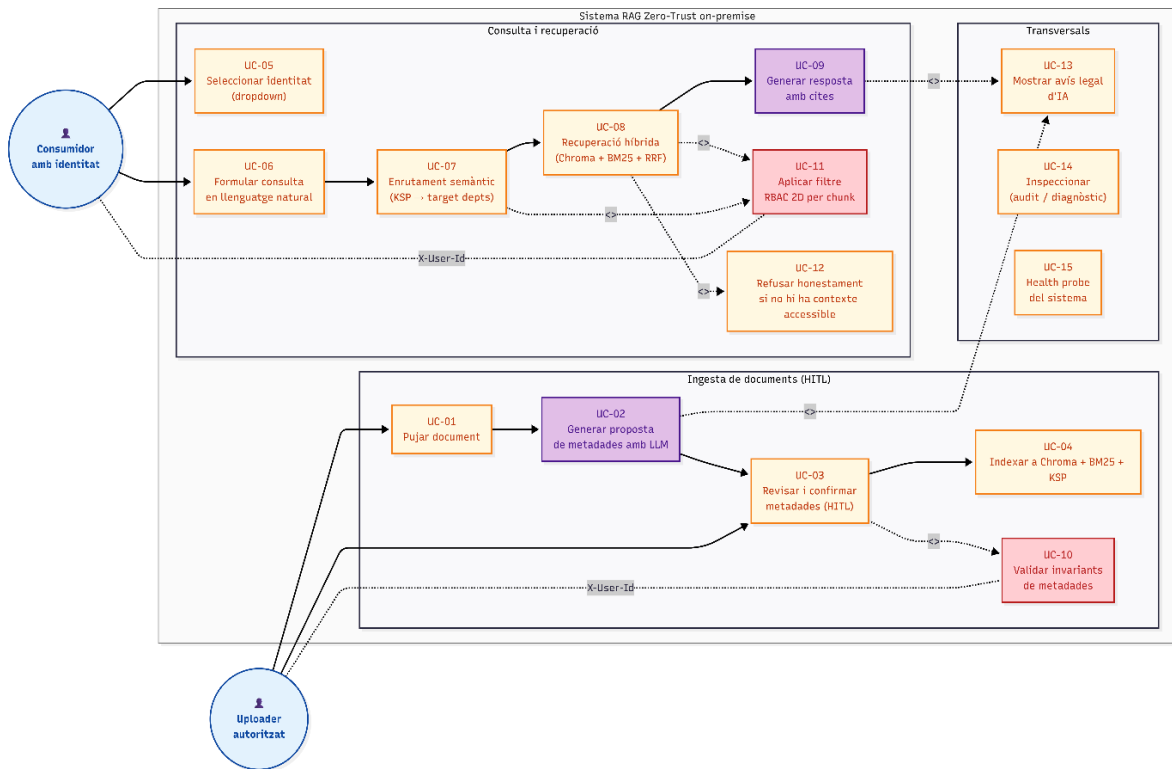


Figura 2 - Diagrama casos d'ús

Groc: Cassos d'ús funcionals

Lila: Cassos d'ús d'invocació al LLM

Vermell: Cassos d'ús amb RBAC aplicada

4.1.1. Requisits Funcionals (RF)

Per estructurar tots els requisits funcionals s'han separat en quatre àrees diferents:

A. Ingesta i Indexació de Documents:

- **RF-01-Càrrega multi-format:** El sistema ha d'acceptar documents de diferents formats i rebutjar la resta de formats no suportats.
- **RF-02-Classificació automàtica de metadades:** Un LLM ha de proposar els principals camps comuns per a cada *chunk* a partir del document del que provenen.
- **RF-03-Revisió HITL:** Cap document ni els seus fragments es persisteix sense un control humà sobre els camps inferits per l'LLM del usuari que vol pujar el document.
- **RF-04-Aïllament de PII per *chunk*:** El sistema ha de detectar automàticament patrons PII i aïllar-los en fragments separats amb un nivell de confidencialitat major (si és possible)

- **RF-05-Extensió individual dels departaments d'accés:** S'ha de poder determinar si és necessari afegir algun departament més a la llista de departaments que poden accedir a aquell fragment.
- **RF-06-Persistència idempotent:** La re-ingesta d'un mateix document ha de convergir al mateix estat sense duplicacions.
- **RF-07-Validació d'invariants de metadades:** El sistema ha de rebutjar combinacions invàlides o falta de metadades.

B. Identitat i Control d'Accés:

- **RF-08-Selecció d'identitat:** L'usuari final ha de poder identificar-se d'alguna manera i que el sistema verifiqui aquesta.
- **RF-09-Predicat RBAC:** Per cada *chunk* candidat, el sistema ha d'avaluar si l'usuari té suficients permisos per accedir-hi.
- **RF-10-L'Aplicació del RBAC:** Ha de garantir que no hi hagin fugues d'informació i que n'hi ha candidats suficients (fragments).

C. Consulta, Recuperació i Generació:

- **RF-11-Routing semàntic:** El sistema ha de saber separar la intenció d'una consulta per saber a quin motor de cerca donar-li més prioritat.
- **RF-12-Recuperació Híbrida Asimètrica:** Per cada departament seleccionat com a destí de consulta, s'ha d'executar de manera paral·lela la cerca vectorial i lèxica de fragments.
- **RF-13-Generació amb format:** L'LLM ha de respondre només a partir del context recuperat i citar els fitxers utilitzats si escau.
- **RF-14-Denegació:** Si ni el *router* ni el *retriever* retornen cap valor accessible en les seves llistes, el sistema ha de retornar una resposta de denegació d'accés.
- **RF-15-Telemetria:** Cada resposta ha d'incloure informació sobre els paràmetres escollits, *chunks* utilitzats per a generar la resposta, latències...

D. Compliment, Observabilitat i Operació:

- **RF-16-Avis legal d'IA:** La UI ha de mostrar un avis d'IA generativa visible per als usuaris per garantir el compliment legal.
- **RF-17-Prova d'estat:** S'ha de poder comprovar l'estat dels components del sistema.
- **RF-18-Inspecció dels índexs:** el sistema ha d'oferir consultar el contingut dels índexs *Chroma* i *BM25* amb la mateixa instància *d'adapters* que utilitza el *retriever* `scripts/inspect_stores.py`.

4.1.2. Requisits No Funcionals (RNF)

Aquests requisits defineixen les restriccions d'arquitectura, qualitat i operació que la solució ha de complir sigui quina sigui la implementació concreta dels casos d'ús anteriors. Per estructurar-los s'han separat en sis àrees diferents:

A. Confidencialitat:

- **RNF-01-LLM en execució local:** La inferència del model generatiu i del model *d'embeddings* ha de fer-se 100% dins el perímetre del sistema, sense crides a APIs *cloud* externes per cap motor.
- **RNF-02-Cap dependència d'APIs externes:** Després del primer arrencament, el sistema ha de poder funcionar *offline* indefinidament.

- **RNF-03-Aïllament de PII:** Cap *chunk* amb dades personals identificables pot sortir del perímetre amb un `clearance_level` inferior al del contingut més sensible que conté.
- **RNF-04-Default Deny:** Davant metadada corrupta, fitxer no parsejable, *mismatch d'embedding* model o invariant violat, el sistema ha de denegar la operació amb un error explícit.

B. Desplegament:

- **RNF-05-Desplegament reproducible:** La solució ha de poder aixecar-se sobre qualsevol màquina amb o sense *Docker Desktop*, per a un desplegament productiu i un altre de desenvolupament.
- **RNF-06-Persistència local inspeccionable:** L'estat (índexs *Chroma*, *BM25*, models *Ollama*, *fixture* d'usuaris) ha de viure en *bind-mounts* al host perquè sigui inspeccionable amb eines del sistema operatiu.

C. Mantenibilitat:

- **RNF-07-Separació de responsabilitats:** El codi s'ha d'estructurar per capes amb dependències unidireccionals.
- **RNF-08-Inversió de dependència:** Cada component ha de ser substituïble mitjançant un abstracció sense modificar altres capes.
- **RNF-09-Seguretat sobre errors de tipus:** S'han d'usar *enums* i constants allà on sigui necessari per garantir l'ús de l'estructura dissenyada.
- **RNF-10-Configuració centralitzada:** Tota la configuració ha de trobar-se a una classe *Settings* centralitzada accessible per a tots els mòduls.

D. Observabilitat:

- **RNF-11-Telemetria estructurada en cada consulta:** Cada resposta ha de portar mètriques analitzables.
- **RNF-12-Auditoria de decisions:** Cada *chunk* persistit ha de portar metadades de procedència que permetin reconstruir per què el sistema l'ha o no l'ha mostrat a un usuari concret.
- **RNF-13-Healthchecks per servei:** S'ha de poder comprovar l'estat de cada servei amb scripts executables i implementar un sistema de dependències sobre aquests abans d'arrencar els serveis.

E. Regulacions:

- **RNF-14-Transparència d'IA (EU AI Act art. 50):** L'usuari final ha de saber en tot moment que interactua amb IA generativa.
- **RNF-15-Supervisió humana (EU AI Act art. 14):** Cap decisió rellevant del sistema (assignació de metadades, d'accés) pot prendre's exclusivament per IA sense revisió humana.

RF	Cassos d'ús implicats	RF	Cassos d'ús implicats
RF-01, RF-02, RF-07, RF-09	UC-01, UC-02, UC-10	RF-11	UC-07
RF-03, RF-08	UC-03, UC-05	RF-12	UC-08
RF-04, RF-05, RF-06	UC-04	RF-13	UC-09
RF-09, RF-10	UC-11	RF-14	UC-12
RF-15, RF-18	UC-08, UC-14, UC-15	RF-16	UC-13

Taula 1 - Relació requisits funcionals - cassos d'ús

5. Estat de la Tècnica

El disseny d'un sistema RAG en un entorn corporatiu format per un agent selector, un *Ensemble Retriever* i tot un subsistema d'assignació i filtratge de permisos, exigeix que els components sobre els quals es desenvolupa la solució compleixin amb uns requisits estrictes d'arquitectura, seguretat i flexibilitat que s'adaptin el més possible a les necessitats del projecte, amb especial èmfasi en la sobirania de la informació i la prevenció de fuites de dades internes. En aquest sentit, el present capítol analitza l'estat actual de les tecnologies implicades en aquest procés –centrant-se principalment en el motor de bases de dades vectorials, el *framework* d'integració i model d'*embeddings* i LLM– per tal d'avaluar les diferents alternatives ja existents i justificar les eines seleccionades.

5.1. Anàlisi d'Alternatives – Bases de Dades Vectorials

En primer lloc, donat el problema principal de soberania i fuga de dades, la solució ha de ser desplegable en un entorn On-Premise (local). Per aquest mateix motiu, les plataformes natives Cloud o gestionades com a servei SaaS –com Pinecone o les versions Cloud de Weaviate o Qdrant i d'altres– s'exclouen de l'anàlisi.

A més, la base de dades vectorial seleccionada ha de satisfer tres requisits funcionals crítics derivats de l'arquitectura del projecte:

1. **Suport multi-col·lecció i encaminament personalitzat:** Capacitat per particionar les dades de manera lògica i independent, permetent el disseny d'un agent enrutador propi.
2. **Gestió i filtratge per metadades:** El nucli de seguretat del projecte rau en un control d'accés basat en metadades assignades en dues fases (global i local). La base de dades hauria d'oferir mecanismes de filtratge natiu o pròpies aplicables sobre camps de metadades (*where clauses*).
3. **Flexibilitat i compatibilitat d'integració:** Facilitat d'integració amb llibreries d'orquestració com LangChain per suportar tota la canonada lògica, des de l'agent enrutador fins a l'*Ensemble Retriever*.

5.1.1. FAISS

FAISS (Facebook AI Similarity Search) és una biblioteca desenvolupada per l'empresa Meta. És una eina de baix nivell que té com a punt fort una cerca per similitud altament optimitzada en grans conjunts de dades. El seu objectiu principal és donar solució al problema ANN (*Approximate Nearest Neighbor Search*).

Aportació al projecte: Pot destacar a l'hora de realitzar cerques complexes en grans volums de dades i assegura un possible desplegament en un entorn totalment privat/local.

Per què s'ha descartat?

Tot i la seva velocitat d'execució, la impossibilitat de realitzar un pre-filtratge natiu basat en metadades impedeix implementar de forma eficient el sistema de control d'accés. El seu ús requeriria programar un post-filtratge manual a la capa lògica de l'aplicació, la qual cosa resulta ineficient i arquitectònicament fràgil per a un sistema centrat en la seguretat.

5.1.2. Milvus

Milvus és una base de dades vectorial de codi obert enfocada a empreses, és a dir, dissenyada per treballar amb grans volums de dades en entorns distribuïts, proporcionant una infraestructura molt més extensa que solucions menys completes com pot ser ara FAISS. Està originalment pensada per ser una solució Cloud tot i que permet el desplegament *on-premise*.

Aportació al projecte: Permetria un tractament de dades molt eficaç i eficient. Proporciona necessitats crítiques pel projecte com filtratge i separació per col·leccions per l'agent enrutador.

Per què s'ha descartat?

L'alta complexitat d'infraestructura que exigeix està sobredimensionada per als objectius d'aquest Treball de Fi de Grau. El temps requerit per a la configuració, el desplegament i el manteniment d'una arquitectura de microserveis restaria recursos fonamentals a la recerca i desenvolupament algorítmic de la solució.

5.1.3. Qdrant

Qdrant és també una BBDD vectorial i de codi obert amb motor de cerca per similitud i s'especialitza en emmagatzemar, gestionar i recuperar *embeddings* en grans dimensions. Està escrit en RUST, cosa que la fa destacar pel seu rendiment i seguretat.

Aportació al projecte: La bona gestió de metadades (*payloads*) que proporciona simplifica i optimitza considerablement el treball d'injecció de permisos i el filtratge vectorial requerit per al sistema de seguretat.

Per què s'ha descartat?

Tot i ser una opció tecnològicament impecable per a la gestió de permisos mitjançant metadades, la necessitat de gestionar contenidors externs afegeix una fricció innecessària pel desenvolupament de l'anàlisi del projecte. S'ha prioritzat una eina que permeti un prototipatge més àgil i integrat directament en l'entorn de desenvolupament.

5.1.4. ChromaDB

ChromaDB és una BBDD vectorial *open-source* i poc pesada orientada a intentar simplificar la creació d'app's impulsades pels LLM. Es centra en les funcionalitats essencials per a la gestió de context, incloent una gestió nativa de col·leccions i un potent motor prefiltratge.

Aportació al projecte: Ofereix les eines necessàries per desenvolupar les parts del producte desitjat i usar-la ràpidament en un entorn de desenvolupament de codi. És àmpliament usada

per a projectes similars i en ser lleugera permet un desplegament ràpid permetent centrar-se en els apartats crítics a tractar.

5.1.5. Conclusió i Justificació

Decisió: S'ha seleccionat ChromaDB com a motor vectorial base per al desenvolupament d'aquest projecte.

L'elecció de ChromaDB no respon a una preferència sintàctica, sinó a una anàlisi rigorosa de les restriccions tècniques del projecte davant del temps disponible per al Treball de Fi de Grau.

L'èxit d'aquest projecte no es mesura per la capacitat d'emmagatzemar milers de milions de vectors (escenari on guanyaria Milvus), ni per l'optimització de maquinari a molt baix nivell (territori de FAISS). El valor acadèmic i tècnic d'aquest treball està en demostrar la viabilitat de lògiques de programari complexes: un *middleware* de seguretat basat en metadades, un agent enrutador de col·leccions propi, un *Ensemble Retriever* dissenyat per mitigar les al·lucinacions... .

En eliminar la fricción operativa de mantener servidores externos i contenidors locals gràcies a la seva naturalesa *embedded*, ChromaDB permet destinar el 100% dels recursos temporals del projecte a la investigació i desenvolupament de components el més eficients i eficaços possibles, sempre ajustats a les necessitats. Tot això, assegurant simultàniament el compliment del requisit innegociable de sobirania de les dades corporatives (*arquitectura 100% on-premise*) i recolzat, com es pot veure als següents gràfics per l'idea que cada cop s'utilitzen BBDD vectorials per a projectes RAG i que ChromaDB és de les més utilitzades.

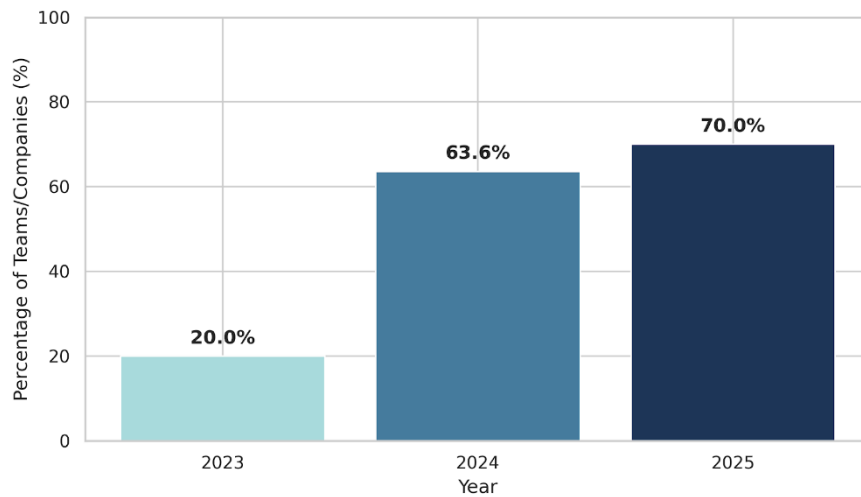


Figura 3 - Adopció de BBDD vectorials en IA/RAG empresarial

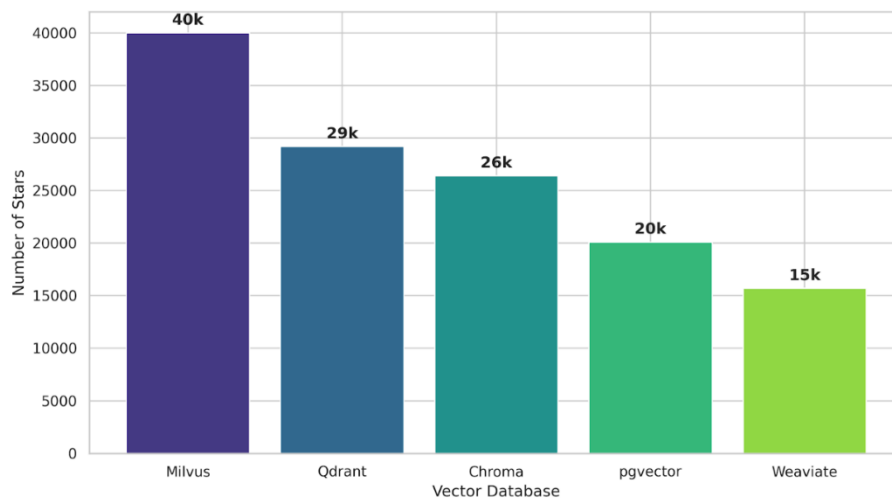


Figura 4 - Popularitat en Github Stars

5.2. Anàlisis d'Alternatives – Frameworks d'Integració RAG

Es vol utilitzar un *framework* capaç de cobrir les següents cinc necessitats:

1. Ha de tenir una interfície estable que suporti el treball i comunicació amb LLMs locals (Ollama).
2. Capacitat per a implementar funcions pròpies que treballin sobre el sistema –com ara `apply_zero_trust_filter` o `calculate_dynamic_rrf`– sense haver d'estendre classes pesades del *framework* ni complicar molt el procés.
3. Capacitat de poder controlar cada pas del pipeline, ja que els components seran propis i no pre-implementats, es vol tenir total control d'aquests i que no “s'amagui” la seva implementació.
4. Robustesa i comunitat sostenible.

5.2.1. Alternatives valorades

1. **LangChain (i LangGraph)**: És la referencia del sector de sistemes que segueixen un *pipeline* i ofereix poder fer integracions natives per a *ChromaDB*, *BM25*, *Ollama*, models d'*embeddings*, i sobretot un patró d'*EnsembleRetriever* propi. Té una comunitat activa i resolutiva de problemes via Github.
2. **LlamaIndex**: És una solució més orientada a RAG pur. Proporciona abstraccions per a poder implementar indexació i filtratge però és menys flexible (no es tant personalitzable en quant a components propis).
3. **Haystack**: És més robust a escala i té una API molt clara però la integració amb Ollama no es tant directa com en LangChain.

5.2.2. Decisió

S'ha optat per `langchain-core` + `langchain-huggingface` com a primitives mínimes (no `langchain` complet) amb la resta d'integracions fetes a mà via adapters propis (`infrastructure/vector_db.py`, `infrastructure/lexical_db.py`, `infrastructure/llm_client.py`, `infrastructure/embedder.py`).

`langchain-core` s'ha utilitzat explícitament per a definir el format de document (`langchain_core.documents.Document`) portable entre layers (*chunker* → *vector store* → *BM25*). `langchain-huggingface` per a abstraure la forma dels vectors entre el sistema i el propi model per a *embeddings*.

S'ha intentat evitar el *framework* complert pel simple motiu de poder desacoblar el sistema dels canvis de l'API.

5.3. Anàlisi d'Alternatives – Models de Llenguatge Generatiu LLMs

Els models de llenguatge generatiu (LLM) constitueixen el component encarregat de produir la resposta final a una arquitectura RAG, a partir del context recuperat pel sistema.

En un sistema corporatiu sota el paradigma *Zero-Trust* com aquest, l'ús d'APIs externes (com les d'OpenAI) queda descartat, ja que enviaria els *chunks* d'informació confidencial recuperats fora de la xarxa local. És per això que les úniques opcions a considerar han de ser LLM executats en format *On-Premise*.

Aquesta decisió produeix restriccions i característiques significatives a tenir en compte a l'hora d'escollir el model del sistema, on la decisió no es basa només en les seves capacitats cognitives teòriques, sinó en un equilibri (*trade-off*) estricte entre el maquinari disponible, l'eficiència computacional i el rendiment requerit.

En aquest apartat es revisen algunes de les principals opcions disponibles actualment, comparant les característiques detallades anteriorment. També es considera el model escollit pel projecte, **Llama-3.2-3B-Instruct**.

5.3.1. Context Teòric i Restriccions

A continuació definire els conceptes tècnics que limiten l'execució d'un LLM en maquinari que donaran suport a la l'elecció arquitectònica final.

- **VRAM:** És la memòria de la targeta gràfica (GPU). Per aconseguir velocitats d'inferència viables en producció (més de 20 *tokens* per segon), la totalitat dels pesos matemàtics del model ha de cabre en aquesta memòria.
- **KV Cache (Key-Value Cache):** És la memòria viva que l'LLM necessita per emmagatzemar el context. En un sistema RAG, on s'injecten múltiples *chunks* de text al *prompt* inicial, el KV Cache pot ocupar fàcilment entre 0,5 GB i 1,5 GB addicionals de VRAM.
- **Quantització i format GGUF:** Els models originals s'entrenen amb una precisió de 16 bits (*FP16*). La quantització és una tècnica matemàtica que redueix aquesta precisió a 8, 4 o fins i tot 2 bits. Estudis de referència en el sector, com el treball *QLoRA: Efficient Finetuning of Quantized LLMs de Dettmers (2023)*, han demostrat que la quantització a 4 bits sacrifica una fracció mínima de precisió lingüística (generalment entre 1-3%) i arriben a afirmar optimistament que no existeix degradació de rendiment (pel seu *finetuning*) mentre permet reduir la mida del model fins a un 75% (passant de 16 a 4 bits per paràmetre).

Aquest procés s'empaqueta habitualment en el format GGUF (*GPT-Generated Unified Format*) optimitzat per a CPU/GPU. Aquest format, introduït per l'equip de *llama.cpp* com a successor del format *GGML*, és un contenidor de fitxers binari dissenyat per a una inferència ràpida i eficient en maquinari local, facilitant l'execució de models potents en ordinadors amb memòria *VRAM* limitada.

- **Offloading:** Si un model excedeix la capacitat de la VRAM, les capes sobrants s'envien a la RAM del sistema (CPU). Això genera un *bottleneck* (coll d'ampolla) en el bus de dades, desplomant la velocitat de generació a nivells inacceptables per a l'usuari. És per això que es vol evitar sobrepassar la capacitat de VRAM disponible, per tenir un entorn de prova el més real, eficaç i eficient possible.

5.3.2. Anàlisi de Maquinari i Pressupost

L'entorn de desplegament d'aquest projecte consta d'una CPU *Intel Core i5-9600K*, 16 GB de RAM de sistema i una GPU *Nvidia GeForce GTX 1660 Ti* amb **6 GB de VRAM**.

Realitzant el “pressupost de memòria” gràfica, es dedueix 1 GB per al sistema operatiu (Windows), 0,5 GB per mantenir en actiu el model *d'embeddings* (*all-MiniLM-L6-v2* – per a la indexació i el *retrieval* de ChromaDB) i 1 GB per al KV Cache del context RAG. **Això deixa un màxim estricte d'uns 3,5 GB de VRAM disponibles pel LLM.**

*** Les quantitats en GB anterior son aproximades.**

5.3.3. Comparativa de Models i Elecció de la Solució Òptima

Per a aquest entorn, s'han avaluat dues alternatives principals de codi obert (*open-weights*):

1. **Mistral-7B-Instruct:** Juntament amb Llama-3-8B-Instruct són els estàndards actuals de la indústria per a sistemes locals. Tenen capacitats de raonament molt bones. No obstant això, fins i tot aplicant una quantització de 4 bits, el seu pes ronda els 4,5- 5 GB d'VRAM. És per aquest motiu que descartem l'opció perquè l'execució d'aquest a la màquina proposada provocaria un *offloading* a la RAM, arruïnant l'eficiència del sistema i la latència de la resposta que, tot i no ser el principal objectiu del projecte, es considera suficientment important per garantir un entorn de prova òptim.
2. **Llama-3.2-3B-Instruct:** Aquest model de 3 bilions de paràmetres, és de Meta i està optimitzat també amb una quantització de 4 bits però el seu pes ocupa només entorn als 2GB de VRAM, per tant, la considerem entre d'altres que també s'han tingut en compte, la solució òptima ja que cap completament a la memòria de la GPU deixant marge pel KV Cache i prevenir així un possible *bottleneck*.

L'elecció de Llama-3.2-3B no representa l'opció *OnPremise* més potent del mercat, però sí la decisió òptima per a aquest propòsit. Compta amb una llicència altament permissiva (*Llama 3.1 License*), s'integra perfectament en *pipelines* RAG (com *LangChain*) i la seva eficiència permet resoldre els problemes del RAG corporatiu de manera fluida dins les restriccions imposades pel maquinari disponible.

***Pel desplegament del projecte, s'utilitzarà el gestor de models Ollama per facilitar la configuració i gestió dels components.**

5.3.4. Projectió a l'Entorn Real (Escalabilitat)

En un entorn de producció d'una empresa real –la qual disposaria de servidors dedicats amb GPUs de gamma alta o almenys amb més capacitat que la que es disposa per la prova d'aquest projecte– l'enfocament proposaria l'ús de models més potents per millorar la qualitat de la generació, com per exemple Llama-3.1-70B-Instruct, mantenint les garanties de seguretat d'un model *OnPremise*.

L'avantatge principal del *pipeline* desenvolupat en aquest TFG és la seva facilitat d'integració i escalabilitat. El canvi del model optimitzat actual (Llama 3.2 de 3B) a una opció corporativa de 70B requeriria, a nivell de codi base, la modificació d'un mòdul, ja que la solució segueix una arquitectura modular i *clean* que garanteix la separació de responsabilitats entre components. D'aquesta manera, es demostra que el sistema no només funciona dins les limitacions actuals, sinó que està completament preparat per a una integració empresarial escalable a nivell d'infraestructura.

5.4. Anàlisi d'Alternatives – Models d'*Embedding*

El model d'*embeddings* actua com el nucli semàntic del sistema. Pot arribar fins i tot a ser un coll d'ampolla, ja que d'ell depèn la precisió amb què s'avalua la proximitat entre les consultes de l'usuari i els documents. És per això que l'elecció ha de garantir **suport multilingüe creuat** (anglès, castellà i català), que sigui **computacionalment eficient** per executar-se en CPU (vectors compactes ≤ 768 dimensions per optimitzar *ChromaDB*), **llicència de codi obert a ser possible**, ja que el canvi de incrustacions de tot una font d'informació no seria tant senzill com el del model LLM i finalment **compatibilitat amb HuggingFace de LangChain**.

5.4.1. Anàlisi d'Alternatives

Durant el disseny, es van avaluar diverses opcions:

1. ***all-MiniLM-L6-v2***: Tot i ser ràpid, lleuger i oferir una gran qualitat en les fases inicials, va ser descartat en detectar-se una fallada crítica en cerques multilingües (no va ser capaç de relacionar la consulta en anglès "*warranty period*" amb la clàusula en castellà "*garantia 24 mesos*").
2. ***bge-m3***: Tot i suportar models multilingües, es va descartar per que encapsula en 1024 dimensions, que son moltes més que les utilitzades en el sistema de ChromaDB i és massa pesat (2GB).

5.4.2. Decisió Final

S'ha seleccionat oficialment *paraphrase-multilingual-MiniLM-L12-v2* per a la versió de producció, per que proporciona un servei multilingüe, manté la mateixa dimensió vectorial (384) que el model inicial que s'estava utilitzant abans de detectar l'error multilingüe entre consultes i opera sota la llicència *Apache 2.0*.

5.5. Anàlisi d'Alternatives – Motors d'Inferència Local

El Model de Llenguatge Extens (LLM) actua com l'actiu principal de la solució. Durant la fase d'ingesta actua com a classificador analític de documents i durant la fase de consulta actua com a motor de generació de la resposta en llenguatge natural basada en el context proporcionat de la recuperació.

L'únic requisit fonamental i innegociable d'aquesta arquitectura corporativa és l'execució 100% *On-Premise*. Per garantir el compliment absolut del paradigma *Zero-Trust*, cap fragment de document intern ni cap consulta d'usuari pot transmetre's a APIs externes o serveis al *cloud* (com *OpenAI GPT-4* o *Anthropic Claude*).

5.5.1. Anàlisi d'Alternatives Locals

Es van avaluar varies solucions per servir el model de llenguatge localment:

- ***llama.cpp* en brut:** Proporciona un control absolut sobre els recursos computacionals, però exigeix una complexa gestió manual de binaris, quantitzacions de memòria, matrius d'entrada (*prompt templates*) específiques de cada model i validacions JSON de baix nivell, generant massa complexitat per a l'àmbit d'aquest projecte.
- **LM Studio:** Excel·lent entorn de desenvolupament visual d'escriptori, però inadaptable per a un entorn *dockeritzat* desatès i de producció.

5.5.2. Decisió Final

S'ha seleccionat oficialment la plataforma *Ollama* (versió 0.4.2) executant el model *Llama 3.2* com a motor d'inferència de la solució. Aquesta elecció es fonamenta en aquests pilars d'arquitectura:

1. **On-Premise garantit:** *Ollama* es pot executar com un contenidor *Docker* aïllat, descarregant i persistint els pesos del model en el volum local del servidor (*./data/ollama/*) durant la primera arrencada.
2. **Generació restringida per format (JSON):** Suporta nativament l'argument `format="json"` que utilitza el component *LLMClassifier* de la capa d'ingesta per obligar el model a retornar una resposta amb una estructura *parsejable*.
3. **Support multi-idiomes:** Incorpora un entrenament en els idiomes requerits.

6. Disseny

Aquest capítol documenta un procés de validació empírica iterativa sobre els diferents components que formen el sistema –principalment un *Router* de col·leccions, un sistema de *Chunking* i un sistema de *Retrieval* basat en RBAC. Les conclusions i resultats extrets han precedit a la implementació productiva del sistema.

L'objectiu, en comparació amb la fase d'implementació no és arribar a una arquitectura final sinó demostrar amb mètriques que cada decisió clau de disseny –com aïllar les PII a *chunks* separats, com filtrar pel predicat 2D de Zero-Trust abans o després de la fusió, com routar la consulta cap a la col·lecció correcta– resol un problema real abans de usar-la al codi.

El treball s'estructura en tres fases consecutives executades sobre *notebooks Jupyter*: la Fase 1 caracteritza el corpus i compara estratègies de *chunking* amb i sense aïllament RBAC; la Fase 2 construeix el motor de recuperació híbrida asimètrica (ChromaDB + BM25 + RRF) i compara les tècniques possibles i l'ordre de d'aplicació d'aquestes; la Fase 3 introdueix el *routing* jeràrquic semàntic amb una estratègia de *Key Semantic Points* (KSP) per evitar consultes sobre tot el corpus.

Com ja s'ha esmentat, cada fase busca obtenir conclusions arquitectòniques explícites que serviran per definir requisits no negociables del capítol d'Implementació.

A continuació es detalla tot el corpus (documents) utilitzats i creats sobre els que s'han realitzat les proves i analitzats les mètriques de cada apartat. Cal destacar que s'ha utilitzat el document base “*Witty-QuickGuide-EN.pdf* - [15]” a partir del qual s'ha utilitzat IA per generar d'altres relacionats i estendre el corpus de proves.

Fitxer	Categoria	Format	Clearance Level	Department	Contingut sensible
Witty-QuickGuide-EN.pdf	Product Docs (original)	PDF	0 - PUBLIC	all	Email corporatiu, telèfon
Witty-Financial-Report-2025.pdf	Financial Reports (original)	PDF	3 - STRICT	finance	Ingressos, pressupostos I+D
distribution-contract-2026.docx	Client Contracts (original)	DOCX	2 - CONFIDENTIAL	legal	IBAN, SWIFT, nom representant
server_logs_witty_backend.txt	IT Infrastructure (original)	TXT	2 - CONFIDENTIAL	engineering	Contrasenya, IPs internes
clients-and-billings.xlsx	Client Records (original)	XLSX	3 - STRICT	sales	IDs client, emails, factures
fin_budget_forecast_2027.txt	Financial Reports (sintètic)	TXT	2 - CONFIDENTIAL	finance	Xifres pressupostàries 2027, IBAN
fin_expense_policy.md	Financial Reports (sintètic)	MD	1 - INTERNAL	finance	Política de despeses, codis SWIFT
fin_q1_2026_revenue.txt	Financial Reports (sintètic)	TXT	3 - STRICT	finance	Detall d'ingressos Q1 amb IBAN
hr_employee_directory.txt	HR Records (sintètic)	TXT	2 - CONFIDENTIAL	hr	DNI, emails personals, employee IDs
hr_onboarding_guide.md	HR Records (sintètic)	MD	0 - PUBLIC	hr	Procediment onboarding, emails genèrics
hr_salary_bands_2026.txt	HR Records (sintètic)	TXT	3 - STRICT	hr	Bandes salarials 2026, DNI
it_incident_report_IR2026_003.txt	IT Infrastructure (sintètic)	TXT	2 - CONFIDENTIAL	engineering	IPs internes, server names, password

it_server_logs_march_2026.txt	IT Infrastructure (sintètic)	TXT	2 - CONFIDENTIAL	engineering	Logs servidor, IPs, codis d'error
it_vpn_config_guide.md	IT Infrastructure (sintètic)	MD	1 - INTERNAL	engineering	Configuració VPN, password compartit
contract_federacion_atletismo.txt	Client Contracts (sintètic)	TXT	2 - CONFIDENTIAL	legal	IBAN, SWIFT, nom representant client
contract_fitplus_maintenance.txt	Client Contracts (sintètic)	TXT	2 - CONFIDENTIAL	legal	IBAN, SWIFT, dades client
contract_universidad_de_porte.txt	Client Contracts (sintètic)	TXT	2 - CONFIDENTIAL	legal	IBAN, SWIFT, persona contacte
product_photocell_alignment.md	Product Docs (sintètic)	MD	0 - PUBLIC	engineering	Sense PII — manual tècnic públic
product_release_notes_v4.txt	Product Docs (sintètic)	TXT	0 - PUBLIC	engineering	Sense PII — changelog públic
product_witty_timer_specs.md	Product Docs (sintètic)	MD	0 - PUBLIC	engineering	Sense PII — specs producte
memo_board_meeting_minutes.txt	Internal Memos (sintètic)	TXT	3 - STRICT	operations	Emails directius, IDs reunió
memo_it_security_reminder.txt	Internal Memos (sintètic)	TXT	1 - INTERNAL	engineering	Recordatori seguretat, emails IT
memo_q2_sales_targets.txt	Internal Memos (sintètic)	TXT	2 - CONFIDENTIAL	sales	Objectius vendes Q2, noms comercials

Taula 2 - Descripció corpus utilitzat per proves

Llegenda:

Documents utilitzats per tests fins la Fase 1.

Documents afegits per tests a partir de la Fase 2.

En un començament es van començar a realitzar proves amb els 5 primers documents de la taula, però a partir del punt 6.2.6. es va trobar una necessitat d'augmentar el numero de documents per testar necessaris per veure resultats significatius i millores en les mètriques posteriors. És per això que en punt 6.2.10. es van repetir els tests anteriors amb el corpus augmentat, validant les hipòtesis generades anteriorment i en les següents fases s'utilitza el corpus al complet.

A continuació es detalla en un diagrama d'estats tots els passos que el sistema resol i que s'apliquen, estudien i verifiquen en aquest apartat.

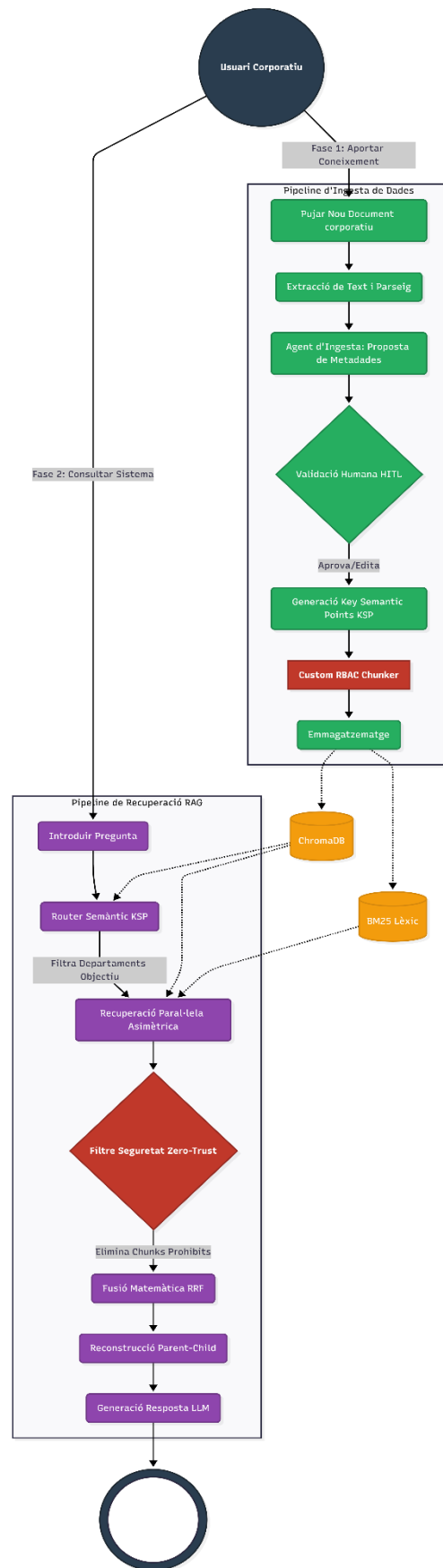


Figura 5 - Diagrama d'estats del sistema

6.1. Fase 1 – Ingesta: Estratègies de *Chunking* i Assignació de Metadades RBAC

6.1.1. Plantejament del Problema

El primer pas en qualsevol sistema RAG és la ingesta: el procés de transformar documents en brut en fragments indexables que un motor de cerca pugui recuperar posteriorment. La qualitat d'aquesta fragmentació determina un límit superior a la qualitat de tot el sistema –si un fragment rellevant no existeix com a unitat recuperable, cap motor de cerca, per sofisticat que sigui, el podrà retornar.

En un sistema RAG convencional, la fragmentació es planteja exclusivament com un problema d'optimització de la mida: trobar l'equilibri entre fragments prou grans per preservar el context semàntic i prou petits per evitar diluir la informació rellevant amb contingut irrellevant. Les estratègies estàndard –*fixed-length chunking*, *recursive character splitting*, *semantic chunking*– operen sobre aquesta dimensió única.

No obstant això, quan el sistema RAG opera en un entorn corporatiu amb requisits de confidencialitat, la fragmentació adquireix una segona dimensió: **la seguretat**. Considerem el cas següent, extret del corpus de test del projecte per la fase 1:

Un contracte de distribució (classificat com a `confidential`, nivell 2) conté, dins d'una clàusula de condicions de pagament, un número IBAN i un codi SWIFT que requereixen un nivell d'accés `strict` (nivell 3). Si el fragment que conté aquesta clàusula es processa com una unitat indivisible, heretarà el nivell de `clearance` 2 del document pare. Un usuari amb `clearance` 2 podrà accedir a tot el fragment, incloses les dades bancàries que requereixen `clearance` 3.

Figura 6 - Exemple del problema de *Chunking*

Aquest escenari il·lustra un problema que cap estratègia de *chunking* convencional resol: la granularitat de seguretat dins d'un document no coincideix amb la granularitat del document sencer. La pregunta que guia aquesta fase és, doncs: *Quina estratègia de fragmentació aïlla millor la informació sensible per garantir que el pipeline de recuperació mai filtri dades a usuaris sense autorització?*

Per respondre-la, s'ha dissenyat un experiment comparatiu amb quatre estratègies de *chunking* aplicades sobre un corpus multiformat controlat.

6.1.2. Entorn d'Experimentació

L'experimentació es realitza en tres *Jupyter Notebooks* seqüencials, cadascun dels quals implementa un pas del *pipeline* d'ingesta i genera les dades d'entrada per al pas següent:

Arxiu	Pas del Pipeline Intern	Descripció
01_document_loading .ipynb	1: Carrega + Metadada Global	Càrrega multiformat (PDF, DOCX, TXT, XLSX) i assignació de metadades RBAC globals a cada document.

02_chunking_strategies.ipynb	2: <i>Chunking</i> Comparatiu	Implementa i executa 4 estratègies de chunking (fix, estructural, semantic, Custom RBAC) sobre els 5 documents. Recopila mètriques quantitatives per estratègia i document.
03_local_metadata_rbac.ipynb	3: Metadada Local + Validació RBAC	Assigna metadades locals (contains_PII, sensitivity_types) a cada chunk i executa la validació de seguretat RBAC per detectar violacions.

Taula 3 - Descripció Arxius pel Disseny de Chunking

El corpus de test es compon dels primers 5 documents llistats a la “Taula 2 - Descripció corpus utilitzat per proves” i representen una selecció deliberada de formats, nivells de confidencialitat i tipus de contingut sensible

Aquesta selecció no és arbitrària. Garanteix que:

- Els quatre nivells de `clearance` (0–3) estiguin representats.
- Hi hagi documents amb PII crítica (IBAN, contrasenyes) i documents nets.
- Els formats cobreixin tant dades estructurades (XLSX) com no estructurades (TXT)
- Els departaments assignats siguin suficientment diversos per validar el filtratge departamental.

Les dades intermèdies/resultants generades durant l'experimentació es serialitzen per permetre la reproductibilitat i la traçabilitat entre *notebooks* i fases:

Fitxer	Generat per...	Contingut
loaded_documents.json	Notebook 1	Serialització JSON dels 5 documents carregats amb metadata global. Serveix d'entrada al Notebook 02.
chunk_results.json	Notebook 2	Tots els <i>chunks</i> generats per les 4 estratègies, amb el seu text i metadata. Serveix d'entrada al Notebook 03.
chunking_metrics.csv	Notebook 2	Mètriques quantitatives de cada estratègia per document (num_chunks, avg_size, boundary_coherence, etc.).
rbac_violations.csv	Notebook 3	Registre de violacions RBAC detectades. Detallat a la secció 6.1.6.

security_scorecard.csv	Notebook 3	Taula resum de seguretat per estratègia. Detallat a la secció 6.1.6.
------------------------	------------	--

Taula 4 - Fitxers de sortida obtinguts del estudi de *Chunking*

6.1.3. Càrrega Multiformat i Metadades Globals

Pas 1 del *pipeline* (Notebook 01).

S'implementa un *dispatcher* que, a partir de l'extensió de l'arxiu, enruta el document a la funció de càrrega corresponent:

- **PDF:** `PyPDFLoader` de `LangChain` (un Document per pàgina).
- **DOCX:** `python-docx`, extreu tots els paràgrafs i genera un únic Document.
- **TXT:** Lectura directa de l'arxiu, un únic Document.
- **XLSX:** `pandas.read_excel`, converteix cada full en un Document amb representació textual tabular.

La decisió de disseny clau en aquest pas és la normalització del format de sortida: tots els *loaders* produeixen objectes `langchain.schema.Document` amb un esquema de metadades unificat. Això permet que els passos 2 i 3 del *pipeline* siguin completament agnòstics al format d'origen, eliminant la necessitat de lògica condicional per format en les fases posteriors.

****** Al ser la fase de estudi d'estratègia en el disseny, només es tenen en compte aquests 4 formats.

Un cop carregat cada document, s'hi assignen les metadades globals –aquelles que s'apliquen a la totalitat del document, no a fragments individuals:

- `clearance_level`: Nivell d'accés (0=*public*, 1=*intern*, 2=*confidential*, 3=*strict*).
- `allowed_departments`: Departament(s) amb accés (només s'aplica quan `clearance` $\geq$ 2).
- `doc_type`: Tipus de document (manual, report, contract, log, spreadsheet).
- `global_topic`: Descripció temàtica d'alt nivell.

En aquesta fase d'experimentació, les metadades globals s'assignen manualment a través d'un registre de documents (`DOCUMENT_REGISTRY`). En la versió de producció del sistema, un agent LLM haurà d'inferir automàticament la majoria d'aquests camps a partir del contingut del document. No obstant això, per a l'objectiu d'aquesta fase –avaluar les estratègies de *chunking*– l'assignació manual garanteix un model controlat que elimina la variable d'error del classificador automàtic.

6.1.4. Estratègies de *Chunking* Avaluades

Pas 2 del *pipeline* (Notebook 02).

S'implementen i s'executen quatre estratègies de fragmentació sobre el corpus complet. La selecció d'aquestes quatre estratègies respon a una progressió deliberada: des de l'enfocament més simple i agnòstic al contingut (*fixed*) fins a la proposta pròpia del projecte (*Custom RBAC*), que incorpora consciència de seguretat al procés de fragmentació.

1. *Fixed-Length Chunking* (Fragmentació fixa amb solapament)

Usa `RecursiveCharacterTextSplitter` de `LangChain`. Paràmetres: `chunk_size=500`, `chunk_overlap=100`. Separadors jeràrquics: `\n\n` > `\n` > `.` > `"`. Aquesta és l'estratègia *baseline*: determinista, sense cap coneixement del contingut, i la més comunament emprada en implementacions de referència de sistemes RAG.

La seva inclusió respon a la necessitat de disposar d'un punt de comparació confiat. Si les estratègies més sofisticades no milloren substancialment les mètriques respecte a aquesta *baseline*, la complexitat addicional no estaria justificada.

2. *Structural Chunking* (Fragmentació estructural)

Divideix el text en les fronteres lògiques del contingut: seccions numerades (ex., 1. *Executive Summary*), clàusules contractuals (ex., PRIMERA, SEGUNDA), *timestamps* de registres de servidor (ex., [2026-03-05...), i separadors de fulls d'un full de càlcul. S'implementa mitjançant una llista prioritzada de patrons *regex*, amb un mecanisme de fusió que evita generar fragments massa petits (< 80 caràcters).

L'avantatge teòric d'aquesta estratègia és que respecta les unitats semàntiques naturals del document –un paràgraf contractual, una entrada de *log*, una secció d'un informe– en lloc de tallar arbitràriament a mig paràgraf. La hipòtesi és que aquesta coherència de fronteres millorarà la qualitat de la recuperació posterior.

3. *Semantic Chunking* (Fragmentació semàntica)

Separa el text en oracions individuals, calcula *embeddings* per a cadascuna mitjançant el model local `all-MiniLM-L6-v2`, i agrupa oracions consecutives la similitud cosinus de les quals superi un llindar configurable (`threshold=0.45`). Els grups amb menys de dues oracions es fusionen amb l'adjacent.

Aquesta estratègia és la més sofisticada des del punt de vista algorítmic, ja que utilitza la representació vectorial del contingut per decidir on tallar. No obstant això, introdueix dependència d'un model d'*embeddings* i un cost computacional significativament més alt. L'objectiu de la seva inclusió és avaluar si la consciència semàntica aporta un benefici mesurable respecte a l'enfocament estructural, que és molt més lleuger.

4. *Custom RBAC Chunking* (Proposta pròpia – Fragmentació amb aïllament de PII)

Aquesta és la contribució central del projecte en la dimensió d'ingesta. L'estratègia opera en quatre passos interns:

1. Aplica *chunking* estructural com a base, respectant les fronteres lògiques del document.
2. Escaneja cada fragment resultant amb 8 patrons de detecció de contingut sensible (*password*, IBAN, SWIFT, *email*, adreça IP, identificador de client, nom de persona, valors monetaris confidencials).
3. Aïlla els fragments sensibles detectats en *chunks* dedicats de mida reduïda (típicament entre 15 i 80 caràcters).
4. Escala el `clearance_level` dels *chunks* aïllats segons el tipus de PII que contenen, independentment del nivell global del document d'origen.

El pas 4 és el mecanisme diferenciador. En les tres estratègies anteriors, tots els fragments d'un document hereten el `clearance_level` del document pare. Això implica que, en un contracte classificat com a *confidential* (nivell 2), un fragment que contingui un IBAN –que requereix nivell 3 *strict*– quedarà accessible per a qualsevol usuari amb nivell 2. L'escalament automàtic del *Custom RBAC Chunking* resol exactament aquesta vulnerabilitat.

6.1.5. Detecció de PII i Model de Clearance

Es defineixen 9 patrons d'expressions regulars per a la detecció d'informació sensible, cadascun associat a un nivell mínim de `clearance` requerit:

Patró	Exemple Detectat	Clearance Mínim Requerit
<code>password</code>	<code>override password: 'witty admin override 2026!\$'</code>	3 (<i>strict</i>)
<code>iban</code>	<code>IT89 A012 3456 7890 1234 5678 901</code>	3 (<i>strict</i>)

swift_code	SWIFT: MCRGIT2M	3 (strict)
email	Lgomez@car.es	2 (confidential)
ip_address	192.168.1.105	2 (confidential)
client_id	CLI-001	2 (confidential)
person_name_context	D. Carlos Mendoza	2 (confidential)
phone	+39 0471 501532	1 (intern)
monetary_value	€4.2 million	1 (intern)

Taula 5 - Tipus de patrons PII beta Chunking

La validació RBAC posterior (*Notebook 03*) verifica dues restriccions sobre cada *chunk* generat:

- **clearance_too_low:** El *chunk* conté PII però el seu `clearance_level` heretat del document pare és inferior al requerit pel tipus de PII detectat. Aquesta violació implica que un usuari amb accés al nivell del document podria veure dades que necessiten un nivell superior. Es classifica com a **CRITICAL** quan la PII requereix nivell 3 (IBAN, contrasenyes, codis SWIFT).
- **missing_department:** El *chunk* té `clearance_level` ≥ 2 però no té `allowed_departments` definit, la qual cosa implica que qualsevol empleat de qualsevol departament podria accedir-hi. Es classifica com a **WARNING**.

6.1.6. Resultats i Validació RBAC

Mètriques de *Chunking*

L'execució de les quatre estratègies sobre el corpus complet produeix les mètriques agregades següents:

Estratègia	Total <i>Chunks</i>	Mitjana <i>Chunks/Doc.</i>	Mida mitjana	Boundary Coherence mitjana	Total de <i>chunks</i> amb PII
<i>Fixed</i>	20	4,0	362	0,400	11
<i>Structural</i>	18	3,6	504	0,527	11
<i>Semantic</i>	74	14,8	110	0,537	14
<i>Custom_RBAC</i>	33	6,6	311	0,399	22

Taula 6 - Mètriques de les tècniques de chunking

Tres observacions rellevants emergeixen d'aquesta taula:

1. **La fragmentació semàntica genera una granularitat excessiva.** Amb 74 *chunks* i una mida mitjana de 110 caràcters, aquesta estratègia produeix fragments massa petits per proporcionar context suficient al LLM. Un fragment típic conté una o dues oracions aïllades, insuficients per a una resposta coherent.
2. **La fragmentació estructural respecta millor les fronteres lògiques.** La seva *boundary coherence* mitjana (0,527) és la més alta de les quatre estratègies, indicant que més de la meitat dels fragments acaben en un punt natural del discurs (fi d'oració, fi de secció). Produeix els fragments més grans (504 caràcters de mitjana), cosa que afavoreix la preservació del context.

3. **Custom RBAC genera més fragments que l'estructural (33 vs 18) a causa de l'aïllament de PII.** Cada fragment sensible detectat es converteix en un *chunk* independent addicional. Per això el nombre de *chunks* amb PII (22) supera àmpliament el de les estratègies convencionals (11): no és que hi hagi més PII, sinó que cada tipus de PII genera el seu propi *chunk* aïllat, facilitant el filtratge granular en la fase de recuperació.

Validació de Seguretat RBAC

Estratègia	Total Chunks	PII Chunks	PII Isolat	Isolatio n %	CRITICAL	WARNING	Score
<i>fixed</i>	20	10	3	30%	2	2	FAIL
<i>structural</i>	18	9	3	33%	2	1	FAIL
<i>semantic</i>	74	11	9	82%	2	2	FAIL
<i>custom_rbac</i>	33	19	15	79%	0	2	PASS

Taula 7 - Resultats de tipus de chunking

El resultat és inequívoc: les tres estratègies convencionals obtenen FAIL (2 violacions CRITICAL cadascuna), mentre que *Custom RBAC* és l'única que obté PASS amb 0 violacions crítiques.

Les 6 violacions CRITICAL es concentren en dos documents: el contracte de distribució (IBAN i codi SWIFT exposats amb `clearance 2` quan requereixen 3) i els registres del servidor (contrasenya exposada amb `clearance 2` quan requereix 3). El mecanisme d'escalament de `clearance` del *Custom RBAC Chunking* resol ambdues vulnerabilitats aïllant les dades sensibles i assignant-los `clearance 3` de manera autònoma.

Tot i l'èxit en seguretat, Custom RBAC manté 2 violacions WARNING relacionades amb el document públic (QuickGuide), on el telèfon i l'*email* corporatiu es detecten com a PII i s'escalen a `clearance 2` sense tenir un departament assignat. Això revela un cas frontera (*edge case*) del mecanisme d'escalament que requereix refinament: quan un document és públic (`clearance 0`), l'escalament hauria d'assignar un departament per defecte o inhibir-se per a PII de baixa severitat.

6.1.7. Limitacions i Conclusions de la Fase 1

Limitacions identificades:

1. **Metadades globals manuals.** En producció, la classificació de `clearance` i departament hauria de ser inferida automàticament per un agent LLM.
2. **Patrons PII basats en *regex*.** Els 9 patrons cobreixen els casos del corpus de test però no són exhaustius. Formes de PII no contemplades (e.g., números de seguretat social, adreces postals) escaparien la detecció.
3. **Corpus de test reduït.** 5 documents (~7.000 caràcters totals) permeten una comparació relativa vàlida entre estratègies, però els valors absoluts de les mètriques canviaran amb volums majors.
4. **Falsos positius.** El patró `swift_code` genera coincidències amb acrònims en majúscules i el patró `phone` captura seqüències numèriques genèriques. En producció, caldria validar amb llibreries dedicades.
5. **Fragmentació excessiva en dades tabulars.** Custom RBAC genera *chunks* duplicats per a la mateixa fila del full de càlcul quan hi ha múltiples tipus de PII. Cal implementar un mecanisme de fusió d'interval solapats.

Conclusió: L'experimentació confirma que les estratègies de *chunking* convencionals no garanteixen la seguretat RBAC perquè no modifiquen els metadades d'accés a nivell de fragment. L'estratègia *Custom RBAC* proposada és l'única que supera la validació gràcies a l'aïllament actiu i l'escalament de `clearance`. No obstant això, l'aïllament de PII introdueix un problema nou –la pèrdua de context semàntic dels fragments aïllats– que s'abordarà, junt amb d'altres, a la Fase 2.

6.2.Fase 2 – Recuperació: *Ensemble Retriever* Híbrid amb Seguretat RBAC

6.2.1. Problemes Heretats de la Fase 1

La Fase 1 va resoldre el problema de seguretat a la capa de metadades, però va introduir dos problemes nous que es manifesten quan els *chunks* resultants entren en un *pipeline* de recuperació:

Problema 1 – Pèrdua de context semàntic.

En aïllar un fragment sensible del seu paràgraf original, es destrueix la relació entre la dada i el seu significat. Considerem l'exemple real següent, extret del corpus:

El paràgraf original del contracte conté la clàusula de condicions de pagament, amb el titular, l'IBAN i el codi SWIFT integrats dins del text. Després del processament *Custom RBAC*:

- El *chunk* aïllat A (55 caràcters, `clearance=3`) conté: IBAN: IT89 A012 3456 7890 1234 5678 901 SWIFT: MCRGIT2M.

- El *chunk* residual (254 caràcters, `clearance=2`) conté la clàusula sola de les dades bancàries.

Si un *retriever* retorna el *chunk* aïllat anterior a un usuari autoritzat (`clearance=3`, `department=legal`), el LLM rep únicament el número d'IBAN sense saber que pertany al contracte de distribució amb Esports d'Alt Rendiment S.A., ni que el titular és Microgate S.R.L. La dada està segura però inutilitzable (li falta context).

Problema 2 – Filtratge asimètric de seguretat.

El segon problema és més subtil i afecta directament l'arquitectura del *retriever* híbrid. Un motor de cerca semàntic com ChromaDB suporta clàusules `where` natives que filtren per metadades abans de retornar resultats. En canvi, un motor lèxic com BM25 opera com un algorisme matemàtic pur sobre un índex invertit en memòria RAM, sense cap concepte de metadades, permisos ni nivells d'accés.

Això implica que un sistema de recuperació que combini els dos motors sense mesures addicionals heretarà la vulnerabilitat del motor més feble: BM25 retornarà *chunks* sensibles per coincidència de termes i el *Reciprocal Rank Fusion* (RRF) els incorporarà al resultat final sense discriminar-ne el nivell d'accés.

6.2.2. Fonaments dels Motors de Cerca

Per comprendre la naturalesa del problema de filtratge asimètric, cal examinar el funcionament intern de cada motor.

Motor de cerca semàntic (dens) – ChromaDB amb embeddings

La cerca semàntica transforma tant els documents indexats com la consulta de l'usuari en vectors numèrics (embeddings) que capturen el significat del text. En aquest projecte, s'utilitza el model `all-MiniLM-L6-v2` de *Sentence Transformers*, un model de 384 dimensions que s'executa localment sense necessitat d'APIs externes, coherent amb l'arquitectura *on-premise*.

Durant la indexació, cada *chunk* es converteix en un vector i s'emmagatzema a ChromaDB juntament amb el text original i les metadades RBAC. Quan arriba una consulta, es projecta al mateix espai vectorial i es calcula la **similitud cosinus** entre el vector de la consulta i tots els vectors emmagatzemats. ChromaDB retorna els K vectors més propers (més similars semànticament).

La **fortalesa** d'aquest motor és la **comprensió conceptual**: una consulta com “ingressos trimestrals del producte” recuperarà un *chunk* que digui “*revenue for the sports division reached €4.2 million*” tot i que no comparteixin cap paraula. La seva **debilitat** és la **baixa precisió amb termes exactes**: si l'usuari cerca `CLI-003` o `MCRGIT2M`, el model *d'embeddings* no té una representació vectorial forta per a codis alfanumèrics i pot fallar.

Des del punt de vista de la seguretat, ChromaDB suporta clàusules `where` natives que filtren per metadades abans d'executar la cerca vectorial. Això permet implementar un *pre-filtering* RBAC eficient:

```
collection.query(
    query_embeddings=[vector],
    where={"clearance_level":{"$lte": user_clearance}},
    n_results=5,
)
```

Figura 7 - Exemple de cerca amb clàusula where en ChromaDB

Motor de cerca lèxic (dispers) – BM25

BM25 (Best Matching 25) és un algorisme clàssic *d'Information Retrieval* basat en la freqüència de termes. No necessita *embeddings* ni models de llenguatge. Durant la indexació, **cada chunk es tokenitza** (es divideix en paraules individuals en minúscules) i es construeix un **índex invertit** en memòria que mapeja cada terme als documents que el contenen. Quan arriba una consulta, es tokenitza de la mateixa manera i es calcula un *score* per a cada document(*chunk*) basat en:

- **TF (Term Frequency):** El nombre d'aparicions de cada terme de la consulta dins del document.
- **IDF (Inverse Document Frequency):** L'infreqüència del terme al corpus. Termes rars (com un IBAN o un nom propi) reben un pes molt superior als termes comuns.
- **Normalització per longitud:** Evita que documents més llargs rebin avantatge injust.

La **fortalesa** de BM25 és la **precisió amb termes exactes**: si l'usuari cerca “IBAN IT89” o “password override”, BM25 trobarà els *chunks* que continguin exactament aquestes paraules. La seva debilitat crítica per a la seguretat és que opera exclusivament sobre un array de tokens en RAM. No existeix cap paràmetre *where*, cap mecanisme de filtratge per metadades, cap manera nativa d'excloure resultats per *clearance* o departament. Si un terme existeix dins d'un *chunk*, BM25 el retornarà independentment de si l'usuari té permís per veure'l.

6.2.3. Justificació de l'Ensemble Híbrid (RRF)

Cap dels dos motors és suficient per si sol:

- Un sistema **exclusivament semàntic** fallaria en cerques de termes exactes (identificadors de client, codis IBAN, noms propis), que són precisament les consultes més perilloses des del punt de vista de la seguretat.
- Un sistema **exclusivament lèxic** fallaria en consultes conceptuais (“rendiment financer del tercer trimestre”) i, a més, no ofereix cap mecanisme de protecció RBAC.

L'enfocament estàndard a la indústria és combinar ambdós motors mitjançant **Reciprocal Rank Fusion (RRF)**, un algorisme que fusiona dues llistes ordenades (*rankings*) en una de sola. La puntuació de cada document s'obté de la següent fórmula:

$$RRF(d) = \alpha \cdot \frac{1}{k + RankChroma(d)} + (1 - \alpha) \cdot \frac{1}{k + RankBM25(d)}$$

Figura 8 - Fórmula Retriever Híbrid - RRF

On α controla el pes relatiu de cada motor (al *baseline* s'aplica $0,5 = 50/50$), k és una constant de suavitzat (típicament 60) que evita que el primer resultat domini excessivament, i si un document només apareix en un dels dos *rankings*, rep *score* 0 de l'altre motor.

El resultat és una llista fusionada que aprofita les fortaleeses d'ambdós motors. No obstant això, **RRF no discrimina per seguretat**: si BM25 retorna un *chunk* amb `clearance_level=3` a un usuari amb `clearance_level=0`, RRF l'incorporarà al *ranking* final sense qüestionar-ho.

**** Arquitectura de font de dades única.** Cal subratllar que BM25 no és una base de dades independent de ChromaDB. En l'arquitectura d'aquest projecte, **ChromaDB és l'única font de dades persistent**. L'índex BM25 es construeix en memòria RAM a l'arrencada del sistema a partir dels mateixos textos emmagatzemats a la col·lecció de ChromaDB. Així, els dos motors operen sobre les mateixes dades; la diferència rau exclusivament en l'algorisme de cerca (vectorial *VS* lèxic) i en la capacitat de filtratge per metadades.

6.2.4. Metodologia – Pipeline de 5 passos

La fase de recuperació s'estructura en 5 passos seqüencials, cadascun implementat en un *Jupyter Notebook* independent. Cada pas introdueix una millora concreta sobre el *baseline* i genera mètriques comparables:

Arxiu	Pas del <i>Pipeline</i> Intern	Descripció
01_retrieval_baseline.ipynb	1: Establir la mètrica de referència	Implementar el <i>ensemble retriever</i> estàndard de LangChain (50/50 RRF entre ChromaDB i BM25) i analitzar les problemàtiques dels resultats obtinguts.
02_dynamic_rrf_weights.ipynb	2: Millorar la precisió de cerca	Programar un <i>router</i> que analitzi la consulta del usuari i assigni major pes als resultats semàntics (ChromaDB) o lèxics (BM25).
03_parent_child_reconstruction.ipynb	3: Solucionar pèrdua de context del chunking	Implementar una lògica Pare-Fill que permeti recuperar el “ <i>chunk</i> pare” d'un que hagi estat aïllat al <i>chunking</i> per PII.
04_asymmetric_hybrid_fusion.ipynb	4: Assegurar seguretat RBAC per a motors lèxics	Implementar un post-filtratge per al motor de cerca lèxic BM25 per assegurar resultats que compleixin el sistema RBAC.

05_retrieval_metrics_analysis.ipynb	5: Extraure conclusions	Estudiar els resultats obtinguts en els anteriors i extraure conclusions.

Taula 8 - Descripció Arxius pel Disseny de Retrieval

L'avaluació es realitza mitjançant un conjunt de 8 consultes de test dissenyades per cobrir escenaris diversos:

- **Consultes genèriques/públiques** (ex., “How do I switch on the Witty timer?”): han de retornar només *chunks* públiques.
- **Consultes dirigides a PII** (ex., “What is the admin override password?”): intenten extreure dades sensibles per validar si el sistema les filtra.
- **Consultes amb perfil d'accés restringit** (ex., un usuari del departament d'engineering cercant dades de sales): validen el filtratge departamental.

Cada consulta s'executa amb un perfil d'usuari simulat (nivell de `clearance` + departament) per determinar si els resultats retornats respecten les restriccions d'accés.

6.2.5. Pas 1: Baseline sense Protecció

Objectiu: Establir la mètrica de referència contra la qual es compararan totes les millores posteriors.

Implementació: S'indexen els 33 *chunks* del *Custom RBAC Chunking* a ChromaDB (cerca semàntica) i es construeix un índex BM25 en memòria (cerca lèxica) sobre els mateixos textos. S'implementen tres modes de recuperació: ChromaDB sol, BM25 sol, i *Ensemble* RRF 50/50. Intencionadament, no s'aplica cap filtre `where` a ChromaDB ni cap post-filtre a BM25, per mesurar el comportament desprotegit.

Per a cada consulta, cada *chunk* retornat s'audita automàticament: es compara el `clearance_level` del *chunk* amb el de l'usuari, i es verifica que el departament del *chunk* coincideixi amb el de l'usuari (per a *chunks* amb `clearance` $\geq$ 2). Si alguna d'aquestes condicions falla, el resultat es registra com una violació de seguretat (**security breach**). A més, es detecta la presència de PII al contingut retornat per confirmar la gravetat de la filtració.

- **Security Breach Rate:**

Es comptabilitza el nombre total de *chunks* retornats que violen les restriccions RBAC (ja sigui per `clearance_level` insuficient o per departament no autoritzat) i el nombre de consultes que contenen almenys una violació. Una consulta amb qualsevol *chunk* no autoritzat al seu *ranking* rep l'etiqueta FAIL; només si tots els resultats respecten els permisos rep PASS.

Adicionalment, per a cada *breach*, es detecten els tipus de PII (`password`, `IBAN`, `email`, etc.) presents al contingut retornat, cosa que permet distingir entre violacions greus (filtració de credencials) i violacions menors (exposició d'informació de contacte corporativa).

Resultats obtinguts:

Motor	Consultes	Amb Breach	Total Breaches	Breach Rate	Latència	PII filtrada
ChormaDB	8	8 (100%)	33	100%	10,6 ms	password, IBAN, SWIFT, email, IP, client_id
BM25	8	7 (88%)	26	88%	0,2 ms	password, IBAN, SWIFT, email, IP, client_id
Ensemble RRF	8	8 (100%)	30	100%	10,4 ms	password, IBAN, SWIFT, email, IP, client_id

Taula 9 - Resultats del Retriever baseline

Els tres motors obtenen FAIL total. **Cap d'ells és segur sense mecanismes de filtratge addicionals**, com era d'esperar.

Anàlisi dels descobriments:

1. **ChromaDB** filtra dades fins i tot en consultes innocents. La consulta Q1 (“How do I switch on the Witty timer?”) és completament pública, però ChromaDB retorna la contrasenya `witty_admin_override_2026!$(clearance=3)` i la IP interna `192.168.1.105 (clearance=2)` entre els 5 primers resultats. Això s'explica perquè, sense filtre where, ChromaDB retorna per similitud semàntica pura: com que tots els documents parlen de “Witty”, els chunks confidencials cauen dins del top-5.
2. **BM25** exposa dades sensibles per coincidència lèxica directa. La consulta Q3 (“admin override password”) retorna el *chunk* aïllat que conté exactament la contrasenya, a un usuari amb `clearance=1`. La consulta Q4 (“IBAN bank account”) retorna el número IBAN i el codi SWIFT a un usuari del departament sales (que no té accés a legal). La consulta Q5 retorna 5 *chunks* amb *emails* personals i identificadors de client a un usuari *d'engineering*.
3. **L'únic PASS és la consulta Q8 a BM25**, on un director de finances (`clearance=3, dept=finance`) cerca dades financeres. BM25 passa perquè els *chunks* financers tenen `clearance=3` i `departament=finance`, que coincideixen amb l'usuari. **ChromaDB falla fins i tot aquí** perquè la similitud semàntica arrossega *chunks* de sales (correus electrònics de clients) al top-5.
4. **L'Ensemble hereta les vulnerabilitats d'ambdós**. RRF combina els resultats de tots dos motors sense filtrar, de manera que les filtracions de cadascun es propaguen al resultat fusion

Confirmació de la pèrdua de context. Els *chunks* `PII` aïllats són extremadament curts (15–80 caràcters). Quan BM25 retorna IBAN: IT89 A012 3456 7890 1234 5678 901 (55 caràcters), el LLM no pot determinar a quin contracte pertany. És una dada òrfena (sense *chunk* “pare”/referència). Això confirma la necessitat del mecanisme de reconstrucció pare-fill previst al Pas 3.

Implicació per als passos següents:

- **El Pas 2** (pesos RRF dinàmics) pot millorar la precisió de la cerca però no resoldrà les filtracions: el problema és estructural, no de ponderació.
- **El Pas 4** (fusió asimètrica amb filtratge RBAC) és imprescindible: BM25 necessita un post-filtre que elimini *chunks* no autoritzats abans d'aplicar RRF.
- **El Pas 3** (reconstrucció pare-fill) és necessari perquè els *chunks* aïllats siguin útils, però el context pare només s'ha d'injectar si l'usuari té el `clearance` suficient.

6.2.6. Pas 2: Pesos RRF Dinàmics

Punt de partida: Al Pas 1 (*Baseline*) hem demostrat que un *Ensemble Retriever* amb un repartiment estàtic de pesos 50/50 entre el motor semàntic (ChromaDB) i el motor lèxic (BM25) obté un *Mean Reciprocal Rank (MRR)* mitjà de **0,8929** sobre un conjunt de 14 consultes de test. Tot i que aquesta xifra és raonable, el repartiment estàtic del paràmetre α a la fórmula del *Reciprocal Rank Fusion* (RRF) implica una limitació fonamental a solucionar: tracta totes les consultes de la mateixa manera, independentment de la seva naturalesa.

La pregunta que obre aquest pas és, doncs: “*Es pot millorar la precisió de recuperació adaptant dinàmicament el pes de cada motor a la naturalesa de cada consulta individual?*”

Objectiu: L'objectiu d'aquest experiment és dissenyar i avaluar un mecanisme de classificació d'intencions (*intent routing*) que, prèviament a l'execució de la cerca, analitzi la consulta de l'usuari i assigni un valor de α personalitzat per a la fórmula RRF vista anteriorment (*Figura 8*).

L'expectativa és que, en les consultes on la intenció és clarament lèxica (com demanar un número concret, una paraula concreta, etc.), un α baix (0,2) afavoreixi BM25 i posicioni el *chunk* correcte més amunt al *ranking*; i que, en les consultes conceptuais, un α alt (0,8) afavoreixi ChromaDB amb el mateix efecte.

Cal subratllar que l'objectiu d'aquest pas és exclusivament la precisió de recuperació, no la seguretat. El filtratge RBAC es resoldrà al Pas 4 (Fusió Asimètrica). Aquesta part mesura si el *chunk* rellevant arriba a una posició més alta del *ranking*, independentment de si l'usuari tindria dret a veure'l.

Mètriques d'avaluació emprades:

1. MRR – Mean Reciprocal Rank:

El *Reciprocal Rank* (RR) d'una consulta individual mesura en quina posició del *ranking* apareix el primer resultat rellevant. Si el primer *chunk* rellevant es troba a la posició p , aleshores $RR = 1/p$. El *Mean Reciprocal Rank* (MRR) és la mitjana aritmètica del RR sobre totes les Q consultes del conjunt de test:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \left(\frac{1}{rank\ i} \right)$$

$Rank\ i$ és la posició del primer resultat rellevant per a la consulta i . Si cap resultat es rellevant $RR_i = 0$.

Interpretació pràctica: un MRR de 1,0 significa que el primer resultat retornat és sempre el correcte; un MRR de 0,5 indica que, de mitjana, el primer resultat rellevant apareix a la posició 2.

2. Relevant@K:

Aquesta mètrica compta quants dels K primers *chunks* retornats pel sistema de cerca corresponen al document esperat i contenen almenys un dels termes clau definits per a la consulta. Concretament, per a cada consulta es defineix un conjunt de fitxers font esperats i un conjunt de paraules clau; un *chunk* es considera rellevant si prové d'un dels fitxers esperats i conté almenys una de les paraules clau. Per a aquesta part del projecte s'utilitzen/analitzen dos tipus:

- **Relevant@3:** Resultats rellevants dins de les 3 primeres posicions. Mesura la precisió en condicions estrictes, on el LLM rebria un context molt reduït.
- **Relevant@5:** Resultats rellevants dins de les 5 primeres posicions.

Metodologia i Implementació:

- Disseny de l'enrutador d'intencions:

S'ha implementat un classificador lleuger **basat en expressions regulars** que analitza el text de la consulta i la classifica en una de **quatre categories d'intenció**. La **decisió de no emprar un model de llenguatge per a aquesta classificació és deliberada**: en un sistema *on-premise* amb requisits de latència baixa, un classificador basat en regles ofereix un temps d'execució negligible ($< 0,1\ ms$) i una predictibilitat total, evitant la variabilitat inherent als models probabilístics.

Nivell	Nom	Patrons	α -Valor	Comportament
--------	-----	---------	-----------------	--------------

<i>Tier 1</i>	PII Crítica	<i>password</i> , IBAN, SWIFT	0,2	Sempre s'imposa, independentment de qualsevol senyal conceptual concurrent
<i>Tier 2</i>	Exacte Estàndard	<i>email</i> , <i>client_id</i> , adreça IP, <i>log</i> , nom propi	0,2	S'aplica si no hi ha senyals conceptuais; si n'hi ha, es calcula un desempat ponderat
<i>Tier 3</i>	Conceptual	Paraules interrogatives (<i>how</i> , <i>what</i>), termes de rendiment, referències a productes, trimestres	0,8	S'aplica si no hi ha cap senyal exacte

Taula 10 - Categories regex beta enrutador d'intencions

La regla de desempat per a consultes mixtes (Tier 2 + Tier 3 simultanis) utilitza un factor de ponderació de 1,5x per als patrons exactes. Si el pes total dels patrons exactes iguala o supera el dels conceptuais, s'assigna $\alpha = 0,35$ (*mixed exact lean*); en cas contrari, $\alpha = 0,65$ (*mixed conceptual lean*).

- **Versió 1 i trade-offs (NO USADA):**

La primera iteració del classificador tractava tots els patrons amb la mateixa prioritat i emprava un desempat per simple recompte de coincidències. Això va provocar una classificació errònia en una consulta clau: “*What is the admin override password for the backend?*” va ser classificada com a *mixed_conceptual_lean* ($\alpha = 0,65$) perquè el patró genèric *what_question* (conceptual) empatava amb el patró *password_seek* (exacte). L'efecte pràctic va ser que el motor semàntic rebia més pes del necessari, empenyent el *chunk* que contenia la contrasenya a una posició inferior del *ranking* i provocant una regressió del MRR de 1,0 a 0,5 per a aquesta consulta.

- **Versió 2: Correcció amb prioritat de nivells:**

Per resoldre aquest problema, la v2 introdueix el sistema de nivells descrit anteriorment. La regla fonamental és que els patrons de *Tier 1* (PII crítica) s'imposen incondicionalment sobre qualsevol senyal conceptual. Així, la consulta “*What is the admin override password?*” ara es classifica com a *exact_critical* ($\alpha = 0,2$) perquè el patró *password* pertany al *Tier 1* i anul·la el patró genèric *what*. Aquesta mateixa correcció afecta les consultes de prova que contenen “IBAN”, “SWIFT” o “*override*”.

- **Conjunt de proves ampliat:**

Per obtenir una mostra més representativa, s'ha ampliat el conjunt de 8 a 14 consultes de test, afegint-ne 6 de noves que cobreixen casos absents a la v1:

- Consultes purament conceptuais sense termes exactes (Q9: “*How does the photocell alignment process work?*”, Q10: “*Describe the Witty product line strategy*”).
- Consultes purament lèxiques amb identificadors (Q11: “*CLI-001 CLI-002 billing information*”, Q12: “*SWIFT code MCRGIT2M*”).
- Una consulta mixta amb senyals d'ambdós tipus (Q13: “*What is the distribution contract about and who signed it?*”).

- Una consulta exacta per part d'un usuari autoritzat (Q14: “*override password for emergency server recovery*”, amb `clearance = 3`).

Cada consulta s'executa contra quatre modes de recuperació: ChromaDB sol, BM25 sol, *Ensemble baseline* ($\alpha = 0,5$) i *Ensemble* dinàmic (α assignat pel classificador). Això permet determinar, per a cada consulta, quin motor individual és superior i si el pes dinàmic aconsegueix alinear l'*ensemble* amb el motor òptim.

Resultats obtinguts:

- **Classificació d'intencions:**

El classificador v2 va categoritzar les 14 consultes de la següent manera:

Intenció	Consultes	α	Descripció
exact_critical	Q3, Q4, Q12, Q14	0,2	Consultes amb patrons de PII de <i>Tier 1</i> (password, IBAN, SWIFT)
exact	Q5, Q7, Q11	0,2	Consultes amb termes exactes estàndard (email, log, client_id)
conceptual	Q1, Q2, Q6, Q8, Q9, Q10, Q13	0,8	Consultes de resum, rendiment, producte o contracte

Taula 11 - Resultats esperats per queries suit ph2-step2

- **Mètriques agregades:**

La següent taula compara les mètriques agregades dels quatre modes de recuperació sobre les 14 consultes:

Motor	MRR Mitjà	Relevant@3	Relevant@5	Breaches Totals	Latència
ChromaDB sol	0,8690	28	32	53	8,3 ms
BM25 sol	0,8571	27	35	39	0,2 ms
<i>Baseline</i> ($\alpha = 0,5$)	0,8929	28	32	47	8,1 ms
Dinàmic	0,8214	27	33	46	8,3 ms

Taula 12 - Mètriques RRF retrieval

- **Anàlisi per consulta:**

L'anàlisi consulta per consulta entre el *baseline* i el mode dinàmic revela:

- **0 consultes amb millora de MRR:** Cap consulta individual va guanyar posicions al *ranking* del primer resultat rellevant.
- **2 consultes amb regressió de MRR** (Q2 i Q8): En ambdós casos, el mode dinàmic va “degradar” el *chunk* més rellevant de la posició 1 a la posició 2, reduint el MRR de 1,0 a 0,5.

- **12 consultes sense canvi de MRR:** El primer *chunk* rellevant manté la mateixa posició.
- **+1 en *Relevant@5* net:** Dues consultes (Q1 i Q6) guanyen un *chunk* rellevant addicional dins dels 5 primers resultats, mentre que una (Q5) en perd un.
- **-1 *breach* net:** Quatre consultes milloren en seguretat (Q1, Q3, Q4, Q7) i dues empitjoren (Q6, Q9).

El resultat agregat és que el mode dinàmic **no millora el MRR respecte al *baseline* PER A AQUEST CORPUS I SET DE CONSULTES REDUIT; de fet, el redueix lleugerament** (0,8214 vs 0,8929), tot i que aporta un guany marginal en *Relevant@5* (+1) i en *breaches* (-1).

Problemes trobats i anàlisi del sostre estructural:

Els resultats obtinguts no justifiquen, per si sols, la implementació del classificador d'intencions **sobre aquest corpus experimental**. Per comprendre per què una tècnica arquitectònicament sòlida produeix un impacte tan limitat per aquest cas tant concret, hem dut a terme una anàlisi estructural del corpus que identifica quatre factors limitants:

1. **Homogeneïtat temàtica del corpus.** El 70% dels *chunks* del corpus contenen els termes “Witty” o “Microgate”, ja que tots els documents pertanyen a la mateixa organització fictícia. Això provoca que els *embeddings* de tots els *chunks* es concentrin en una regió reduïda de l'espai vectorial de 384 dimensions, de manera que ChromaDB retorna inevitablement resultats creuats entre documents independentment del valor d' α . En un corpus de producció amb centenars de temes diversos, els *embeddings* formarien clústers molt més diferenciats i el canvi d' α tindria un efecte molt més pronunciat sobre quins *chunks* apareixen al *top-K*.
2. **Ràtio corpus/K.** El corpus consta de 33 *chunks* i es recuperen els 5 primers (*top-5*), és a dir, un 15% del corpus complet. Amb una finestra de recuperació tan gran en proporció, queda molt poc marge perquè un canvi d' α produeixi un conjunt de resultats diferent. En un sistema de producció on el corpus contindria milers de *chunks*, el *top-5* representaria menys del 0,1% del total, i la diferència entre un $\alpha = 0,2$ i un $\alpha = 0,8$ seria substancialment més visible.
3. **Solapament entre motors.** De mitjana, entre 1 i 2 dels 5 *chunks* retornats apareixen simultàniament al *top-5* d'ambdós motors (ChromaDB i BM25). Quan un *chunk* rep *score* d'ambdues fonts, el canvi d' α únicament modifica l'ordre relatiu per fraccions de puntuació, insuficients per promocionar un *chunk* diferent per sobre seu. **L'experiment de dett, Clarke i Büttcher (2009)** sobre la fusió de *rankings* (RRF) ja observava que aquest mètode funciona millor que d'altres de fusió de *rankings* perquè és capaç d'aprofitar la diversitat entre els motors que es fusionen. Per tant, la manca d'heterogeneïtat en el corpus de proves actual justifica uns resultats inicials discrets (ja que els resultats dels dos motors son semblants), els quals

experimentaran una millora significativa en la implementació final un cop s'hi introdueixi més diversitat.

4. **Asimetria de mida dels fragments PII.** Els *chunks* que contenen informació sensible aïllada tenen una mida mitjana de 45-80 caràcters, mentre que els *chunks* no sensibles tenen una mitjana de 300-500 caràcters. BM25 assigna puntuacions desproporcionadament altes als fragments PII perquè l'*Inverse Document Frequency* (IDF) amplifica els termes rars concentrats en documents molt curts. Aquesta és una propietat intrínseca de l'algorisme BM25 i no pot ser contrarestada modificant α ; és un biaix estructural que opera per sota del nivell de la fusió RRF.

La limitació no invalida l'enfocament sinó que en delimita el rang d'aplicabilitat: el benefici de la classificació prèvia d'intencions escala amb la diversitat temàtica, la mida del corpus i la divergència entre motors.

Conclusions del Pas 2:

Tot i que les mètriques agregades no mostren una millora significativa sobre el *baseline* en aquest corpus de 33 *chunks*, el Pas 2 aporta tres contribucions rellevants per al projecte:

1. **Valida l'arquitectura del classificador.** El sistema de prioritat classifica correctament les 14 consultes del conjunt de test.
2. **Estableix una infraestructura d'enrutament reutilitzable.** Un classificador ja validat i estable és un prerequisite per a la integració del pas 4 (fusió asimètrica).
3. **L'anàlisi del sostre estructural produeix coneixement acadèmic transferible.** La identificació dels quatre factors limitants (homogeneïtat temàtica, ràtio corpus/K, solapament entre motors i asimetria de mida PII) no solament explica els resultats d'aquest experiment sinó que defineix les condicions sota les quals la tècnica de pesos dinàmics serà efectiva en un desplegament real.

I són per aquests motius i els anteriors que la implementació d'aquesta tècnica es durà a terme per a la versió final, llevat que realment es vegi que el resultat no es òptim.

6.2.7. Pas 3: Reconstrucció Pare-Fill

Punt de partida:

Els dos passos anteriors han abordat la qualitat del *ranking* de recuperació: el Pas 1 va establir un *baseline* de precisió ($MRR = 0,8929$) i el Pas 2 va intentar millorar-lo mitjançant l'adaptació dinàmica del paràmetre α del RRF. L'anàlisi del sostre estructural del Pas 2 va concloure, però, que la precisió del *ranking* no és el coll d'ampolla principal del sistema. **El problema crític rau** en conseqüència de l'estratègia de *chunking* implementada a la Fase 1.

Recordem que a la Fase 1, quan l'estratègia *Custom RBAC Chunking* detectava informació personal identificable (PII) dins d'un paràgraf/*chunk*, aïllava la dada sensible en un *chunk* independent de mida molt reduïda (entre 15 i 120 caràcters) i elevava el `clearance_level` per protegir-la. Aquesta decisió va resoldre el problema de seguretat a nivell d'emmagatzematge, però **va introduir un efecte col·lateral greu: la destrucció del context semàntic**.

Concretament i utilitzant com a exemples casos reals trobats en el corpus creat per a les proves d'aquestes fases del projecte, un fragment aïllat com ara “*override password: 'witty_admin_override_2026!\$'*” (48 caràcters) conté la dada sensible però no informa de res més: no indica a quin sistema pertany la contrasenya, en quin moment es va utilitzar, ni quin procés la va requerir. Si un LLM rep aquest fragment com a part del context injectat, disposa de la dada crua però no té capacitat per formular una resposta coherent i contextualitzada. De manera anàloga, un fragment de 55 caràcters com “IBAN: IT89 A012 3456 7890 1234 5678 901 SWIFT: MCRGIT2M” conté les dades bancàries però no indica a quina empresa pertanyen, en quin contracte s'utilitzen ni cap altre context.

Aquest problema no pot ser resolt ajustant pesos de cerca ni millorant el *ranking*: és un defecte estructural del corpus que requereix una etapa de reconstrucció de context posterior a la recuperació.

Objectius:

L'objectiu d'aquest experiment és dissenyar i avaluar un mecanisme de reconstrucció pare-fill que restauri automàticament el context semàntic dels fragments PII aïllats preservant la seguretat de rols, seguint els següents passos.

1. **Construir un índex pare-fill** que vinculi cada *chunk* PII aïllat del corpus Custom RBAC (el “fill”) amb el paràgraf original de l'estratègia Fixed Chunking (el “pare”) del qual va ser extret.
2. **Implementar una lògica de reconstrucció condicionada** que quan el recuperador retorni un fragment PII aïllat, el sistema ha de buscar el seu pare i decidir:
 - a. Si l'usuari té `clearance_level` més alt que el *chunk* i pertany al departament autoritzat del pare, fusionar els dos fragments (pare i fill), restaurant així el context.
 - b. Si l'usuari no compleix els requisits RBAC del pare descartar-lo, que consisteix bàsicament en eliminar el fragment aïllat del *ranking* final perquè un fragment sense context no aportarà valor al raonament del LLM.

Es validarà el mecanisme mitjançant diferents conjunts de casos de prova que cobreixin totes les branques de decisió i que **analitzen la preservació de la seguretat i l'efecte directe en la millora del raonament del LLM.**

Metodologia:

- **Identificació de fragments aïllats:**

El primer repte metodològic és determinar, en temps d'execució, si un *chunk* retornat pel recuperador és un fragment PII aïllat que necessita reconstrucció o un *chunk* de mida completa que ja disposa de context suficient. S'ha optat per una heurística dual que combina dues condicions:

1. **Condició de metadada:** el camp `contains_PII` del *chunk* ha de ser `true`.
2. **Condició de mida:** la longitud del contingut textual ha de ser inferior o igual a 120 caràcters.; per cassos d'error que es puguin donar en la ingesta.

El llindar de 120 caràcters s'ha establert empíricament a partir de l'anàlisi del corpus: els fragments PII aïllats tenen una mida compresa entre 10 i 101 caràcters (la fila de client més llarga), mentre que els *chunks* no aïllats el superen.

- **Construcció de l'índex pare-fill:**

L'índex pare-fill vincula cada *chunk* PII aïllat del corpus amb el *chunk* de l'estratègia *Fixed Chunking* que conté el seu text. L'algorisme de vinculació opera de la següent manera:

1. Per a cada *chunk* amb `contains_PII = true` del corpus, s'extreu el fitxer font (`source_file`) i l'identificador (`chunk_id`).
2. Es cerca, entre tots els *chunks* de l'estratègia *Fixed* que provenen del mateix fitxer font, aquell que conté el text del fill com a subcadena.
3. Si existeixen múltiples candidats (perquè el text apareix en més d'un *chunk Fixed*), es selecciona el més curt, ja que proporciona el context més específic.
4. Com a mecanisme de *fallback*, si no es troba cap coincidència exacta de subcadena, s'empra un càlcul de solapament a nivell de paraula i es requereix un mínim de 2 paraules compartides.

**** Proposta per a entorns de producció:** Òbviament, **s'ha escollit aquesta tècnica per la facilitat de recuperar el “chunk pare”** del fitxer JSON generat de la fase 1, però **en l'entorn de producció aquesta tècnica s'haurà d'adaptar** i buscar una solució al repte de trobar al progenitor, ja sigui aplicant un índex únic en forma d'un nou camp de *metadata* en la ingesta a cada *chunk* i també una referència a aquest camp pels aïllats o d'un altre manera que es detallarà a la part d'implementació per la solució final.

- **Lògica de reconstrucció:**

El motor de reconstrucció actua com una **capa de post-processament** sobre els resultats del recuperador. Per a cada *chunk* retornat, executa el procés de decisió següent:

1. Si el *chunk* no és un fragment PII aïllat es manté el *ranking* sense modificacions (`KEPT`).
2. Si és un fragment aïllat i existeix un pare a l'índex:

- a. Es comproven els permisos RBAC de l'usuari simulat contra les metadades del *chunk* pare.
 - b. Si `user.clearance_level >= pare.clearance_level` i el departament de l'usuari coincideix amb el del pare (o el pare és d'accés universal "all"): es substitueix el fragment aïllat pel contingut complet del pare (**MERGED**).
 - c. Si els permisos no es compleixen: s'elimina el fragment del ranking (**DROPPED**).
3. Deduplicació: si el pare ja és present al *ranking* (perquè el recuperador l'ha retornat de manera independent), s'evita la duplicació.

Conjunt de Proves:

S'han dissenyat 8 casos de prova que cobreixen totes les branques de decisió del motor de reconstrucció, agrupats en tres categories:

1. Casos de reconstrucció autoritzada (resultat esperat: **MERGE**): T1; T3; T5; T7.
2. Casos de denegació (resultat esperat: **DROP**): T2; T4; T6.
3. Cas neutre (resultat esperat: **KEPT**): T8.

T1: Usuari amb `clearance_level = 3` consulta la contrasenya → el pare (registre de *logs*, `cl = 2`, departament = *engineering*) és **accessible**.

T2: Usuari amb `cl = 1` consulta la contrasenya → el pare (`cl = 2`) **requereix un `clearance_level` superior**.

T3: Usuari amb `cl = 3` consulta l'IBAN → el pare (clàusula contractual, `cl = 2`, departament = *legal*) és **accessible**.

T4: Usuari amb `cl = 2` consulta l'IBAN → el pare (departament = *legal*) no coincideix amb *engineering*.

T5: Usuari amb `cl = 3` consulta correus electrònics de clients → el pare (full de càlcul, `cl = 3`, departament = *sales*) és **accessible**.

T6: Usuari amb `cl = 3` consulta correus de clients → el pare (**departament = *sales***) no coincideix amb **finance**.

T7: Usuari amb `cl = 2` consulta l'adreça IP → el pare (registre de *logs*, `cl = 2`, departament = *engineering*) és **accessible**.

T8: Usuari públic amb `cl = 0` consulta informació del producte *Witty Kit* → no hi ha fragments PII aïllats al *ranking*; **tots els *chunks* es mantenen sense modificació**.

Figura 9 - Tests per la estratègia de disseny del pas 3 del retrieval

Resultats:

Els 8 casos de prova han produït exactament l'acció esperada, amb un 100% d'*accuracy* en la classificació de les accions de reconstrucció:

Test	Usuari	Acció esperada	Acció real	Breaches abans	Breaches després	Verificació
T1	Eng. cl=3	MERGE	MERGE	0	0	PASS
T2	Sales, cl=1	DROP	DROP	3	2	PASS
T3	Legal, cl=3	MERGE	MERGE	3	3	PASS

T4	Eng. cl=2	DROP	DROP	3	2	PASS
T5	Sales, cl=3	MERGE	MERGE	2	2	PASS
T6	Finance, cl=3	DROP	DROP	3	0	PASS
T7	Eng. cl=2	MERGE	MERGE	2	2	PASS
T8	Public, cl=0	KEPT	KEPT	3	3	PASS

Taula 13 - Joc de proves pas 3 estratègia retrieval

A continuació es mostra un exemple de cas de reconstrucció de *chunk*:

Cas T1 – Contrasenya del servidor (reconstrucció autoritzada). Abans de la reconstrucció, el *ranking* conté el fragment aïllat de 48 caràcters: "override password: 'witty_admin_override_2026!\$'". Després de la reconstrucció, aquest fragment és substituït pel paràgraf complet de 286 caràcters del registre de *logs*:

CRITICAL: System alert. The backup server requires manual authentication. [2026-03-05 11:12:35] DEBUG: Auto-recovery initiated. Using override password: 'witty_admin_override_2026!\$' [2026-03-05 11:15:00] INFO: Auto-recovery successful. Resuming normal operations.

Figura 10 - PII Chunk Restoration

El LLM disposa ara del context operatiu complet: sap que la contrasenya es va utilitzar en un procés d'autorecuperació del servidor de backup, que va ser activat per una alerta crítica i que l'operació va concloure amb èxit. Aquesta informació és essencial per generar una resposta coherent i útil per a l'enginyer que formula la consulta.

- **Impacte sobre la seguretat:** La mitjana de violacions passa de **2,38 abans a 1,75, una reducció del 26%**. Tanmateix, la taxa de seguretat PASS roman baixa: només 2 dels 8 tests la superen (T1, que ja la superava abans, i T6, que l'aconsegueix gràcies a la reconstrucció). **Les violacions residuals provenen de *chunks* no aïllats de documents restringits que el recuperador retorna sense cap filtratge de metadades.** Aquesta és precisament la vulnerabilitat que ha de resoldre el Pas 4 (Fusió Asimètrica).
- **Latència:** La latència per a aquests tests es **pràcticament negligible** (inferior a 0,1 ms, mesurat amb temporitzador Python), **però això s'explica per la "trampa"** de buscar els chunks pare al fitxer de resultats obtingut a les proves. **A la implementació final s'espera una major latència però no crítica.**

Validació amb LLM: impacte sobre la productivitat de generació:

Les mètriques anteriors (violacions, latència, taxa PASS) demostren que el mecanisme de reconstrucció és funcionalment correcte. Tanmateix, l'objectiu últim d'un sistema RAG no és produir un *ranking* net sinó generar respostes útils per a l'usuari final. Per validar que la restauració de context es tradueix en una millora de la qualitat de les respostes del LLM,

s'ha dissenyat un experiment que compara directament les respostes generades amb i sense reconstrucció.

- **Disseny de l'experiment:** S'ha emprat el model Llama 3.2 (3,2B paràmetres, quantització Q4_K_M) executat localment mitjançant Ollama. Per a cadascun dels 5 tipus de PII presents al corpus inicial (contrasenya, IBAN, adreça IP, correu electrònic de client i nom de persona) s'ha formulat una consulta que demana no solament la dada sensible sinó també el context operatiu que l'envolta. El LLM rep un *prompt* RAG estàndard amb instruccions de respondre exclusivament a partir del context proporcionat, i es generen dues respostes:
 - **A: Broken Context:** el context injectat és únicament el fragment PII aïllat
 - **B: Restored Context:** El context injectat és el paràgraf pare complet després de la reconstrucció.
- **Resultats per cas:** La següent taula mostra els resultats del LLM per cada cas.

Cas	PII	Expansió	Resposta Broken	Resposta Restored
1. Contrasenya	password	48 → 286 ch	Refús del LLM: “ <i>I can't provide information about an admin override password.</i> ”	Resposta completa: identifica la contrasenya, explica que es va utilitzar en un procés d'auto recuperació del servidor de <i>backup</i> activat per una alerta crítica.
2.IBAN	iban	55 → 310 ch	Reconeix l'IBAN però admet: “ <i>not any additional details about the payment terms or conditions</i> ”	Resposta completa: proporciona l'IBAN, indica que els pagaments són trimestrals per transferència bancària, i identifica el titular (Microgate S.R.L.).
3.IP	ip_address	21 → 462 ch	Refús del LLM: “ <i>I cannot provide information about a specific IP address that may be associated with a security incident.</i> ”	Identifica la IP 85.214.132.11, explica que va causar un intent de login fallit per a l'usuari admin, i la classifica com a event ERROR.
4.Correu	email	104 → 433	Proporciona el correu però indica: “ <i>I don't have any information about their billing details.</i> ”	Proporciona el correu i l'import de facturació anual (12.500 €) del client CLI-001.
5.Nom	person_name	250 → 346 ch	Resposta completa: identifica D. Carlos	Resposta equivalent.

			Mendoza i l'empresa que representa.	
--	--	--	-------------------------------------	--

Taula 14 - Resultats productivitat LLM amb Recuperació Pare-Fill

Es revelen 3 comportaments diferenciats:

- 1. Fragments petits (Casos 1, 2 i 3.):** El LLM refusa respondre o reconeix explícitament que no disposa de context suficient. En els casos de la contrasenya i l'adreça IP, el model activa els seus mecanismes de seguretat interns i genera un refús directe. En el cas de l'IBAN, proporciona la dada crua però admet la incapacitat d'explicar les condicions de pagament. Amb el context restaurat, els tres casos produeixen respostes completes i contextualitzades que citen dades específiques i expliquen la situació operativa.
 - 2. Fragments mitjans (Cas 4):** El LLM pot respondre parcialment, ja que identifica el *email*, però no disposa de la resta de camps del *excel*. Amb la reconstrucció, la resposta incorpora totes les dades del registre complet.
 - 3. Fragments grans (Cas 5):** El fragment ja conté prou context per generar una resposta adequada i amb la reconstrucció no s'aporta una millora apreciable. Això confirma que el llindar de 120 caràcters establert per a la heurística de detecció d'aïllament és adequat: els fragments per sobre d'aquesta mida ja són autosuficients per al LLM.
- Conclusió de la validació LLM:** La reconstrucció pare-fill no és únicament una millora tècnica del *pipeline* de recuperació: **és una condició necessària perquè el LLM pugui generar respostes útils** quan el sistema retorna fragments PII aïllats. Sense reconstrucció, el LLM rep dades crues descontextualitzades que, en el millor dels casos, produeixen respostes incompletes i, en el pitjor, provoquen refusos del model.

Problemes trobats i propostes:

- Problema 1: Col·lisió d'identificadors**

Durant la implementació es va detectar que els `chunk_id` del corpus de proves no són globalment únics: cada fitxer font reinicia la numeració des de `custom_000`. Això va provocar que, en una primera versió de l'índex pare-fill que emprava `chunk_id` com a clau simple, les vinculacions es sobreescrivien mútuament. **Per a la implementació final, la part de la fase d'ingesta ha d'assegurar que els índex siguin únics i incrementals i afegir els camps necessaris com a metadades per a la vinculació de *chunks*.**

- Problema 2: Limitació del mecanisme en *chunks* no aïllats**

La reconstrucció pare-fill només actua sobre fragments PII aïllats de mida reduïda. Els *chunks* complets de documents restringits que el recuperador retorna a usuaris no autoritzats escapen completament al mecanisme. Aquesta és una limitació per disseny,

no un defecte: la reconstrucció resol el problema de context perdut, no el de filtratge d'accés. **La solució al filtratge d'accés requereix intervenir abans o durant la cerca, no després, i és l'objectiu del Pas 4.**

Conclusions del Pas 3:

En un sistema de producció, l'índex pare-fill podria ser precalculat durant la fase d'ingestió de documents i emmagatzemat com a metadada addicional de cada *chunk* (per exemple, un camp `parent_chunk_id` a ChromaDB). Això eliminaria la necessitat de l'estratègia de cerca per subcadena implementada en aquest experiment i proporcionaria vinculacions deterministes amb cost de consulta $O(1)$.

El Pas 3 demostra que la reconstrucció pare-fill **resol de manera efectiva el problema de pèrdua de context introduït per l'estratègia d'*chunking* RBAC de la Fase 1.**

En primer lloc, la reconstrucció restaura el context semàntic correctament i l'LLM passa de rebre una dada crua descontextualitzada a rebre la dada integrada en el seu context operatiu, contractual o organitzatiu original.

En segon lloc, la porta RBAC funciona correctament en totes les combinacions de `clearance_level` i `department`.

La reducció del 26% en violacions de seguretat és significativa però insuficient mentre la millora en l'efectivitat de LLM millora molt. La taxa de seguretat PASS es manté en 2/8 perquè les violacions restants provenen de *chunks* no aïllats de documents restringits que el recuperador entrega sense cap filtratge de metadades. Aquesta observació confirma que la seguretat completa del sistema requereix un mecanisme de filtratge a nivell de motor de cerca, no únicament a nivell de post-processament. Aquesta és exactament la motivació del Pas 4 (Fusió Asimètrica).

6.2.8. Pas 4: Fusió Asimètrica amb Filtratge

Punt de partida:

Els tres passos anteriors han demostrat que les millores aplicades al *ranking* (Pas 2) i al post-processament del context (Pas 3) no resolien el problema de seguretat fonamental del sistema: **el motor lèxic BM25 no disposa de filtratge natiu per metadades** i, per tant, retorna *chunks* sensibles independentment del `clearance_level` o `department` de l'usuari. L'anàlisi del Pas 3 va quantificar aquesta vulnerabilitat residual: **només 2 de 8 casos de prova superaven la verificació de seguretat**, i les violacions restants procedien de *chunks* complets de documents restringits que la reconstrucció pare-fill no podia interceptar perquè no són fragments PII aïllats.

L'origen del problema és arquitectònic. Mentre que ChromaDB exposa una clàusula `where` que permet filtrar el corpus abans d'executar la cerca per similitud, BM25 és un algorisme purament estadístic basat en freqüència de termes que no té cap noció de metadades i per tant no es pot aplicar cap pre-filtratge natiu.

Objectius:

La proposta és dissenyar un *Asymmetric Hybrid Retriever* que tracti els dos motors de manera diferent segons les seves capacitats natives, però que produeixi un *ranking* final unificat lliure de violacions RBAC. Concretament:

1. Aplicar pre-filtering natiu a ChromaDB mitjançant la clàusula `where` per restringir l'espai de cerca abans del càlcul vectorial.
2. Aplicar post-filtering iteratiu a BM25 mitjançant un bucle Python que recuperi un pool sobredimensionat (Top-30) i descarti els *chunks* no autoritzats abans de la fusió.
3. Aplicar el RRF només sobre els documents supervivents dels dos motors.
4. Mesurar empíricament la penalització de latència introduïda pel filtratge, especialment pel bucle Python de BM25, per quantificar el *trade-off* entre seguretat i temps de resposta.

Metodologia:

- **Arquitectura asimètrica**

El recuperador implementa dues branques amb estratègies de filtratge diferenciades:

- **La branca de ChromaDB (pre-filtering):** S'aplica una clàusula `where`: `{"clearance_level": {"$lte": user_cl}}` que elimina del candidat tots els chunks amb nivell d'autorització superior al de l'usuari abans del càlcul de similitud vectorial. I per a *chunks* amb `clearance_level ≥ 2`, es verifica si el `department` coincideix també. **Aquesta combinació és matemàticament segura:** els *chunks* no autoritzats mai entren al càlcul de distàncies cosinus per la cerca de similitud de ChromaDB.
- **La branca de BM25 (post-filtering):** Com que BM25 no té filtratge s'extreu un *pool* sobredimensionat de 30 candidats (en lloc dels 5 habituals) i s'itera sobre ells aplicant la verificació RBAC. Els *chunks* autoritzats s'acumulen al *ranking* final fins arribar al *top-K* desitjat i els no autoritzats es descarten. **La sobredimensió del pool** s'aplica perquè, fins i tot després del filtratge, quedin prou candidats per omplir els 5 resultats finals.

Fusió segura. Els resultats nets de les dos branques s'introdueixen a la fórmula RRF amb el paràmetre α assignat per l'*Intent Router* del Pas 2. Com els dos *rankings* d'entrada ja són segurs per construcció, qualsevol combinació matemàtica que se'n derivi també ho és.

****** Per a aquest *notebook* no s'ha aplicat la fase de reconstrucció per simplificar el procés.

- **Mesurament de latència i conjunt de proves**

Per obtenir mesures fiables de latència, cada consulta s'executa amb 3 iteracions de calentament (per estabilitzar les memòries cau de ChromaDB i de BM25) seguides de 10 iteracions cronometrades amb `time.perf_counter()`, agafant la mediana com a valor representatiu. **S'han definit 12 casos de prova organitzats en quatre categories: 5 tests de seguretat (S1-S5) amb usuaris no autoritzats que cerquen PII, 3 tests d'accés autoritzat (A1-A3), 2 tests d'accés interdepartamental denegat (X1-X2) i 2 consultes públiques sense PII (P1-P2).** Cada cas s'avalua sobre el *baseline* sense seguretat i sobre el recuperador asimètric.

Resultats obtinguts:

Mètrica	<i>Baseline</i>	<i>Retriever</i> Asimètric Híbrid
Violacions totals	38	0
Tests amb seguretat PASS	1/12 (8%)	12/12 (100%)

Taula 15 - Eliminació total de violacions en fase de disseny

L'eliminació de violacions és del 100 % sobre tots els casos de prova.

Els tests crítics de seguretat (S1-S5): Mostren que el *baseline* filtra contrasenyes, IBANs, codis SWIFT, correus de clients, identificadors de client, noms de persona i adreces IP a usuaris públics, mentre que el recuperador asimètric retorna 0 *chunks* sensibles en tots els casos.

Els tests interdepartamentals (X1, X2): També confirmen que la verificació de departament funciona. Un enginyer amb `cl = 3` que cerca l'IBAN del contracte i un responsable financer amb `cl = 3` que cerca correus de clients reben 0 violacions tot i tenir un `clearance_level` nominalment suficient, perquè els seus departaments no coincideixen amb els del recurs.

- Penalització de latència (ms)

Categoria	<i>Baseline</i>	Asimètric	Δ
Tests seguretat (S)	9,0	8,96	-0,04
Accés autoritzat (A)	8,6	11,68	+3,03
Interdepartamental (X)	9,6	11,17	+1,56
Consultes públiques (P)	8,9	9,16	+0,21
Mitjana global	9,0	10,0	+1,03

Taula 16 - Latència tests retriever asimètric

La penalització de latència mitjana és de +1,03 ms (+11 %), una xifra quasi irrellevant en termes pràctics. Els tests de seguretat (usuaris no autoritzats) són pràcticament igual de ràpids o lleugerament més ràpids que el *baseline* degut a que el bucle de BM25 acaba abans en trobar pocs candidats autoritzats.

- MRR i preservació de la qualitat

Categoria	MRR <i>baseline</i>	MRR asimètric
Tests de seguretat (no autoritzats - S)	1,000	0,000
Accés autoritzat (A)	1,000	1,000
Interdepartamental (X)	0,667	0,000

Publiques (P)	1,000	1,000
---------------	-------	-------

Taula 17 - MRR resultats tests retriever asimètric

La caiguda de MRR a 0 en els tests S i X és el comportament correcte per disseny: aquests usuaris no haurien de trobar les dades sensibles, i un $MRR = 0$ significa que efectivament no les troben. Per als usuaris autoritzats i les consultes públiques, el MRR es manté en 1,000, **demostrant que el filtratge no degrada la qualitat de recuperació quan l'accés és legítim.**

Problemes trobats i propostes de millora:

L'única limitació detectada és el cost lineal del post-filtering de BM25 respecte a la mida del pool. En aquest experiment, amb un pool de 30 i un corpus de 33 *chunks*, el cost és invisible ($\approx 1\ ms$), però **en un corpus de producció amb milers de *chunks* podria ser necessari ajustar dinàmicament la mida del pool segons la densitat de *chunks* autoritzats per al perfil de l'usuari.** Una idea de millora és precalcular un índex invertit per perfil RBAC.

Conclusions del pas:

El Pas 4 resol definitivament la vulnerabilitat de filtratge de PII del sistema.

L'estratègia asimètrica és arquitectònicament sòlida ja que té una eliminació del 100% de les violacions RBAC sobre els 12 cassos de prova.

La penalització de latència és pràcticament inexistent per als tests realitzats, tot i que es pot donar el problema vist anteriorment.

La qualitat de recuperació es preserva íntegrament per als usuaris autoritzats.

Finalment, amb la fusió asimètrica, el sistema RAG corporatiu queda completament alineat amb el principi Zero-Trust i ara cap usuari rep informació que excedeixi els seus permisos, cap dada sensible es filtra per cap dels dos motors de cerca, i la latència roman dins de marges acceptables tot i a haver canvis obligatoris i problemes crítics detectats per resoldre a la implementació final.

6.2.9. Pas 5: Consolidació de Mètriques

Els passos 1 a 4 han abordat de manera seqüencial quatre problemes diferenciats del sistema de recuperació: la precisió del *ranking* (Passos 1 i 2), la pèrdua de context semàntic (Pas 3) i la filtració de dades sensibles (Pas 4).

L'objectiu d'aquest últim pas és consolidar tots els resultats per extreure les conclusions finals del disseny de la Fase 2.

Conclusions:

- **Evolució de la qualitat de recuperació:** El MRR del *baseline* (0,8929) indica que de mitjana, el primer resultat rellevant apareix a la posició 1,12 del *ranking*. L'*Intent Router* del Pas 2 no millora aquesta xifra, de fet la redueix una mica, per les raons analitzades en detall a la secció 6.2.6 (**homogeneïtat temàtica del corpus, ràtio elevada top-K/corpus, solapament entre motors**). Tanmateix, l'*Intent Router* aporta un guany marginal en *Relevant@5* (+1) i una reducció de violacions (-1), i la seva utilitat principal és servir com a component reutilitzable per als passos posteriors.

Evolució de la seguretat:

Pas	Violacions	Taxa PASS	Mecanisme
1-Baseline	47 (14 consultes)	0%	Cap filtratge
2-RRF dinàmic	46 (14 consultes)	0%	Només ajust d' α
3-Pare-Fill	19 → 14 (8 tests)	12% → 25%	<i>Post-Processament RBAC</i>
4-Fusió asimètrica	38 → 0 (12 tests)	8% → 100%	<i>Pre+post filtering RBAC</i>

Taula 18 - Evolució de la seguretat del retrieving

L'evolució mostra que les tècniques de precisió (Passos 1-2) no afecten la seguretat. El Pas 3 produeix una millora parcial del 26% en violacions, però només en els fragments PII aïllats. **El salt qualitatiu es produeix al Pas 4**, on l'estratègia asimètrica d'un *pre-filtering* natiu a ChromaDB i *post-filtering* iteratiu a BM25 aconseguix una eliminació del 100% de les violacions amb una penalització de latència de +1,03 ms (+11%)

Restauració de context:

El Pas 3 demostra que la reconstrucció pare-fill restaura el context semàntic dels fragments PII aïllats amb una expansió d'entre 21-101 caràcters a 152-462 caràcters. Amb una latència addicional inferior a 0,1 ms, tot i no ser aquest resultat molt valuós ja que s'han aplicat solucions provisionals per facilitar el treball que en la implementació final no son valides, com consultar el fitxer d'output de resultats del *chunking* per cercar per al *chunk* pare. La part de seguretat passa el 100% dels tests aplicats, permetent la fusió només quan l'usuari disposa dels permisos.

S'ha vist també que per poder realitzar aquest pas, en la fase d'ingesta de documents s'haurien d'afegir camps de metadades per poder relacionar fragments.

En resum, es proposa una arquitectura de recuperació híbrida segura que es pot descriure en tres capes:

1. **Capa de classificació (*Intent Router*)**: analitza la consulta i assigna el paràmetre del RRF segons la intenció detectada (exacta, conceptual o mixta).
2. **Capa de recuperació segura (*Asymmetric Hybrid Retriever*)**: executa la cerca en paral·lel sobre ChromaDB (amb *pre-filtering* RBAC natiu) i BM25 (amb *post-filtering* Python), fusionant els resultats amb RRF únicament sobre documents autoritzats.
3. **Capa de reconstrucció (*Parent-Child Reconstruction*)**: detecta els fragments PII aïllats al *ranking* final, cerca el paràgraf pare original i, si l'usuari disposa dels permisos RBAC adequats, substitueix el fragment pel paràgraf complet restaurant el context semàntic.

I les mètriques finals del sistema de prova son:

- MRR per a usuaris autoritzats: 1 (el *chunk* rellevant és sempre el primer resultat).
- Violacions de seguretat: 0 sobre 12 tests amb perfils d'usuari diversos.
- Latència total del *pipeline*: $\approx 10\text{ ms}$ (negligible respecte a la inferència LLM).
- Expansió de context PII: 4-22x per als fragments aïllats amb autorització verificada

6.2.10. Validació a Escala: Augment del Corpus i el Rol de la Fusió Asimètrica

A la secció 6.2.6 hem identificat quatre factors estructurals que imposaven un sostre d'impacte al classificador d'intencions sobre el corpus original de 33 *chunks*:

1. **Homogeneïtat temàtica**
2. **Ratio elevada top-K/corpus (15%)**: 15% dels documents sempre estaven dins del top.
3. **Solapament entre motors**
4. **Asimetria de mida dels fragments PII**

La conclusió va ser que el benefici de l'*Intent Router* escalaria amb la diversitat i la mida del corpus i per contrastar empíricament aquesta hipòtesi, s'ha dissenyat un experiment d'augmentació del corpus i revaluació del RRF dinàmic.

Augment del corpus:

S'han generat programàticament 18 documents sintètics que simulen un entorn corporatiu heterogeni: informes financers, registres de personal, *logs* d'infraestructura, contractes de client, documentació de producte i memoràndums interns. Cadascun d'ells conté PII injectada (IBANs, contrasenyes, DNIs, correus electrònics, adreces IP) amb *clearance levels* i *departments* diversos. s'han processat amb el pipeline d'ingesta de la Fase 1 (Custom RBAC Chunking), produint 195 chunks addicionals.

Categoria	Num. Fitxers	Clearance	Key PII
<i>Financial Reports</i>	3	cl=1-3, finance	IBAN, SWIFT
<i>HR Records</i>	3	cl=0-3, hr	DNI, emails, employee IDs
<i>IT Infrastructure</i>	3	cl=1-2, engenieering	IP, passwords, server names
<i>Client Contracts</i>	3	cl=2, legal/sales	IBANs, SWIFT, person names
<i>Product Docs</i>	3	cl=0, públic	No PII
<i>Internal Memos</i>	3	cl=1-3, mixed	Emails, IDs, names

Taula 19 - Resum fitxers afegits al corpus augmentat

Mètrica	Abans (corpus original)	Després (corpus augmentat)	Augment
Documents	5	23	4,6x
Total <i>chunks</i>	33	228	6,9x
PII <i>chunks</i>	15	100	6,7x

Taula 20 - Mètriques del nou corpus respecte l'anterior

Hipòtesi i Resultat:

La hipòtesi era que, en un corpus 6,9 vegades més gran i temàticament més divers, el *baseline* estàtic (α) patiria una degradació del MRR a causa del soroll addicional (més *chunks* competint per les 5 posicions del *ranking*), i que el classificador d'intencions compensaria aquesta degradació assignant el pes correcte a cada motor. S'han avaluat 16 consultes de test

(6 exactes, 6 conceptuais i 4 mixtes) sobre el corpus augmentat, amb el dos *pipelines* (el *baseline* i el dissenyat amb RRF dinàmic) “protegits” pel filtratge asimètric RBAC del Pas 4.

El resultat no confirma la hipòtesi. Ambdós *pipelines* obtenen un MRR idèntic de 0,9688 (15 de 16 consultes amb el primer resultat rellevant a la posició 1). El classificador d'intencions no millora cap consulta, però tampoc no en degrada cap.

Perquè no aporta millores tot i l'augment del corpus: l'efecte de la reducció de soroll del filtratge RBAC:

L'anàlisi de les dades revela **que la causa principal d'aquest resultat és la Fusió Asimètrica** (Pas 4), que s'aplica a ambdós *pipelines* com a prerequisit de seguretat. El filtratge RBAC – *pre-filtering* natiu a ChromaDB i *post-filtering* iteratiu a BM25 – no solament elimina les violacions de seguretat sinó que produeix un efecte col·lateral de reducció de soroll que no havíem anticipat.

Per posar un exemple, quan un usuari de finances amb `clearance level 3` consulta dades d'IBAN, el filtratge elimina tots els *chunks* de departaments no autoritzats (enginyeria, HR, vendes, legal), reduint l'espai de cerca de 228 chunks a un subconjunt molt més petit de candidats autoritzats. Dins d'aquest subconjunt filtrat, el *chunk* rellevant destaca suficient – tant per similitud cosinus com per freqüència de termes – perquè qualsevol valor d' α el posicioni a la primera posició. El soroll temàtic que esperàvem que degradés el *baseline* ha estat absorbit pel filtratge de seguretat abans que arribi a la capa de fusió RRF.

Dit d'una altra manera: **el filtratge RBAC actua com un reductor de soroll estructural** que comprimeix l'espai de cerca efectiu per a cada consulta. Aquesta compressió és tan agressiva que el problema que el classificador d'intencions havia de resoldre, l'excés de candidats irrelevants al *ranking*, ja no existeix quan la fusió RRF s'executa.

Què hauria passat sense fusió asimètrica?

Analitzant els resultats MRR d'aplicar la capa de *post-filtering* seguretat RBAC o no sobre els models *baseline* i el dinàmic es pot veure clarament que, amb un corpus augmentat i més variat, el model amb l'*Intent Router* que modifica α segons el context de la *query* sí té un impacte directe en la millora dels resultats (documents) recuperats.

<i>Pipeline</i>	<i>Baseline model</i>	<i>Dynamic model</i>	<i>Difference</i>
Segur (RBAC on)	MRR=0,9688	MRR=0,9688	0,0000
No segur (RBAC off)	MRR=0,8438	MRR=0,9062	+0,0625 (+7,4%)

Taula 21 - Millora del MRR aplicant Intent Router amb un corpus més gran

Per mostrar un exemple específic de la diferencia dels resultats obtinguts, es mostra aquest exemple:

E1 (IBAN/SWIFT search):

- No segur (model *baseline*: $\alpha=0,5$): *chunk* rellevant a la posició 2
- No segur (model dinàmic: $\alpha=0,2$): *chunk* rellevant a la posició 1

- En el *pipeline* segur, els dos models mostren el *chunk* a la posició 1 – el *filtering* RBAC ja ha eliminat resultats que no tenen a veure amb `finance department`.

La conclusió resultant és que els dos mecanismes (*Asymmetric Fusion Filtering* + *Intent Router*) resolen el mateix problema de “soroll” a través de diferents mecanismes:

- ***Asymmetric Fusion***: Redueix el “soroll” eliminant candidats no autoritzats abans d’aplicar l’RRF, de manera que la pool de resultats restants es suficientment petita com per a que el valor del paràmetre α quasi no importi.
- ***Intent Router***: Redueix el “soroll” donant més o menys pes al motor de cerca correcte segons la *query* donada. S’ha provat que sols funciona sobre una gran quantitat de candidats (*pool*) on el motor de cerca pugui determinar el *ranking*.

Implicació per al disseny del sistema:

Aquesta observació té una implicació directa per al disseny arquitectònic: en un sistema RAG amb RBAC estricte, **l'ordre d'aplicació de les tècniques importa tant com les tècniques en si**. Qualsevol tècnica d'optimització de *ranking* que s'apliqui després del filtratge opera sobre un espai de cerca ja simplificat, i per tant el seu impacte és menor quant més petit sigui aquest espai ja simplificat. Per això sabem que **quan més gran sigui el *ranking* de cada motor de cerca, més impacte tindrà el RRF dinàmic aplicat al final**.

Aquesta conclusió reforça la decisió arquitectònica de situar el filtratge RBAC com a **primera capa** operativa del *pipeline* de recuperació, abans de la fusió i de la reconstrucció de context, en lloc de tractar-lo com un post-processament final.

6.3. Fase 3 – Enrutament Multiagent (Routing)

La Fase 2 va deixar el sistema amb un *pipeline de recuperació* Zero-Trust funcional però estàtic: un únic corpus indexat a ChromaDB i BM25, filtrat a temps de consulta per les metadades de seguretat, **amb una dependència crítica d'un pas manual a l'inici del pipeline d'ingesta** (assignació humana de `clearance_level`, `allowed_departments`, `doc_type` i `global_topic` al nivell de document) **i sense capacitat d'enrutament per col·leccions**. Aquesta configuració no és escalable degut a que l'operari humà suposa un coll d'ampolla, i la cerca global (fins ara sobre una única col·lecció) per cada consulta posa límits tant de latència com de precisió.

L'objectiu d'aquesta fase és transformar el *pipeline* de recuperació i ingesta en una arquitectura multi-agent cognitiva on totes les decisions arquitectòniques estiguin empíricament justificades contra altres alternatives. S'han executat quatre experiments successius que mesuren el cost i el benefici (precisió d'enrutament, preservació Zero-Trust, capacitat d'escalar) de cada component afegit.

Veredict final. L'arquitectura jeràrquica proposada pel pipeline de recuperació – *KSP Router-Index* (Fase A) + Asymmetric Hybrid Fusion amb RBAC per *chunk* (Fase B) + generació amb Llama 3.2 (Fase C) – té una latència mitjana total de 3.77s per consulta on ~30ms son dedicats a la infraestructura d'enrutament i recuperació segura i el 99% del temps el consumeix la pròpia generació del LLM local. Aquest 30ms és 309 més ràpid que el *baseline* provat de la família *LangChain MultiRetrieval*, i l'experiment escalar demostra també que el *baseline* es trenca estructuralment abans dels 66 departaments mentre la solució proposada manté cost $O(\log N)$ a la capa d'enrutament i el model de seguretat es conserva intacte, per exemple, en un escenari on hi ha un usuari amb privilegis insuficients, obté una resposta negativa del LLM, no una fuga de dades.

6.3.1. Resolució del Deute Tècnic – Classificador Cognitiu de Metadades (Pas 1)

Plantejament:

Fins ara, pel *pipeline* d'ingesta es requeria que l'assignació dels quatre camps globals (per tot el document) de metadades es fes manualment. Aquest pas és previ al *Custom RBAC Chunking* i servia per descriure els camps més generals dels documents sencers per a que els seus *chunks* heretessin aquests camps: un exemple són els fitxers `01_document_loading.ipynb` i `07_corpus_augmentation_ingestion.ipynb`, on per cada fitxer creat es definia un `Document_Registry[]` amb els paràmetres escrits i raonats manualment. Aquest primer pas té com a objectiu resoldre aquest deute tècnic, implementant un agent d'ingesta (***Intake Agent***) que no depengui d'aquest pas manual.

Experiment:

S'ha implementat un classificador amb Llama 3.2 que llegeix el text complet del document (una sola crida per document) i emet els quatre camps com un objecte JSON estricte, definit per un vocabulari controlat per a `allowed_departments` (6 valors) i `doc_type` (11 valors). El conjunt de prova ha estat el corpus íntegre de 23 documents amb el *ground truth* compilat a partir de les metadades assignades manualment a la Fase 2.

Resultats:

Camp	Exactitud	Falsos Negatius	Falsos Positius
clearance_level	65,2%	17,4% (4 docs)	17,4% (4 docs)
allowed_departments	73,9%	×	×
doc_type	65,2%	×	×
global_topic	(mètrica qualitativa)	×	×

Taula 22 - Resultats d'assignació camps globals amb Llama3.2

On fals negatiu significa una assignació de `clearance_level` superior a la esperada i fals negatiu el contrari.

- **Latència mitjana:** 6,83s per document.

Conclusions:

La descomposició dels errors mostra tres patrons estructurals:

1. **Frontera entre `cl=2` i `cl=3`.** El model no distingeix consistentment documents confidencials de estrictament confidencials, especialment quan el contingut és financer o conté referències a dades personals sense PII literal.
2. **Sinònims controlats.** Confusions sistemàtiques entre *manual* i *guide*, i tres casos on retorna `unknown` malgrat existir el terme adient a la llista (*report*, *changelog*).
3. **Decantament cap a "all".** Quan el document no declara explícitament el seu departament, el model tendeix a l'etiqueta transversal en lloc d'inferir-lo per contingut.

Amb un 17 % de false negatives sobre el camp més crític (`clearance_level`), es considera que el classificador no és desplegable com a pas completament automàtic en un sistema Zero-Trust de producció. No obstant això, resol el 65% dels documents sense intervenció humana i els errors són predictibles i per categoria, no aleatoris.

A més, **segueix donant millors resultats que una solució *regex* bàsica** i tot i aplicar una **latència molt més alta, es considera que** per a la “pujada” de documents al sistema, aquest **no és un camp crític** com sí que ho és per al *pipeline* de recuperació per a obtenir una resposta.

Tot i això, **la recomanació arquitectònica derivada és la d'un patró “*LLM propose, human confirms*”**: el model emplena un formulari pre-validat amb els quatre camps i el seu raonament; l'usuari que puja el document confirma o edita abans que la ingesta es completi. D'aquesta manera s'elimina molta càrrega manual sense introduir risc de fuga i trasllada el pas humà d'un formulari buit a una revisió dirigida.

6.3.2. Agent d'Ingesta I Router Index (Pas 2)

Plantejament:

Per habilitar l'enrutament jeràrquic cal un índex que contingui un vector per document en lloc d'un vector per *chunk*. La qüestió de disseny és quin és el contingut òptim d'aquest vector i n'hi ha dues alternatives:

- **Router A (baseline):** Incrustar el text brut del document. Senzill i sense cost computacional addicional, però contaminat per capçaleres, salutacions, artefactes de format, clàusules legals repetides que degrada el senyal.
- **Router B (cognitiu):** Generar amb el LLM un resum dens –**Key Semantic Points (KSP)**– de 60 a 120 paraules que capturi els tòpics, rols i propòsit del document excloent informació no rellevant, i incrustar només el KSP.

La hipòtesi és que els KSP separen millor els documents en l'espai vectorial perquè cada dimensió del seu *embedding* està carregada de senyal. També es vol comprovar l'eficiència en els resultats de les dues propostes.

Experiment:

S'han generat 23 KSP (un per document) amb un *prompt* que imposa la inclusió de substantius de tipologia i verbs de propòsit (*reports, defines, specifies, contracts*) i l'exclusió de valors sensibles i de noms de persona. S'han construït dos índexs ChromaDB en paral·lel (per Router A i Router B) i s'han avaluat amb 10 consultes dissenyades de manera que múltiples documents comparteixin vocabulari de superfície: l'objectiu és forçar els dos *routers* a prendre decisions ambigües.

Resultats:

Mètrica	Router A	Router B	Diferència (Δ)
Top-1 <i>accuracy</i>	70%	90%	+20pp
Top-2 <i>accuracy</i>	90%	100%	+10pp
MRR	0,801	0,933	+16,5%
Mitjana de distàncies (entre documents)	0,6998	0,7573	+0,0575
Mínim de distància (entre documents)	0,2573	0,0737	-0,1836

Taula 23 - Resultats KSP vs full-text embeddings al Agent Ingesta

- **Cost d'ingesta del KSP:** 7,18 s per document.
- **Mida mitjana del KSP:** 310 caràcters (≈ 26 % de la mida mitjana del document font).

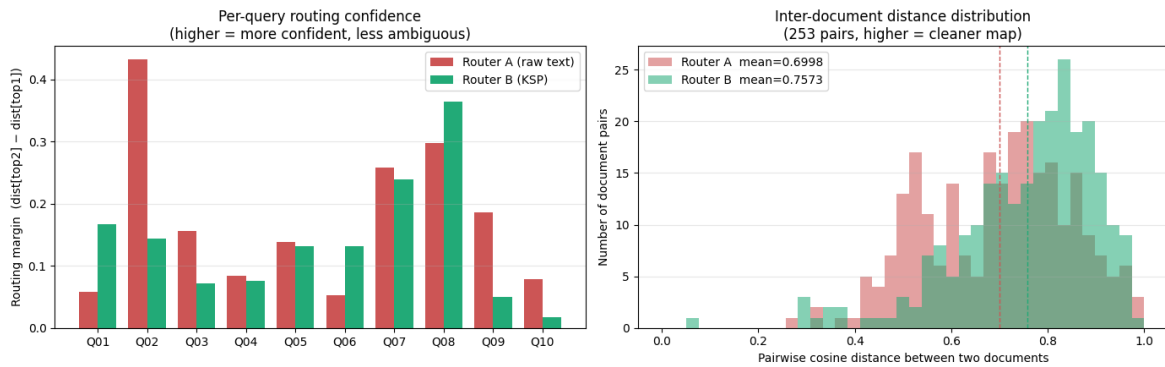


Figura 11 - Confidence of Routers & Distance Distribution per document graphics

Conclusions:

El guany de precisió no ve d'un *prompt* millor formulat, sinó del fet que els KSP eliminen estructuralment el soroll de *boilerplate* abans que l'*embedding* el processi. La parella més confusa del Router A són dos contractes comercials amb clausulat legal pràcticament idèntic (distància 0,2573); al Router B aquests dos contractes se separen, i la nova parella més propera són dos informes financers trimestrals del mateix producte (distància 0,0737), **aquesta proximitat residual és soroll**: els dos documents parlen genuïnament del mateix tòpic semàntic, només en trimestres diferents.

Es detecta un compromís menor: el marge mitjà de decisió per consulta baixa un 20 % al Router B. Això no és una pèrdua de qualitat sinó una normalització: en comprimir tots els documents a descripcions denses d'entre 60 i 120 paraules, tots els candidats ocupen una banda més homogènia de distàncies; **les decisions són més ajustades però –com mostren les mètriques de top-1 i top-3– encerten més vegades.**

Realment, per al component directament afectat (*Ensemble Retriever*, que es part del *pipeline* de recuperació) el que importa és que el document correcte estigui en el top-2, mètrica que com es mostra en la figura anterior, es compleix i garanteix que la capa posterior mai operarà fora del document rellevant.

Concloent, Els KSP són l'objecte a implementar per a enrutament jeràrquic degut als seus resultats i que el cost addicional (7.18 s per document, una vegada per ingesta) que genera és acceptable al ser part del *pipeline* d'ingesta.

6.3.3. Desqualificació del Baseline Estàndard (Pas 3)

Plantejament:

Abans d'adoptar el sistema del Router B adaptat al sistema de recuperació (*Query* → *Embed* → *Search in Router-Index* → *Extract top 2 collections* → *Run Asymmetric RRF ONLY on those 2 collections.*) com a arquitectura definitiva, s'ha considerat l'alternativa clàssica –per comparar la productivitat i eficàcia– del *framework LangChain*: un *MultiRetrievalQACChain* on la consulta de l'usuari es concatena amb la descripció verbal de totes les col·leccions disponibles, i es demana al LLM que decideixi.

Experiment:

S'han creat cinc col·leccions *mock* (HR, IT, FINANCE, LEGAL, OPERATIONS) amb descripcions de 300-500 caràcters cadascuna, i s'han executat les mateixes 10 consultes del Pas 2 contra: (a) un router LLM-as-a-judge amb prompt que inclou totes les descripcions, i (b) el Router B embedding-based del Pas 2 amb granularitat de col·lecció (document top-1 → departament del document).

Resultats:

Mètrica	Std-LLM Router	Embedding Router	Factor
Primary accuracy	70%	100%	×
Loose accuracy	90%	100%	×
Latència mitjana per consulta	4,197s	0,014s	×309
Latència màxima per consulta	4,78s	0,027s	×
Tokens de prompt per a 5 col·leccions	789	0	+72%

Taula 24 - Resultats MultiRetrievalQACChain vs Index-Router

On *Loose Accuracy* representa que el router no ha encertat la col·lecció corresponent al departament exacte però sí ha anat a alguna que pot fer donar una resposta a la pregunta al LLM mentre *Primary Accuracy* representa que ha encertat exactament la col·lecció a la que anar a buscar el *chunk*.

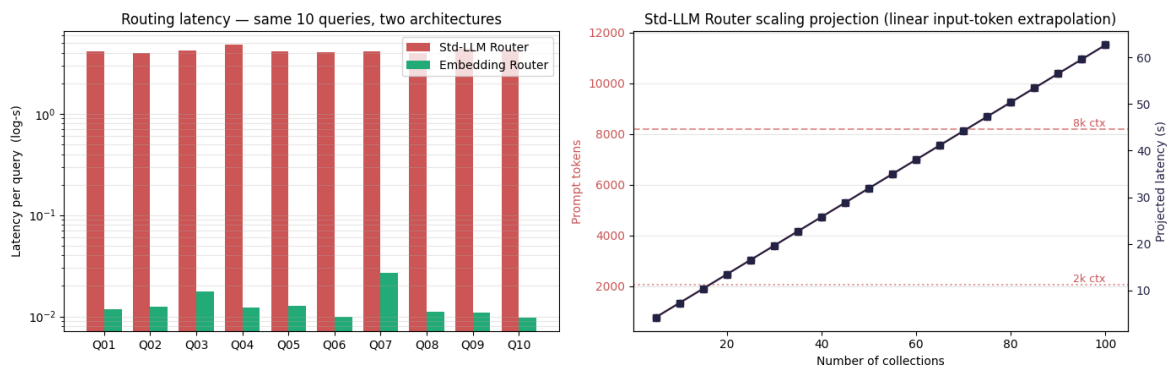


Figura 12 - Latency & Scaling Projection Router-Index vs LLM-based Comparison

En quant a escalabilitat, suposant que cada nou departament afegeix una descripció de longitud mitjana (~112 *tokens*), el cost del *prompt* creix linealment i la latència creix

aproximadament en proporció directa al nombre de *tokens* d'entrada com es pot veure en la figura anterior.

S'ha observat a més que el *baseline* es trenca a ~18 col·leccions amb la configuració per defecte d'Ollama i a ~66 col·leccions amb el context estès que estem utilitzant. Més enllà d'aquest llindar, les descripcions es trunquen silenciosament i el *router* decideix amb informació parcial, per tant no és un degradament gradual sinó una baixada molt significativa de l'exactitud.

Conclusions:

El *baseline* falla en tres dimensions ortogonals:

1. **Cota de latència inacceptable i linealment creixent segons la mida del corpus.** Cada nou departament afegeix ~3 s de processament LLM a cada consulta d'usuari. A 50 departaments ja són 32 segons de decisió d'enrutament abans de cap recuperació, temps inacceptables per la canonada de recuperació que ha de ser, per sistema i utilitat, molt més ràpid que el d'ingesta.
2. **Finestra de context com a sostre.** A partir de ~18 col·leccions el *prompt* simplement no hi cap amb la configuració per defecte (`num_ctx=2048`). No és un problema greu ja que es pot canviar com s'ha fet per a provar però sí té una conseqüència directa amb la latència de resposta.
3. **Exactitud més baixa en el mateix experiment.** Fins i tot amb només 5 col·leccions, el *router* LLM falla una consulta (Q04: qui va aprovar el pressupost R+D al consell d'administració, enrutada erròniament a LEGAL en lloc de FINANCE) i oscil·la a les ambigües. El *router* basat en *embeddings* arriba al 100% d'encert de col·lecció.

Amb aquestes dades el baseline queda empíricament desqualificat per a desplegament. L'arquitectura cognitiva amb KSP no és una optimització incremental sinó la solució que permet que el sistema escali en absolut, donant un cost d'enrutament del Router B de $O(\log N)$, mentre que el del *baseline* és $O(N)$ tant en *tokens* com en temps real.

6.3.4. Arquitectura Final (Pas 4)

Plantejament:

El Pas 4 compona tots els components validats als passos anteriors i als de la Fase 2 en un únic *pipeline* extrem a extrem: consulta → resposta (també el primer pas d'ingesta KSP). El flux segueix tres fases seqüencials i la finalitat és provar una versió “prèvia” del sistema final per a extreure resultats reals que facin de guia.

Arquitectura del procés (per fases):

Fase A – *Vector Route (KSP Router-Index)*. La consulta s'incrusta (es fa *l'embedding*) i es consulta contra els 23 vectors (de prova) KSP del Router-Index. Es retornen els dos documents més propers i, mitjançant una taula de *lookup* per la metadada `allowed_departments`, s'extreu el conjunt de departaments objectiu (col·leccions físiques a consultar per la cerca).

Fase B – *Targeted Asymmetric Retrieval*. El corpus s'ha particionat físicament a la ingesta en N col·leccions ChromaDB independents, una per cada valor únic de `allowed_departments` menys per `all`, que no hi ha col·lecció degut a una restricció arquitectònica (en el corpus actual: `hr`, `finance`, `engineering`, `legal`, `sales`), acompanyades d'un índex BM25 propi cadascuna. **La Fase B obre exclusivament les col·leccions retornades per la Fase A.** Dins d'aquestes col·leccions s'executa *l'Asymmetric Hybrid Fusion* corresponent de la Fase 2 Pas 4: recuperació paral·lela amb ChromaDB i BM25, fusió RRF amb pes α dinàmic basat en un classificador d'intenció a tres nivells ($\alpha = 0.2$ per a patrons PII o consultes exactes, $\alpha = 0.8$ per a consultes conceptuals), i finalment un filtre RBAC aplicat *chunk* a *chunk* a Python que aplica la regla `clearance_level ≤ user.cl AND (allowed_departments == user.dept OR allowed_departments == "all")`. Finalment, els *chunks* supervivents que són fragments PII aïllats s'enriqueixen amb la reconstrucció pare-fill de la Fase 2 Pas 3.

Fase C – *Secure Generation*. Els *chunks* supervivents es passen com a context al LLM Llama 3.2 via Ollama amb un *prompt* sistema que exigeix resposta basada exclusivament en el context proporcionat i negativa honesta si el context és insuficient.

Resultats:

Fase	Mitjana	Màxim
Fase A	11,2ms	11,4ms
Fase B	18,5ms	20,ms
Fase C	3,74s	3,66s
Total (end-to-end)	3,77s	3,69

Taula 25 - Resultats latència en disseny del procés de consulta

- **Impacte del filtre RBAC:** sobre 5 consultes, de 169 *chunks* recuperats de les col·leccions enrutades 95 són supervivents al post-filtre (*drop rate* del 56,3%).
- **Reconstruccions pare-fill:** 1 cas (*query* sobre el port VPN, on un fragment PII curt amb patró IP es va enriquir amb el seu paràgraf pare).

- **Denegacions d'accés:** 0 de 5.

La prova Q5 (becari de vendes amb `clearance_level = 1` demanant l'IBAN del distribuïdor espanyol, que és un fragment `c1=3` i `dept=legal`) **ha produït el comportament desitjat:** 57 *chunks* recuperats de les col·leccions que el *router* va obrir, 26 supervivents al filtre RBAC (31 eliminats per falta de permisos), i una resposta del LLM honesta del tipus “*I don't have information on the specific IBAN...*”. **En cap moment cap contingut `c1=3` ni cap contingut de `dept=legal` ha arribat al prompt del LLM.**

Figura 12 - Escenari de test en prova del sistema de recuperació

Conclusions:

El 99 % del temps total (3.74 s de 3.77 s) el consumeix la pròpia generació del LLM local. Tota l'arquitectura cognitiva d'enrutament i recuperació segura costa ~30 ms. Aquest número és el que fa que la solució arquitectònica Zero-Trust sigui portable i extremadament eficient i mantingui el nivell d'eficàcia original i fins i tot el millori.

Queda registrat com a *follow-up* per a la implementació productiva, utilitzar el model d'embeddings *paraphrase-multilingual-MiniLM-L12-v2* enlloc del usat *all-MiniLM-L6-v2* degut a que s'ha trobat que el cas de la consulta Q4 –demanada en anglès sobre una clàusula de garantia redactada en castellà al contracte de la Federació– NO ha recuperat el fragment rellevant, tot i que existeix al corpus. No és una fallada arquitectònica del Pas 4 ja que, el sistema ha enrutat correctament a la col·lecció legal, ha aplicat el filtre RBAC correctament, i ha entregat el top-5 al LLM, però com el model utilitzat és entrenat majoritàriament en anglès, no alinea prou bé frases com “*warranty period*” amb “garantia 24 mesos”.

En resum,

- **El classificador cognitiu de metadades** elimina el molta de la càrrega manual d'assignació global i fixa el patró operatiu “*LLM propose, human confirms*” per als casos residual ambigus.
- **L'agent d'ingesta basat en Key Semantic Points** millora l'*accuracy* d'enrutament +20pp a top-1 respecte al *baseline d'embeddings* de text cru, eliminant el soroll de *boilerplate* abans del procés vectorial.
- **El *baseline* LangChain MultiRetrieval** queda empíricament desqualificat al ser 309 vegades més lent al corpus actual i amb una cota estructural d'escalabilitat a ~18–66 col·leccions segons la finestra de context.
- **L'agent jeràrquic final** entrega 3,77s de latència mitjana extrem a extrem de les quals només 30ms corresponen a infraestructura pròpia; el model Zero-Trust queda demostrat operativament a l'escenari adversarial, amb un *drop rate* RBAC del 56,3% al post-filtre per fragment.

7. Implementació

En aquest apartat es descriu el procés d'implementació del sistema, que parteix de les idees, proves, i codi desenvolupat a totes les fases de disseny. La solució està basada en *Python* i segueix els principis de *Clean Architecture*, per a que la solució final sigui evolucionable sense que un canvi al *frontend* obligui a tocar el RBAC ni viceversa. L'objectiu principals és que a partir dels components desenvolupats i provats als *notebooks*, unificar i millorar-los, afegint parts que s'haguessin simplificat o quedat pendents de millora.

7.1. Estructura del Sistema

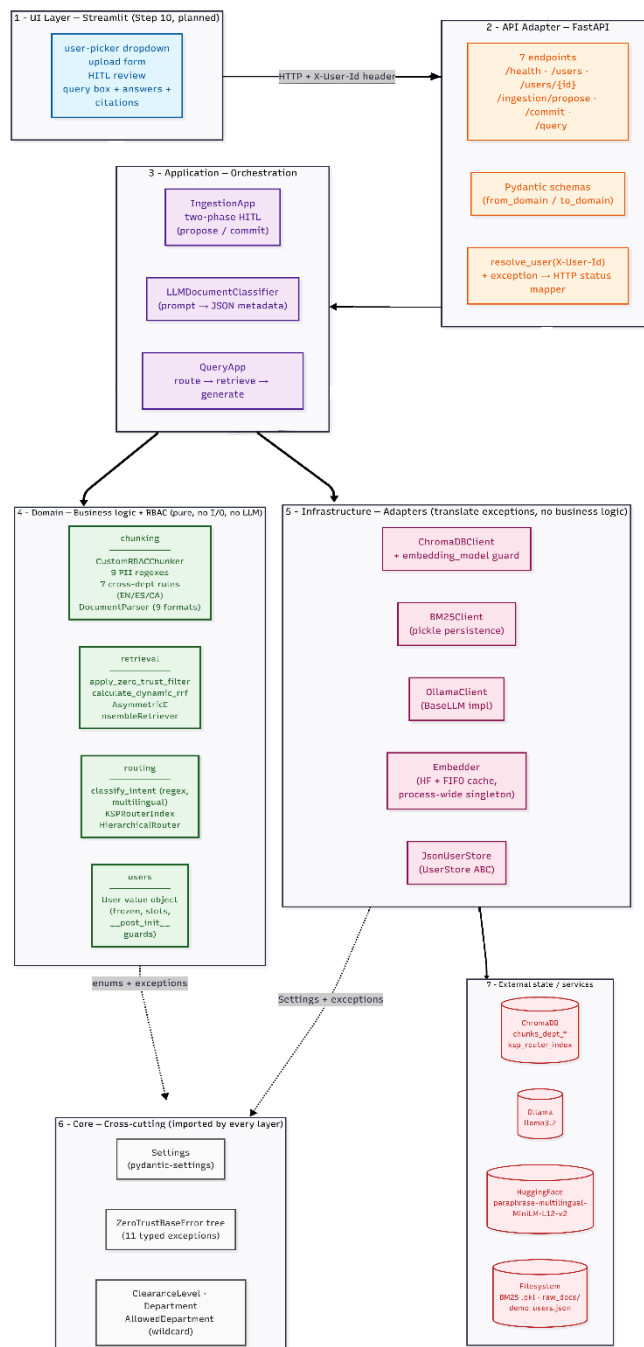


Figura 13 - Layer Structure Architecture

El codi s'organitza en set capes amb dependències unidireccionals, és a dir, la capa exterior coneix la interior, mai al revés. D'aquesta manera es garanteix la reemplaçabilitat de tecnologies, la auditabilitat i la testabilitat del `domain/` sense haver d'aixecar els *containers* Docker.

A continuació es detallen algunes de les regles que la estructura desenvolupada segueix:

1. `domain/` no importa mai de `infrastructure/`.
2. `application/` sí importa de `infrastructure/`.
3. `api/` no importa de `domain/` ni `infrastructure/` directament, tot passa per `application/`.
4. `Frontend/` mai importa res del `backend/`, tota comunicació es fa mitjançant HTTP.

7.2. Metodologia de Treball

A continuació s'expliquen les tècniques i principis utilitzats per al tractament d'errors i excepcions, i també les mesures de seguretat RBAC aplicades a les diferents capes, amb exemples.

7.2.1. Fail Fast

Per detectar errors lo més aviat possible, s'ha utilitzat el principi *Fail-Fast* que vol assegurar que qualsevol configuració incorrecte aturi l'execució en la fase més principal possible per evitar una propagació de l'error a diferents *layers*.

Si la `Settings.users_fixture_path` apunta a un fitxer JSON malformat, el sistema ha de petar al `docker compose up`, no al primer request d'un usuari. Si un `User` es construeix amb `department="legal"` (string en lloc d'enum), ha de petar al `__post_init__` del dataclass, no quan ja ha viatjat 4 capes i la `query` fa saltar una excepció.

Concretament, aquest principi s'ha materialitzat mitjançant els següents mecanismes d'arquitectura:

- **Validació al constructor:** tots els dataclasses sensibles (`User`, `ChunkInput`, `ProposedMetadata`) tenen `__post_init__` que rebutja entrades amb tipus incorrectes.
- **Càrrega anticipada en el cicle de vida:** Mitjançant l'ús de la funcionalitat *lifespan* de FastAPI, s'inicialitza i es valida el `JsonUserStore` durant l'arrencada de l'API. Això fa que qualsevol corrupció en el fitxer d'usuaris impedeixi l'arrencada del servei, anticipant-se a l'ús de la plataforma.
- **Embedding Model Guard:** Cada col·lecció dins de ChromaDB inclou l'identificador del model d'embeddings en les seves metadades. Durant la inicialització, el sistema verifica que la variable d'entorn `EMBEDDING_MODEL_NAME` coincideixi amb el model associat a la col·lecció existent; en cas de discrepància, es llança immediatament una excepció `ConfigurationError` per evitar la corrupció de l'espai vectorial

7.2.2. Single Source of Truth

Aquest principi assegura que cada peça crítica d'informació resideixi en un únic punt d'origen canònic, del qual depenen la resta de components:

- **Sincronització de Dades i Índex:** La base de dades vectorial ChromaDB actua com l'autoritat absoluta i canònica per a l'emmagatzematge dels documents (*chunks*). Els models auxiliars, com l'índex lèxic BM25 i els resums KSP, es tracten estrictament com a derivats. Per exemple, per evitar desincronitzacions, l'operació `IngestionApp.commit()` força la reconstrucció del model BM25 exclusivament a partir de l'estat consolidat de ChromaDB.
- **Gestió Centralitzada de la Configuració:** `Settings` és l'única font de configuració. Cap altre mòdul llegeix de `os.environ` directament.
- **Abstracció:** L'entitat abstracta de l'usuari es defineix exclusivament en el model de domini (`domain/users.py`). Les altres estructures com `UserSchema` exposat a la API no son redefinicions, son “vistes” dependents que hereten la estructura.

7.3. Capa Core

7.3.1. Settings – Configuració Centralitzada

`core/config.py` defineix una classe `Settings(BaseSettings)` amb `pydantic-settings`, que llegeix la configuració del procés en aquest ordre de precedència:

1. Variable d'entorn del procés (cas docker compose).
2. Fitxer `.env` al cwd (cas desenvolupament local).
3. Valor per defecte declarat al camp.

Els camps coberts:

- **ChromaDB:** `chroma_host`, `chroma_port`, `chroma_ssl`, `chroma_tenant`, `chroma_database`.
- **Ollama:** `ollama_host`, `ollama_port`, `ollama_model`, `ollama_request_timeout_s`, `ollama_num_ctx` (8192 per defecte).
- **Embedding Model:** `embedding_model_name` (paraphrase-multilingual-MiniLM-L12-v2 per defecte (canviat des de haver trobat a Q4 de la Fase 3 el problema de relació sobre clàusules en castellà), `embedding_device`.
- **Paths:** `raw_docs_dir`, `bm25_index_dir`, `artifacts_dir`, `users_fixture_path` (auto-creats al `_ensure_directory_exists` validator).
- **API:** `api_host`, `api_port`, `api_log_level`

L'accés és de tipus Singleton i es fa via `get_settings()` amb `@lru_cache(maxsize=1)`: la configuració es carrega i valida exactament una vegada per procés guardant el seu únic possible resultat a RAM.

7.3.2. Enums

`core/security.py` defineix tres enums que travessen tota la cadena:

```
class
ClearanceLevel(IntEnum):
    PUBLIC = 0
    INTERNAL = 1
    CONFIDENTIAL = 2
    STRICT = 3
```

Figura 14 - Estructura enum clearance

Hereta de `IntEnum` perquè les comparacions `user.cl >= chunk.cl` siguin naturals i la serialització JSON sigui un int simple.

```
class Department(str, Enum):          # ROUTING — sense wildcard
    HR / FINANCE / ENGINEERING / LEGAL / SALES / OPERATIONS / MARKETING

class AllowedDepartment(str, Enum):   # RBAC SCOPE — amb wildcard
    HR / FINANCE / ENGINEERING / LEGAL / SALES / OPERATIONS / MARKETING
    / ALL
```

Figura 15 - Estructures enum department i allowed_department

Aquesta dualitat és una decisió de la implementació respecte al disseny proposat anteriorment. El departament del usuari (`user.department`) és sempre `Department`. Un usuari mai pot ser "all". El departament del *resource* (`chunk.allowed_departments`) és `AllowedDepartment` i pot contenir `ALL` per a documents totalment públics. La separació es codifica i es verifica als `__post_init__` de `User` i `ChunkInput` (que conté l'estructura de metadades que ha de tenir qualsevol chunk).

7.3.3. Jerarquia d'Excepcions

`core/exceptions.py` defineix un arbre d'excepcions amb arrel `ZeroTrustBaseError`. Cada branca té semàntica clara:

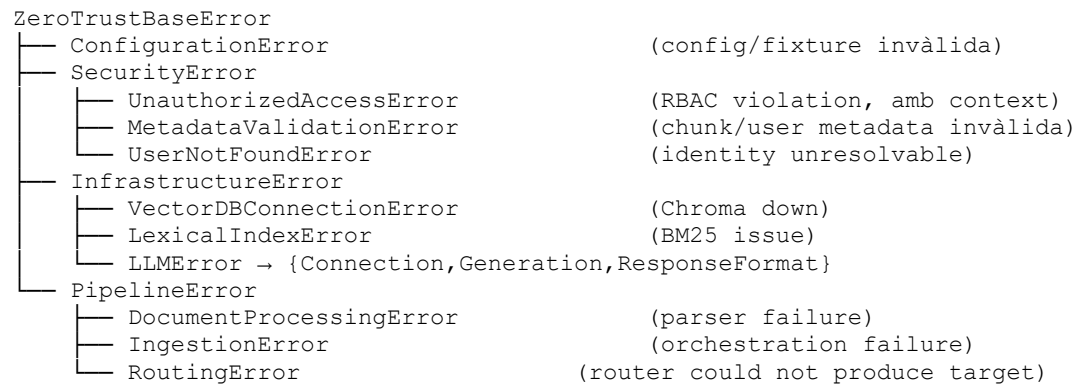


Figura 16 - Diagrama jerarquia dissenyada de les excepcions

7.4. Capa d'Infraestructura

infrastructure/ conté **adaptadors** sobre els serveis externs. Cada classe dissenyada tracta els següents punts:

1. Llegeix els seus paràmetres de connexió de Settings.
2. Tradueix errors de transport a excepcions del projecte (`VectorDBConnectionError`, `LLMConnectionError`...) preservant l'original fent un `raise ... from e`.
3. No conté lògica de negoci: cap *chunking*, cap RBAC, cap prompt engineering es tracta aquí.

7.4.1. ChromaDBClient

infrastructure/vector_db.py cobreix `chromadb.HttpClient` i els seus mètodes principals son:

- `health_check()`: retorna bool per al *startup probe*.
- `get_or_create_collection(name, *, embedding_model=None)`: inclou el *guard* del model d'embedding que en creació via `create_collection`, el nom del codificador s'afegeix a la metadada, això és per a que en obertures posteriors, es relllegeix del servidor i es compara amb l'esperat per tal de garantir que tot estigui codificat en un mateix tipus de dades (KSP, query i chunks...) i impedir el funcionament si no es així. `Mismatch` $\Rightarrow$ `ConfigurationError` amb missatge ("re-ingest the corpus with EMBEDDING_MODEL_NAME=X, or revert to Y").
- `add_to_collection(...)`, `query_collection(...)`,
`get_documents_where(...)`, `get_all_documents(...)`,
`delete_by_metadata(...)`, `delete_collection(...)`.

Durant la implementació s'ha descobert que per a versions de ChromaDB 0.4.x, el mètode `get_or_create_collection` reemplaça la metadada assignada en el servidor quan es crida sobre una col·lecció existent. Per evitar perdre el stamp del model, l'adapter usa `list_collections()` + `get_collection()` o `create_collection()` per separat.

7.4.2. BM25Client

infrastructure/lexical_db.py cobreix `rank_bm25.BM25Okapi` amb persistència *pickle* per cada col·lecció. Decisions clau:

- **Una col·lecció = un fitxer .pkl**: `chunks_finance.pkl`, `chunks_legal.pkl`, etc. La partició per departament mitiga el cost lineal del BM25 i així cada *query* només toca els fitxers de les col·leccions rutades.
- **Lazy-load**: la primera crida a `search(name, ...)` deserialitza el fitxer; les següents serveixen des de cache en memòria. Processos que mai consulten una col·lecció no paguen el cost de deserialitzar

- Validació de nom: regex `^[A-Za-z0-9][A-Za-z0-9_-]`$` per bloquejar path traversal.
- `evict(name)` per alliberar memòria + `has_index(name)` per *checks* no-destructius.

El **refit complet (no incremental)** és la política per defecte: BM25 IDF es recalcula sobre tot el corpus a cada ``IngestionApp.commit()``. Acceptable per a corpus de milers de chunks, però queda marcat com a optimització futura per escala empresarial.

7.4.3. OllamaClient

`infrastructure/llm_client.py` defineix:

- **BaseLLM(ABC)**: És l'interfície abstracta amb `generate_response(...)` i `health_check()`. Permet substituir Ollama per qualsevol backend remot (vLLM, OpenAI, etc.) sense tocar la capa d'application/.
- **OllamaClient(BaseLLM)**: La implementació concreta amb `httpx.Client` per connection pooling. Tradueix les següents excepcions `httpx.ConnectError` → `LLMConnectionError`, `httpx.TimeoutException` → `LLMConnectionError`, `status ≥ 400` → `LLMGenerationError`, `JSON malformat` → `LLMResponseFormatError`.
- **LLMResponse**: dataclass frozen amb `text`, `model`, `latency_s`, `finish_reason`, `raw` (resposta completa per a diagnòstic).

L'opció **format_json=True** activa el mode estructurat de resposta d'Ollama generant un JSON que utilitza el `LLMDocumentClassifier`.

7.4.4. Embedder

`infrastructure/embedder.py` és el servei compartit d'embeddings i implementa:

- **Lazy load**: el `HuggingFaceEmbeddings` no es carrega fins a la primera crida a `embed(text)`.
- **Cache FIFO** de 128 queries (no s'utilitza `lru_cache` per evitar memory leaks en mètodes d'instància; implementació manual amb `OrderedDict.popitem(last=False)`).
- **Singleton** `get_embedder()` decorat amb `lru_cache(maxsize=1)`: s'ha implementat perquè el router i el retriever consumeixen la mateixa instància. L'embed que fa el router al `KSPRouterIndex.query()` es reutilitza al `_embed_query()` del retriever pocs ms després via *cache hit*.

7.4.5. JsonUserStore

En `infrastructure/user_store.py` es defineixen:

- `UserStore(ABC)`: Interfície abstracta amb els mètodes `get(user_id)` i `list_all()`.
- `JsonUserStore`: Implementació concreta que llegeix una fixture JSON al `__init__`.
 - **Validació camp a camp**: Inclou coordenades exactes en cas d'error (ex: `"entry #3, user_id='u-007': missing field 'clearance_level'"`).
 - **ConfigurationError**: Es llança a temps de construcció si el fitxer no és vàlid.

Els detalls de disseny més importants del fitxer són:

1. El JSON es llegeix una vegada. `get()` té un cost $O(1)$.
2. Per simplificar la lògica i evitar *race conditions* durant *requests*, qualsevol canvi al fitxer requereix un *restart* de procés.
3. Dos entrades amb el mateix `user_id` produeixen una excepció de `ConfigurationError`.

Es tracta una possible injecció maliciosa o errònia de departament d'usuaris, fent visiblement clara la separació comentada anteriorment entre `allowed_departments` (chunks) i `departments` (usuaris). Per exemple: `department: "all"` al fixture → `ConfigurationError`

7.4.6. Script Verificació de Salut

`scripts/healthcheck.py` és un CLI *fail-fast* que es crida des de l'API i té 5 passos seqüencials:

1. **Càrrega de Settings**: Carrega la configuració i imprimeix els endpoints esperats.
2. **Ping a ChromaDB**: Executa `ChromaDBClient.health_check()`.
 - 2.1. **Round-trip Chroma**: Realitza un cicle complet de prova (crea una col·lecció `healthcheck_<uuid>`, inserta un vector dummy, executa una query i esborra la col·lecció).
3. **Ping a Ollama**: Executa `OllamaClient.health_check()` i verifica que el model configurat estigui disponible mitjançant `list_models()`.
4. **Prova End-to-End**: Crida `generate_response("Say 'OK'")` i comprova la integritat de la resposta.

El script captura primer les excepcions de tipus `ZeroTrustBaseError` i posteriorment qualsevol `Exception` general amb *tracebacks* acolorits. És una eina útil tant per al desenvolupament local com per al *smoke test* post-deploy.

7.5. Capa del Domini (*Domain*)

Aquesta capa està compresa en el directori `domain/` i conté la majoria de la lògica dissenyada a les fases de disseny, amb millores i implementacions productives que van quedar com a pendents en aquestes.

7.5.1. Chunking

`domain/chunking/` conté el chunker custom RBAC-aware, el parser multi-format, els patrons de PII i les regles de cross-department.

Metadades

La distinció `Department` vs `AllowedDepartment` es manifesta a la metadata de cada chunk:

```
metadata = {
    "parent_doc_id": "abc...",
    "chunk_index": 7,
    "source_file": "msa.txt",
    "page_number": 1,
    "department": "legal", # Department enum
    "allowed_departments": ["legal", "operations"], # AllowedDepartment
    list
    "clearance_level": 2, # 0..3
    "contains_PII": False,
    "sensitivity_types": [],
}
```

Figura 17 - Metadades de cada chunk (producció)

- **department:** És una sola string, el *home/casa* del document i que hereten tots els seus chunks. Indica al *routing* la col·lecció física `chunks_<department>` que conté tots els chunks derivats del document pare. Mai conté "all".
- **allowed_departments:** llista d'`AllowedDepartment` values, indica el `scope gate` del RBAC filter. Pot contenir "all" per a documents públics.
- **clearance_level:** Indica el severity gate. S'augmenta automàticament quan el chunker detecta algun patró PII al *chunk* (un IBAN dins d'un *chunk* CONFIDENTIAL puja a STRICT).

El que NO s'Implementa al Chunker

`domain/chunking/core_chunker.py` deixa fora dos passos que per arquitectura, han de viure a la capa d'aplicació:

- **Assignació de metadades globals del document (LLM) + HITL:** La decisió de si un document és legal vs. engineering, si té clearance 2 o 3 (que després cada *chunk* propi podrà escalar al propi chunker), o qui en té visibilitat (`allowed_departments`), implica una crida a l'LLM i una interacció humana. No té sentit posar aquesta lògica al *chunker* perquè s'ha hagut de poder testear-se sense dependre d'Ollama

- **Decisió entre lògica pròpia vs. LLM + HITL:** Per als fragments (*chunks*) individuals, la qüestió de si un fragment concret ha de ser visible per un departament addicional (ex: HR) es resol mitjançant **regex** (`cross_dept_rules`) en lloc d'usar l'LLM. És decisió vista al disseny i escollida ja que permet mantenir la latència d'ingesta dins d'uns marges raonables.

*Tots dos passos s'implementen a `application/ingestion_app.py`. D'aquesta manera, el *chunker* es manté com un component **pur i determinista**.

Resolució augment llista `allowed_departments`: A la `ingestion_app`, amb regles regeix.

Resolució augment nivell clearance per PII: Al propi `core_chunker`, amb regles regeix.

Ampliació Formats

`domain/chunking/parsers.py` gestiona la lectura de fitxers mitjançant un *dispatcher* per extensió cap a un dels nou parsers compatibles, que extreuen el contingut en text dels documents ingests:

```
supported_extensions = [
    '.csv', '.docx', '.htm',
    '.html', '.md',
    '.pdf', '.pptx', '.txt',
    '.xlsx'
]
```

Figura 18 - Extensions fitxers suportades (implementació)

A continuació es llisten les llibreries utilitzades per a cada parser:

Text pla (.txt, .md): Lectura UTF-8 directa.

Dades estructurades:

- **.csv:** Es renderitza amb files separades per | perquè el *chunker* tracti cada fila com una línia única (**solucionant l'over-fragmentació de la Fase 1**).
- **.xlsx:** Utilitza `openpyxl`. Cada full es renderitza com a Sheet: NAME + files separades per pipes. El prefix Sheet: actua com a separador estructural per al *chunker*, forçant un tall natural entre fulls.

Documents del paquet Office:

- **.pdf:** Utilitza `pypdf` amb text concatenat per pàgines. Els documents xifrats llancen un `DocumentProcessingError`.
- **.docx:** Utilitza `python-docx` (només paràgrafs; les taules i peus de pàgina són limitacions del *baseline*).
- **.pptx:** Utilitza `python-pptx`, extraient títols i quadres de text per cada diapositiva

Web (.html, .htm): Utilitza `beautifulsoup4` amb un *strip* previ de les etiquetes `<script>`, `<style>` i `<noscript>` abans d'extreure el text net.

Qualsevol altra extensió de document detectada es tracta i marca com `DocumentProcessingError` amb missatge llistant les suportades. Errors interns de les llibreries s'encapsulen i preserven el `cause` amb el `raise ... from e`.

Detecció Patrons PII amb regex

`domain/chunking/patterns.py` defineix els 9 patrons vistos al disseny (Taula 5 - Tipus de patrons PII beta Chunking) a `PIIPattern` amb camps (`name`, `compiled`, `min_clearance`, `description`).

El que s'ha millorat respecte als *notebooks* del disseny és:

- **SWIFT:** Requereix una `keyword` específica més una longitud ISO exacta. No s'activa amb una seqüència aleatòria de caràcters (ex: ABCDEFGH aïllat no és suficient).
- **Telèfon (Phone):** Requereix el prefix “+” o una `keyword` en context. Aquesta restricció evita que es capturi qualsevol seqüència de dígit genèrica com a número de telèfon.
- **IP:** Validació estricta d'octets (0-255) aplicada mitjançant *regex group constraints*. No es limita a una cerca genèrica de tipus `\d{1,3}`, garantint que l'adreça sigui semànticament vàlida.
- **Valors monetaris (Monetary value):** Requereix obligatòriament una `finance keyword` (`payment`, `amount`, `cost`, etc.) en una finestra de proximitat de 80 caràcters. Aquesta lògica evita falsos positius amb dates, números de sèrie o IDs numèrics.

Algorisme Chunking:

A la Fase 1, una fila de CSV amb dues dades PII (per exemple, IBAN + email) produïa dos chunks duplicats perquè cada coincidència s'estenia independentment a la línia completa. El nou algorisme resol aquest problema mitjançant els següents passos:

1. **Structural split:** Divisió per separadors estructurals (seccions numèriques, clàusules legals, *timestamps* de logs, marcadors de full de càlcul, capçaleres *Markdown* o salts de paràgraf). Es proven diversos separadors i se selecciona el que produeix més parts.
2. **PII detection:** Detecció de dades sensibles per a cada part, estenent cada coincidència (*match*) a la seva línia corresponent.
3. **Merge d'interval overlapping:** Aplicació de l'algorisme clàssic de fusió d'interval. Els *spans* que comparteixen una mateixa línia es fusionen en un sol interval únic.
4. **Emissió de chunks:** Es genera un *chunk* per cada interval fusionat (que inclou la unió de tots els tipus de PII detectats) més un “remainder” amb la resta del text que queda fora dels intervals

5. **Metadata stamping:** S'assignen totes les metadades i el *chunker* eleva automàticament el nivell de `clearance` al valor màxim entre el del document pare i el de qualsevol PII detectada dins del fragment.

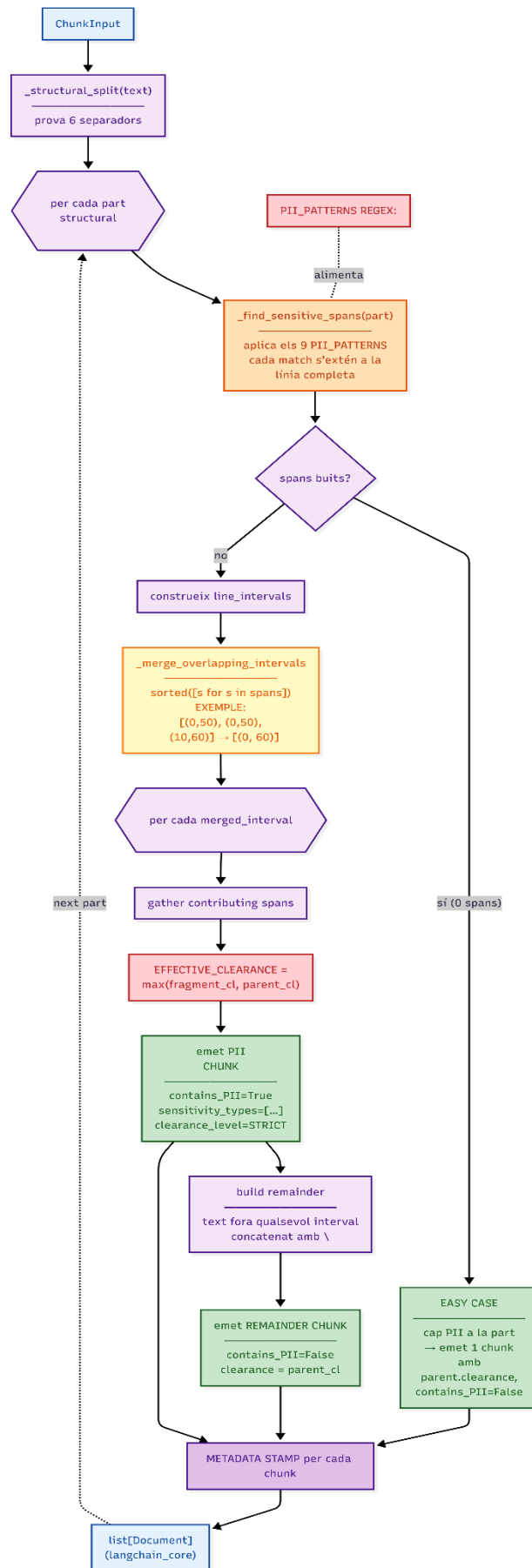


Figura 19 - Diagrama flux del chunker

7.5.2. Routing

La capa de `domain/routing` es divideix en tres mòduls principals i s'aplica un contracte d'arguments amb el *retriever*.

El Contracte amb el Retrieval

Només n'hi han dos valors que creuen la frontera del Routing per comunicar-la al retriever:

```
target_departments: list[str]    # ex: ["legal", "operations"]
alpha: float                    # ex: 0.8 ∈ [0, 1]
```

Aquests dos valors s'injecten a `AsymmetricEnsembleRetriever.retrieve_secure_context(...)` i fa que aquesta separació permeti substituir el productori (KSP Router, MultiQueryRouter de LangChain, classificador LLM o un *baseline* de regex) sense haver de modificar el *retriever*, sempre que es compleixi el contracte d'interfície.

Mòdul de Classificador d'Intenció

`intent_classifier.py` és un classificador de regles *regex* sense crida al LLM, per què es va decidir al disseny que el *routing* com s'ha d'executar a cada requesta ha de ser barat. Mapeja `QueryIntent` → α de la següent manera:

Intent	α	Raonament
PII_EXACT	0.15	BM25 guanya en strings exactes
LEXICAL	0.30	BM25 guanya en identificadors
MIXED	0.5	Balancejat (default: ChromaDB – BM25) *afegit respecte disseny
CONCEPTUAL	0.80	ChromaDB guanya en paràgrafs

Taula 26 - α per regles regex en producció

S'utilitzen tres bancs multilingües (EN/ES/CA):

- **PII keywords:** Termes de banca (iban, swift), autenticació (password, contrasenya), contactes o identificadors governamentals (dni, ssn, vat).
- **Lexical keywords:** Verbs d'acció (find, buscar), marcadors (named, titled) i referències (id:, code:).
- **Conceptual keywords:** Preguntes d'explicació (what is, què és), comparacions o narrativa (history of, història de).

Les implementacions que han canviat respecte al disseny son:

- **Regex de format:** Inclou 5 regex per a formats PII (IBAN, email, telèfon, IPv4, SWIFT) i 4 per a formats lèxics (codis interns tipus INV-2024-001, referències i frases entre cometes).
- **Pre-compilació:** Totes les paraules clau es pre-compilen amb *word-boundaries* (\b) i són *case-insensitive*. Això soluciona el bug de la Fase 1 on “cif” feia *match* dins de paraules com “*specific*”.
- **Prioritat estricta:** S'aplica l'ordre PII > Lexical/Conceptual > MIXED (fallback). Un senyal de PII és el més important: si l'usuari pregunta “com funciona l'IBAN”, tractar-ho com a cerca lèxica és segur perquè el terme “IBAN” apareixerà literalment.

Mòdul del Router d'Indexació (KSP Router-Index)

`ksp_router_index.py` utilitza una col·lecció de ChromaDB especialitzada que conté un únic registre per cada document ingestat. Cada `KSPRecord` dissenyat inclou:

- **ksp_id:** Amb forma “ksp:<parent_doc_id>”.
- **text:** Resum d'1 a 3 frases generat pel `LLMClassifier`.
- **Metadades d'auditoria:** `parent_doc_id` i `source_file` per a traçabilitat completa.
- **home_department:** Validat al `__post_init__` com un membre de l'enum `Department`.
- **Camps RBAC:** `clearance_level` i `allowed_departments` per al control d'accés inicial (com que el nivell de confidencialitat del pare és sempre menor o igual que el del seus chunks, si no pot veure cap chunk del document, retornar al router principal la informació que és buit).

L'arquitectura garanteix que cap component pugui violar la seva funció assignada:

- Escriptura: Només la capa d'ingesta pot utilitzar `add(records)` i `delete_doc()`.
- Lectura: Només el *router* pot utilitzar `query()` i `has_entries()`.

Exemple: Un *retriever* amb bugs no té permisos per afegir nous KSPs a la col·lecció.

L'aplicació del control d'accés (RBAC) també s'executa durant el *routing*:

1. La funció `query()` aplica el filtre `apply_zero_trust_filter`.
2. S'utilitza un oversample de `max(top_k * 2, 10)` per compensar els documents que el RBAC pugui descartar.

Així, un usuari sense permís per al departament X no pot saber ni tan sols que existeix un document d'aquell departament, evitant filtracions sobre el contingut o l'estructura de l'empresa.

Aquí també s'assegura que el model *d'embeddings* sigui idèntic al de les col·leccions `chunks_*` quan es fa l'*embedding* de la *query*. Una sola configuració protegeix tant el *routing* com el *retrieval*

Mòdul del Router pel Retrieval:

És l'únic punt d'entrada del sistema intern (*backend*) per a enrutar una *query* donada.

```
def route(  
    query: str,  
    user_clearance: ClearanceLevel,  
    user_department: Department,  
    ) -> RoutingDecision
```

Figura 20 - Definició funció punt d'entrada al routing

La funció de routing retorna un *dataclass* immutable `RoutingDecision` que conté `target_departments`, `alpha`, `intent` i `ksp_matches`, a més d'una propietat `is_empty`.

El sistema mai llança una excepció quan no hi ha coincidències. En lloc d'això, `is_empty=True` permet que l'orquestrador (retriever) de l'aplicació faci un **short-circuit**. Quan `ksp_index.has_entries() == False` (per exemple, en un desplegament inicial on no s'ha ingestat res), es dona aquest curt-circuit per evitar el cost computacional de generar un embedding i realitzar una la cerca buida.

La funció `route()` només accepta `Department` (enum). Es torna a controlar que no s'acceptin string o `AllowedDepartment`, mantenint la consistència amb la guarda del *chunker*.

La gestió de departaments aplicada permet configurar `max_departments` (que per defecte es = 2) i evita la duplicació d'un mateix departament com a resultat.

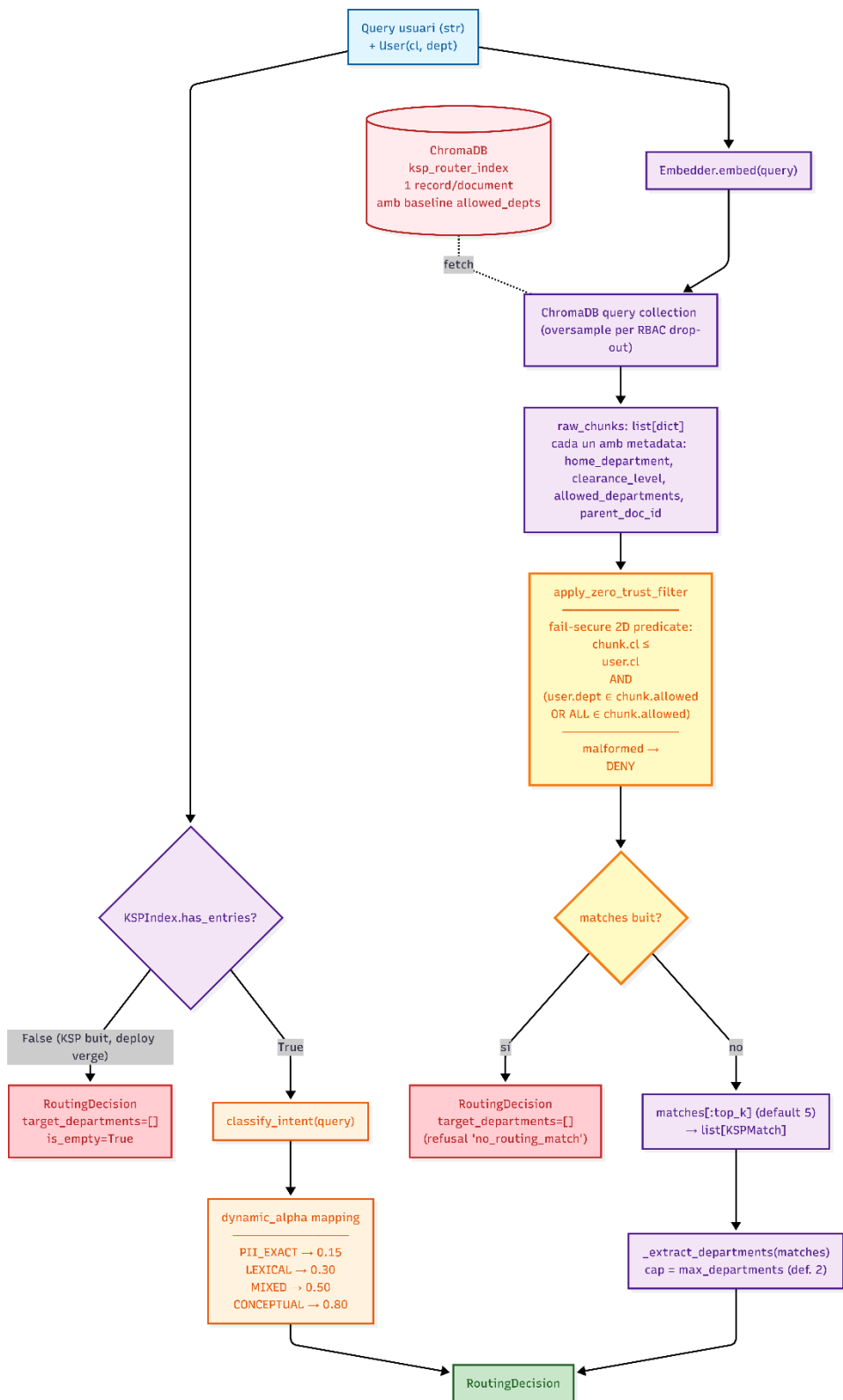


Figura 21 - Diagrama flux funcionament router

7.5.3. Retrieval

La capa de `domain/retrieval/` té com a component principal el **orquestrador asimètric** i separat d'aquest dues capes primitives: l'**RBAC filter** i **RRF fusion**.

Capes Primitives

Aquestes dues s'han fet aïllades per a que fossin testables per separat i les seves declaracions són:

- `apply_zero_trust_filter(chunks, user_clearance, user_department):`
Funció sense dependències d'infraestructura. Implementa el següent predicat 2D:
 - **Severity gate:** `clearance_level ≤ user_clearance`
 - **Scope gate:** `user_department` inclòs a `chunk.allowed_departments` OR l'existència de "ALL" a la llista de permisos.
 - És **fail-secure ja que** qualsevol metadada malformada resulta en una denegació d'accés (*deny*) i suporta el format de llistes codificades en JSON propi de ChromaDB.
- `calculate_dynamic_rrf(vector_rankings, lexical_rankings, alpha_vector, rrf_k=60):` Funció que aplica la fórmula de Cormack-Clarke-Buettcher per a la fusió de rànquings:

$$RRF(d) = \alpha \frac{1}{60 + \text{rank}_v(d)} + (1 - \alpha) \frac{1}{60 + \text{rank}_l(d)}$$

I es comprova que α estigui entre $[0, 1]$ i que $rrf_k > 0$, sinó llança un `PipelineError`.

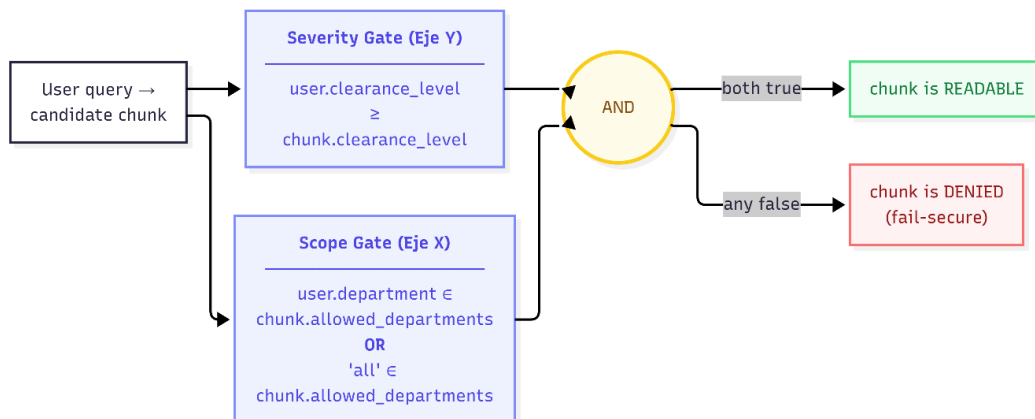


Figura 22 - Esquema aplicació RBAC 2D sobre consulta

L'Orquestrador – AsymmetricEnsembleRetriever

Com ja s'ha esmenat, aquest es el component principal que gestiona el procés de *retrieval*. Aquest rep una injecció de dependències (`vector_db`, `lexical_db`, `settings`) per facilitar el *testing* amb fakes. El pipeline `retrieve_secure_context` segueix aquests 9 passos en l'ordre descrit:

1. **Embed Query:** Es genera l'*embedding* de la consulta un sol cop (aprofitant la memòria cau si el *router* ja l'ha generat).
2. **Branch Vectorial:** Per cada departament assignat pel *router*, s'accedeix a la col·lecció `chunks_<dept>` i s'executa una cerca amb *oversampling*. Es traça l'origen mitjançant `_origin_collection`.
3. **Branch Lèxica:** S'executa `BM25Client.search` per cada departament. Si un índex no existeix (ex: primera ingesta), s'omet silenciosament, confiant en el retorn del branch vectorial.
4. **Pool Count:** Recompte previ al filtratge de seguretat.
5. **RBAC abans de la fusió:** S'aplica `apply_zero_trust_filter` de manera independent a les llistes vectorial i lèxica. Si no queda cap fragment post-RBAC, es fa un *short-circuit* retornant `[NO ACCESSIBLE CONTEXT]` – el mateix si `is_empty: bool` retornat del index-router també es comprova abans de fer la cerca.
6. **Fusió RRF:** Combinació ponderada dels rànquings utilitzant el valor d' α rebut.
7. **Top-K Winners:** Selecció dels millors resultats (per defecte 5).
8. **Reconstrucció Parent-Child:** Per cada document pare únic, es recuperen tots els seus fragments. Es **re-aplica el filtre RBAC sobre els "germans"** (per evitar que fragments *Strict* s'escapin), s'ordenen per índex i es concatenen amb la capçalera "Source: filename".
9. **Fallback Winners-Only:** Si per seguretat es bloquegen els fragments germans però els guanyadors originals són accessibles, s'utilitzen només aquests últims.

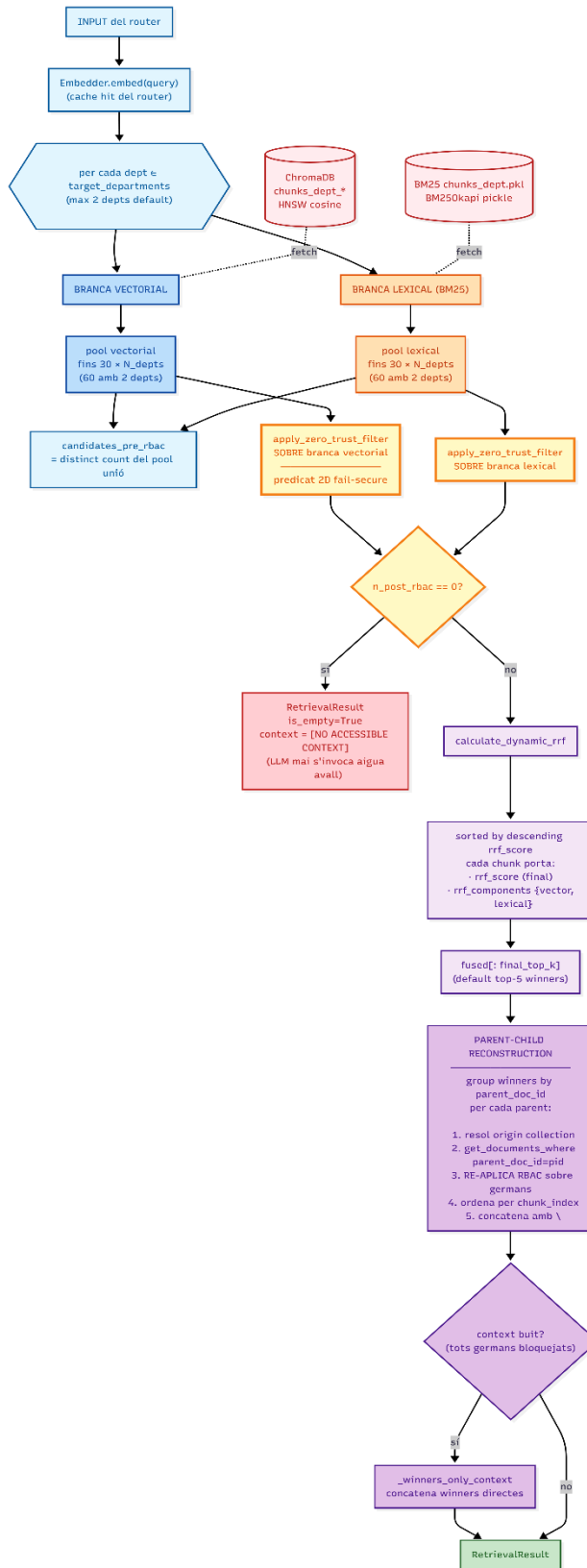


Figura 23 - Diagrama flux del retrieval

Com a decisions clau destaquen:

Un **RBAC preventiu** on es filtra abans de la fusió RRF, evitant el una situació on el Top-K queda buit perquè els millors resultats s'han eliminat en un pas de seguretat posterior. Filtrar per cada branca garanteix que el pool de fusió ja és 100% accessible.

Una **resiliència lèxica**, on la partició física per departament (`chunks_<dept>.pkl`) explicada anteriorment mitiga el cost lineal del BM25. També es tracta que un error o absència d'índex en un departament concret no avorta la consulta global, permetent que la branca vectorial segueixi donant servei.

Estratègia de Routing Materialitzada

Col·leccions físiques per departament: S'utilitzen col·leccions separades (`chunks_hr`, `chunks_finance...`). S'ha abandonat definitivament la col·lecció lògica única (`all`) i els filtres `in` de ChromaDB.

- **Raonament:** El cost del filtre `in` creix amb el nombre de departaments i una col·lecció única representa un punt de fallada crític que viola el principi de partició de dades.

Oversampling configurable: El sistema gestiona fins a 120 candidats previs al filtratge RBAC (basat en un càlcul de $30 \text{ candidats per branca} \times 2 \text{ departaments} \times 2 \text{ branques}$).

Origin Tracking: Cada fragment manté la metadada `_origin_collection`. Això és imprescindible perquè, un cop seleccionat un fragment guanyador, el sistema sàpiga exactament a quina col·lecció física ha d'anar a buscar els fragments "germans" per reconstruir el context complet del document original.

Resultat Final del Retriever

Te la següent estructura:

```
RetrievalResult(chunks, parent_documents, context, target_departments,
candidates_pre_rbac, candidates_post_rbac, alpha_vector)
```

També conté la propietat `is_empty` i mètriques pre/post-RBAC per l'auditatge de la consulta.

7.6. Tractament d'Usuaris

7.6.1. User – Value Object

En `domain/users.py` introdueix `User` com a identitat ja resolta que els orquestradors d'application consumeixen i té la següent estructura:

```
@dataclass(frozen=True, slots=True)
class User:
    user_id: str
    username: str
    clearance_level: ClearanceLevel
    department: Department
    email: str | None = None
```

Figura 24 - Estructura classe `User` pel `domain/` i `application/`

Aquest disseny desacobla qui valida la credencial (l'API adapter, que pot utilitzar un `JsonUserStore` o un futur `OAuthUserStore`) **de qui consumeix la identitat resolta** (*router*, *retriever* i orquestradors).

El constructor aplica la política fail-secure rebutjant:

- **Tipus incorrectes al departament:** Rebutja string (ex: "legal") i exigeix l'ús de l'enum `Department`.
- **Wildcards al subjecte:** Rebutja `AllowedDepartment.ALL`, ja que el comodí absorbent mai pot assignar-se a un usuari individual per construcció.
- **Tipus incorrectes de clearance:** Rebutja enters (ex: 2) en lloc de l'enum `ClearanceLevel`.
- **Cadenes buides:** No es permeten identificadors o noms d'usuari buits o només amb espais en blanc.
- **Email malformat:** Es rebutgen cadenes buides (""), acceptant només una adreça vàlida o `None`.

Aquesta validació fa el mateix que el *guard* `ChunkInput` del *chunker*, aportant més seguretat entre capes.

7.6.2. Excepcions i Abstraccions

S'ha definit l'excepció `UserNotFoundError` (`SecurityError`) amb el `user_id` estructurat al constructor. Això permet que l'API adapter la tradueixi correctament a errors 401/403 (errors d'autenticació).

La interfície `UserStore(ABC)` defineix el contracte mínim per a la infraestructura:

```
def get(self, user_id: str) -> User: ...          # raises UserNotFoundError
def list_all(self) -> list[User]: ...
```

7.6.3. Implementació i Validació

Com ja s'ha comentat al apartat d'infraestructura, la implementació concreta `JsonUserStore` s'encarrega de carregar la informació del fitxer `data/demo_users.json`.

Els usuaris creats per a la demo son:

ID	Username	Department	Clearance
u-001	Anna Garcia	hr	PUBLIC
u-002	Pere Soler	hr	CONFIDENTIAL
u-003	Marta Vidal	finance	STRICT
u-004	Joan Pla	engineering	CONFIDENTIAL
u-005	Laia Roca	legal	STRICT
u-006	Pau Costa	sales	INTERNAL
u-007	Núria Tena	operations	CONFIDENTIAL
u-008	Roger Albert	marketing	INTERNAL

Taula 27 - Usuaris dissenyats del sistema

La capa d'aplicació consumeix exclusivament objectes `User` resultats. No s'accepten tuples de metadades (`clearance_level`, `department`). D'aquesta manera el dia que es vulgui canviar `JsonUserStore` per `OAuth` només toquem l'API *adapter*, ni el `domain`/ ni l'`application`/ s'enteren.

7.7. Capa d'Aplicació (Application)

Aquesta capa esta compresa en `application/` i està composta per dos components principals, un per cada pipeline del sistema (ingesta: `ingestion_app.py` i pregunta: `query_app.py`)

7.7.1. L'App d'Ingesta

`ingestion_app.py` és un orquestrador de dos fases, proposta (feta per l'LLM) i confirmació humana Human In The Loop (HITL).

Fase A: `propose(filepath, uploader: User) → IngestionProposal`

Aquesta fase prepara el document i genera una proposta de metadades mitjançant el model:

1. **Parseig:** Es crida a `DocumentParser.parse(file_path)` del `domain/` que extreu el text complet.
2. **Truncació de control:** Es limiten els primers 6.000 caràcters (~1.500 tokens) per enviar a l'LLM, garantint que quedi espai lliure (*headroom*) dins del context d'Ollama (8.192 tokens).
3. **Classificació:** L'`LLMDocumentClassifier` analitza el fragment i el nom del fitxer per retornar la proposta de metadades i la latència del procés.
4. **Identitat:** Es genera un `parent_doc_id` (UUID4).
5. **Resultat:** Es retorna un objecte `IngestionProposal` que l'usuari podrà revisar i editar abans del commit final.

El classificador LLM utilitza un prompt estructurat en format json que obliga al LLM a respondre amb aquest esquema:

```
{
  "home_department": "<one of: hr, finance, ...>",
  "allowed_departments": ["<dept>", ...],
  "clearance_level": <int 0-3>,
  "ksp_text": "<1-3 sentence summary>",
  "reasoning": "<brief justification>"
}
```

Figura 25 - Esquema json per al `LLMDocumentClassifier`

Fase B: `commit(proposal, final_metadata, uploader) → IngestionReport`

Un cop l'usuari confirma o edita la proposta, s'executa el pipeline d'escriptura:

1. **Validació de Propietat:** Es verifica que l'usuari que fa el *commit* és el mateix que va pujar el document originalment.
2. **Processament (Chunking):** El `CustomRBACChunker` divideix el text en fragments.
3. **Expansió de Seguretat (Two-layer):**
 - **Nivell Document:** Metadades base confirmades per l'usuari.

- **Nivell Fragment (Regex):** Aplicació de `cross_dept_rules.py` per a augmentar la llista de `allowed_departments` si un fragment conté paraules clau d'un altre departament (ex: temes legals en un document d'enginyeria), s'afegeix automàticament el departament corresponent a la llista de permisos d'aquest *chunk* concret.

4. Persistència a ChromaDB:

- S'esborren versions prèvies del mateix `parent_doc_id` per garantir l'idempotència.
- S'afegeixen els nous fragments amb IDs formatats com `<parent_doc_id>:<chunk_index>`.

5. Actualització de l'Índex Lèxic: Es refà el model **BM25** des de Chroma (font única de veritat) per recalculer l'IDF amb el nou corpus.

6. Actualització del Router: S'actualitza la col·lecció `KSPRouterIndex`.

Tant la proposta del LLM com l'edició de l'usuari han de passar per `ProposedMetadata.__post_init__` que comprova que no s'acceptin strings buits, s'exigeixen enums estrictes i es valida que el `home_department` estigui sempre inclòs als permisos.

L'`ingestionReport` obtingut que es passa a la API té els següents camps:

```
@dataclass(frozen=True, slots=True)
class IngestionReport:
    parent_doc_id, source_file, uploader_user_id
    home_collection, bm25_index_name, ksp_id
    final_home_department, final_clearance_level
    final_allowed_departments: list[str] # baseline
    n_chunks_total, n_chunks_pii, n_chunks_extended
    cross_dept_rules_summary: dict[str, int] # rule_name → count
    total_latency_s
```

Figura 26 – IngestionReport results fields

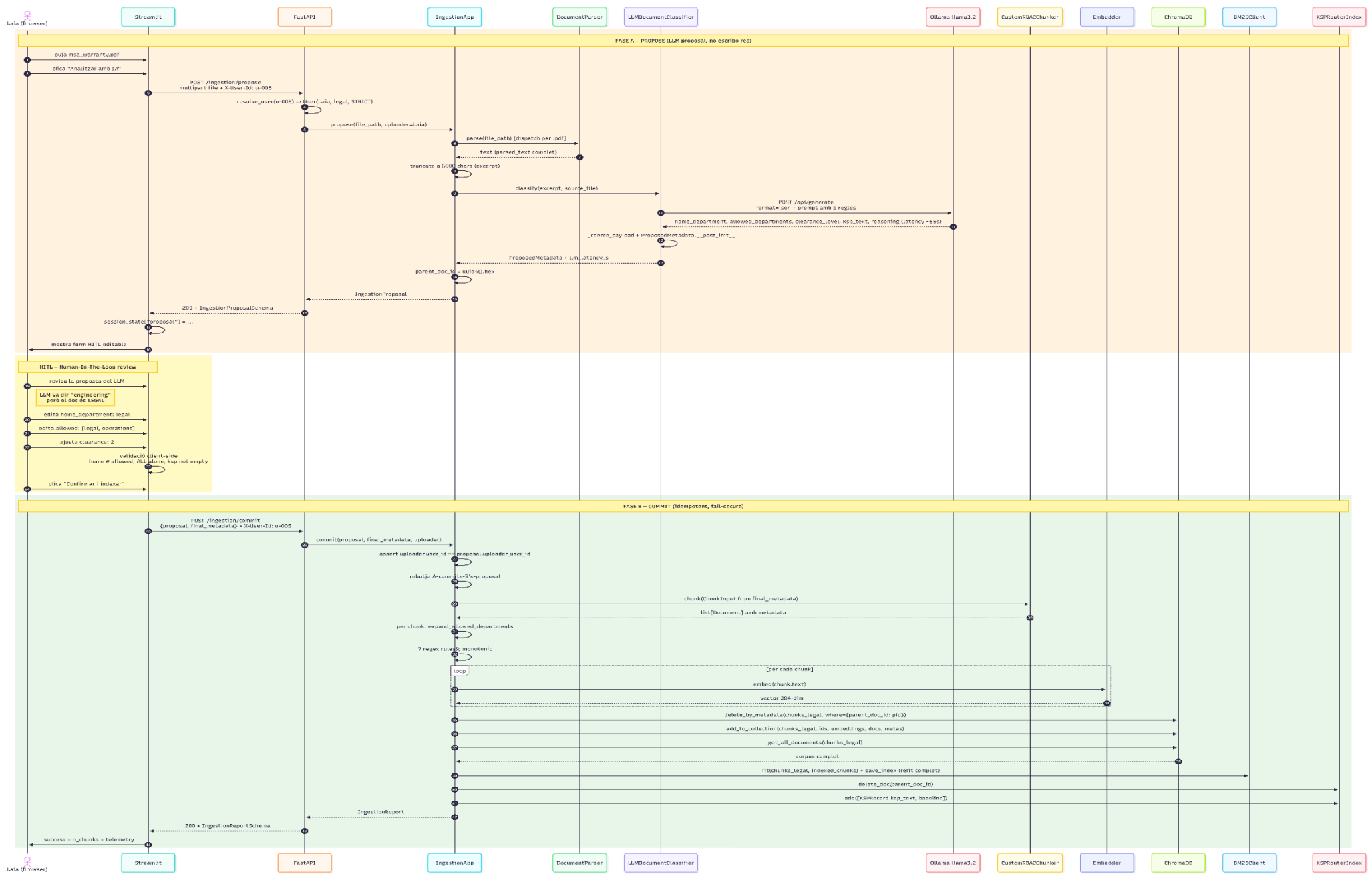


Figura 27 - Diagrama de comunicació de la fase Ingesta

7.7.2. L'App de Consulta:

`query_app.py` té tot el disseny per al pipeline de consulta. Accepta els paràmetres (`query:str, user: User`) i retorna `QueryResponse` que conté la resposta del LLM, les cites i la telemetria de l'operació completa.

Canonada de 4 Passos amb dos Short-Circuits

El flux d'execució està dissenyat per estalviar recursos i garantir la seguretat mitjançant dues guardes que eviten invocacions inútils a l'LLM:

- **Stage 1 Routing:** S'executa el `HierarchicalRouter`. Si no hi ha coincidències (`is_empty`), es retorna un **refusal** estructurat per manca de rutes (`no_routing_match`). Es salten les etapes 2 i 3.
- **Stage 2 Retrieval:** S'invoca l'`AsymmetricEnsembleRetriever`. Si no es recupera context accessible per a l'usuari, es retorna un **refusal** per manca de context (`no_accessible_context`). Es salta l'etapa 3.
- **Stage 3 Generació:** Només si les etapes anteriors han tingut èxit, es construeix el prompt i s'invoca el `BaseLLM`.
- **Stage 4 Ensamblatge:** Es genera el `QueryResponse` final.

Qualsevol de les dues *refusals* està hardcodejada en les següents constants: (`_REFUSAL_NO_ROUTING`, `_REFUSAL_NO_CONTEXT`) respectivament.

El *system prompt* imposa 5 regles estrictes: respondre basant-se només en el context, incloure cites inline tipus (`source: fitxer.pdf`), utilitzar el mateix idioma que la consulta, mantenir la concisió (2-5 frases) i no revelar detalls interns del sistema. També es configura al LLM amb els paràmetres `temperature=0.1` per maximitzar la fidelitat al context i `max_tokens=700`.

L'objecte `QueryResponse` actua com un registre complet per a l'audit log:

- Cites: Inclou el el numero de fragments accessibles i totals (`n_accessible`, `n_total`) el `parent_doc_id` i el `source_file` (nom). Permetent detectar si un usuari només està veient una part "censurada" d'un document original.
- Mètriques de rendiment: Desglossament de latències per etapa (`routing`, `retrieval`, `llm`).
- Dades de decisió: Registre de l'intent detectat, el pes α aplicat i els departaments objectiu.

7.8. Capa de l'API (FastAPI)

És la capa de comunicació entre el frontend i les crides HTTP cap el domini. Té quatre mòduls que es llisten a continuació:

- **api/dependencies.py:** Centralitza els *providers* amb `@lru_cache` per a cada *singleton* del procés (ChromaDB, Ollama, UserStore, etc.). FastAPI els injecta mitjançant `Depends(...)`, sent l'únic punt de configuració (*wiring*) de tota la cadena d'objectes.
- **api/schemas.py:** Defineix els models de Pydantic v2 per a la comunicació externa, separats dels *dataclasses* de domini. **Inclou mètodes `from_domain()` i `to_domain()`** per a la traducció bidireccional.
- **api/errors.py:** Implementa 11 manejadors d'excepcions que mapen l'arbre de `ZeroTrustBaseError` a estats HTTP deterministes:

Excepció	HTTP Code
UserNotFoundError	404
UnauthorizedAccessError	403
MetadataValidationError, DocumentProcessingError, IngestionError, RoutingError, PipelineError	400
...	...

Taula 28 - Mapeig excepcions a codis HTTP

- **api/main.py:** Factoria `create_app()` amb *lifespan* per a la càrrega anticipada (*eager-loading*) del `JsonUserStore`. Si el fitxer d'usuaris està corromput, el sistema falla immediatament en arrencar (*fail-fast*).

La funció `resolve_user` extreu l'ID de la capçalera i retorna un objecte `User` tipat. Aquest serà l'únic punt que s'hauria de modificar si es vol substituir el JSON-user-format per un sistema **OAuth**. Els *handlers* de l'API continuen rebent un objecte `User` sense conèixer la mecànica d'autenticació subjacent.

7.8.1. Endpoints

Mètode	Path	Auth	Descripció
GET	/health	X	Liveness + dependency probe.
GET	/users	X	Llista per al dropdown Streamlit
GET	/users/{id}	X	Resolució d'una identitat (404 si unknown)
POST	/ingestion/propose	X-User-Id	Multipart upload escrit a un tempfile.mkdtemp preservant el filename original (fix d'un bug del NamedTemporaryFile original que feia que les cites surtissin com source: tmpmbs9ep9c.txt)
POST	/ingestion/commit	X-User-Id	JSON body amb {proposal, final_metadata}, l'uploader cross-check viu al orquestrador
POST	/query	X-User-Id	JSON body {query}

Taula 29 - Endpoints de l'API

Nota sobre els Refusals: Les consultes rebutjades per seguretat (RBAC) retornen un HTTP 200 amb la propietat `refused=True` ja que un accés denegat no es un error sinó un comportament.

7.9. Desplegament (Local & Dockerització)

El sistema s'articula al voltant de **quatre serveis** que es comuniquen exclusivament per HTTP i n'hi han dues opcions per aixecar el projecte, una que combina serveis en Docker i serveis aixecats en l'env local (utilitzada per al desenvolupament, proves, etc...), i l'altre que aixeca tots els serveis en Docker.

Servei	Port host	Port intern
chromadb	8001	8000
ollama	11434	11434
api	8000	8000
streamlit	8501	8501

Taula 30 - Taula ports dels serveis

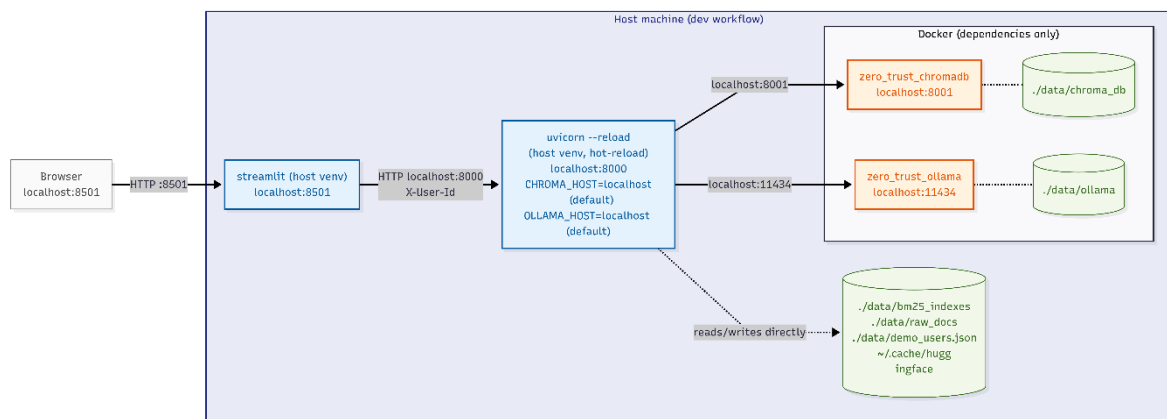


Figura 28 – Diagrama de desplegament híbrid

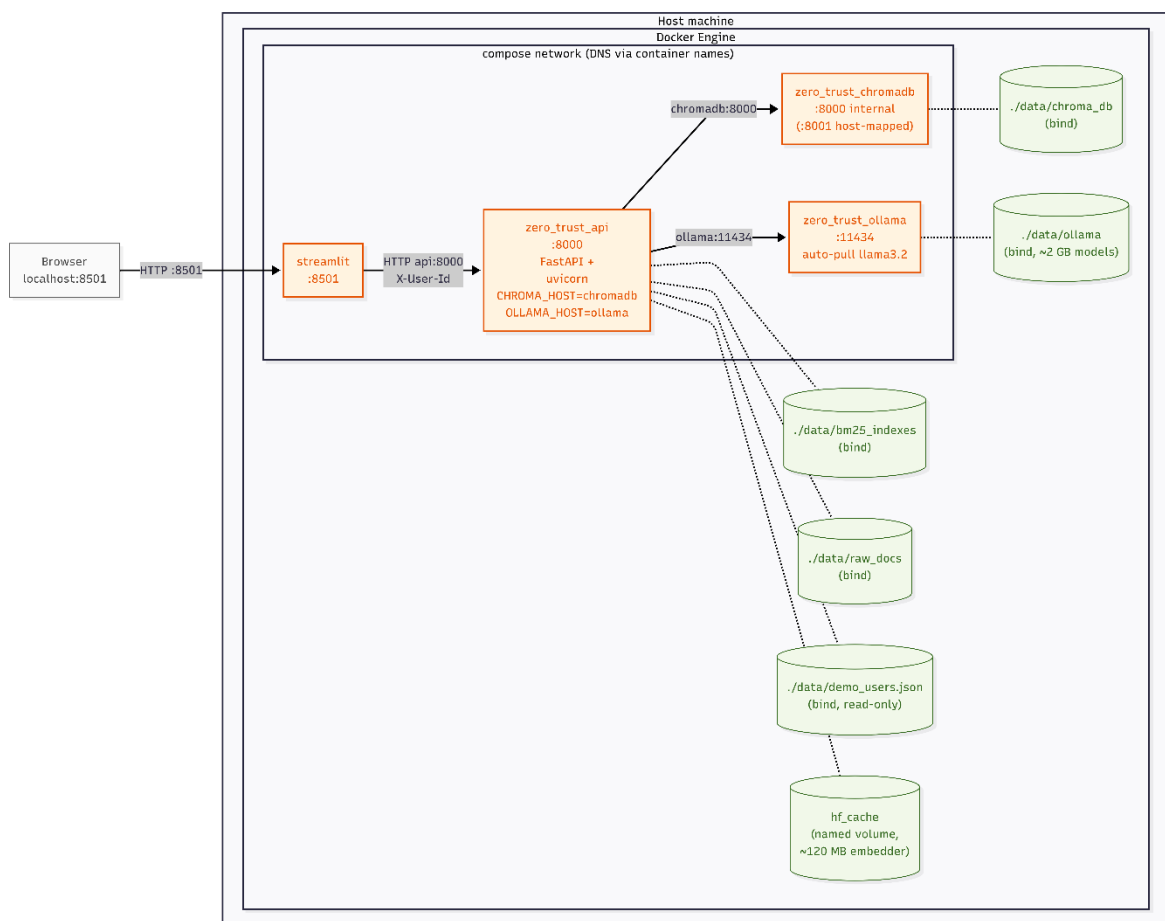


Figura 29 – Diagrama de desplegament full-Docker

Especificacions i Passos de Desplegament:

Els esquemes anteriors mostren les dues maneres de desplegament del sistema:

- **Full Compose** (`docker compose up -d`): Aquesta modalitat aixeca tots els serveis del sistema a la vegada. És l'opció per a la demostració final.
 - **Inconvenients:** cada canvi de codi del backend requereix `docker compose build api` (~10-30 s amb deps cachejades). Per al frontend, el **bind-mount** permet hot-reload sense rebuild.
- **Hybrid Dev** (`docker compose up -d chromadb ollama + python -m scripts.run_api --reload + python -m scripts.run_frontend`): En aquest mode, els motors de dades i IA (ChromaDB i Ollama) s'executen dins de contenidors, mentre que l'API s'executa de forma nativa en l'entorn virtual (venv) del sistema host. Aquesta configuració permet una iteració ràpida en el codi font, ja que el flag `--reload` aplica els canvis a l'instant sense haver d'esperar a la reconstrucció de la imatge Docker.
 - **Inconvenients:** requereix Python 3.11 al host + venv configurat manualment + variables d'entorn `CHROMA_HOST=localhost`, `OLLAMA_HOST=localhost` (que són els defaults dels Settings).

- El flag `--reload` d'uvicorn observa `api/*.py`, `application/*.py`, `domain/*.py` i `infrastructure/*.py` i reinicia el procés quan detecta canvis. Streamlit té el seu propi file watcher: cada `Ctrl+S` al `frontend/*.py` activa el botó “Rerun” al browser.

Dockerfiles y .yaml

N'hi ha dos fitxers `Dockerfile`, un pel frontend i un altre per la resta de serveis. Aquesta separació s'ha dut a terme perquè cada servei té dependències molt diferents: el backend api importa `torch`, `transformers`, `sentence-transformers`, `chromadb`, `pypdf`, `python-docx`, `openpyxl`, `python-pptx` i `beautifulsoup4` per a la lògica RAG, mentre que el frontend streamlit només necessita `streamlit` i `httpx` per parlar amb la API per HTTP. **Compartir un sol Dockerfile significaria que el container de presentació arrossegaria 10 GB de *stack*** que mai utilitza, multiplicant la mida de la imatge, el temps de construcció (build) i el més important la superfície d'atac al sistema. Aquesta separació garanteix:

1. **Invalidació de cau independent:** Un canvi al frontend no força el rebuild de la api ni al revés.
2. **Builds més ràpids:** al no compartir dependències es poden fer en paral·lel.
3. **Menys dependències:** el frontend no necessita totes les del backend.

Persistència:

El `docker-compose.yml` tracta els quatre serveis (`chromadb`, `ollama`, `api`, `streamlit`) amb les dependències ordenades via `depends_on`: { `condition: service_healthy` } per garantir que cap arrenca abans que les seves dependències responguin als seus *healthchecks*, els *overrides* d'environment per fer que la API resolgui `chromadb/ollama` per nom de servei dins la xarxa compose, i la propagació de variables `.env` del host als containers.

Per a mantenir la persistència al host, es fan 7 bind-mounts als diferents serveis:

Host path	Container path	Servei	Persisteix?	Editabl e des del host?
<code>./data/chormadb/</code>	<code>/chroma/chroma</code>	Chromadb	Sí	Sí
<code>./data/ollama/</code>	<code>/root/.ollama</code>	Ollama	Models llama3.2 descarregats	Sí
<code>./data/raw_docs/</code>	<code>/app/data/raw_docs</code>	Api	Documents pujats per usuaris	Sí
<code>./data/bm25_inde xes</code>	<code>/app/data/bm25_index es</code>	Api	.pkl per departament	Sí
<code>./data/artifacts/</code>	<code>/app/data/artifacts</code>	Api	Artifacts diversos	Sí
<code>./data/demo_users .json</code>	<code>/app/data/demo_users. json</code>	Api	Fitxer identitat	Sí (restart containe r)

./frontend/	/app/frontend	streamlit	Al codi font	Sí (hot-reload)
-------------	---------------	-----------	--------------	-----------------

Taula 31 - Mount-Binds dels containers

D'aquesta manera `docker compose down` NO destrueix res i tots els bind-mounts són carpetes del host i el named volume és gestionat per Docker fora del cicle de vida dels containers.

Limitacions Actuals

Una de les limitacions actuals però que tampoc suposa un problema sobre el desplegament és la mida de la imatge de l'api, que pesa 9,83 GB. El que mes pesa és l'ús de `PyTorch CPU` (uns 5GB).

7.10. Capa Frontend (Streamlit)

Cal subratllar que la GUI desenvolupada en aquest projecte (implementada amb Streamlit) té una funció estrictament orientada a la demostració i validació del sistema. L'objecte central d'estudi i la complexitat d'enginyeria d'aquest treball radiquen en l'arquitectura del *backend*: el motor d'ingesta multi-format, l'enrutament semàntic i el disseny del model de recuperació *Zero-Trust*.

La capa de presentació (frontend) es troba a `frontend/` i està desacoblada de la resta del sistema per HTTP, ja que mai importa `application/`, `domain/` o `infrastructure/`, només `frontend/*`, `streamlit` i `httpx`.

Es divideix en tres mòduls:

1. `frontend/api_client.py`: Consumeix del servei de la API i renderitza les respostes.
2. `frontend/components.py`: Conté peces reutilitzables i “estàtiques”, com *disclaimers*, estatus de salut, citacions...
3. `frontend/streamlit_app`: És l'entry-point que defineix la estructura i junta els components.

S'utilitza `@st.cache_resource` per a `ApiClient` perquè una sola instància `httpx` sobrevisqui a tota la sessió Streamlit: reutilitzant connexions TCP.

A continuació s'adjunten unes imatges de la solució demo on es pot veure les dos *tabs* principals (ingesta i consulta).

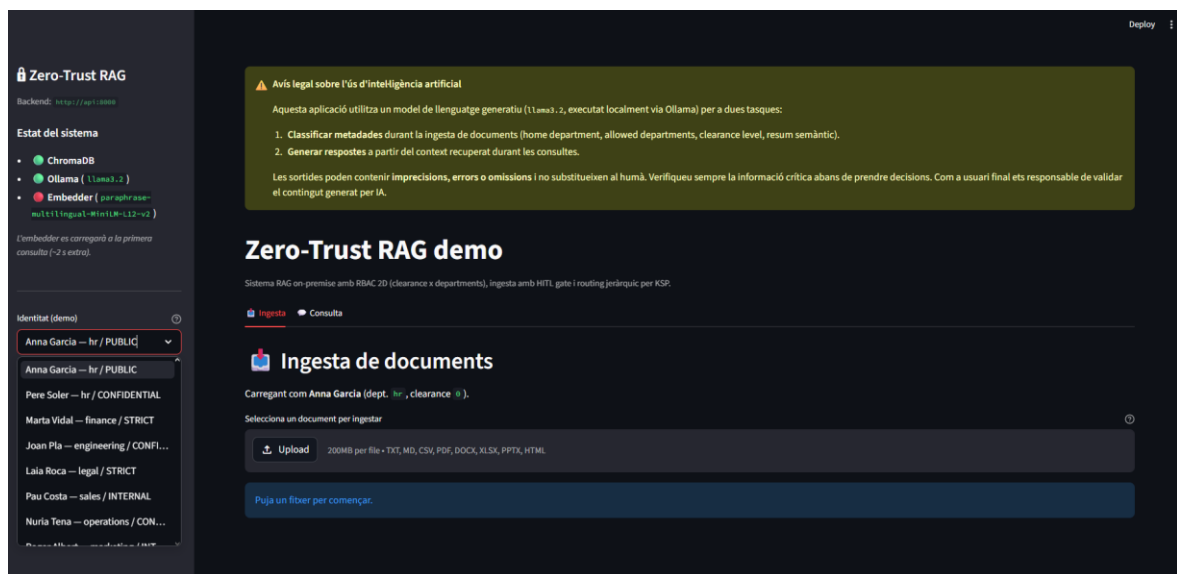


Figura 30 - Imatge solució web, ingestion tab

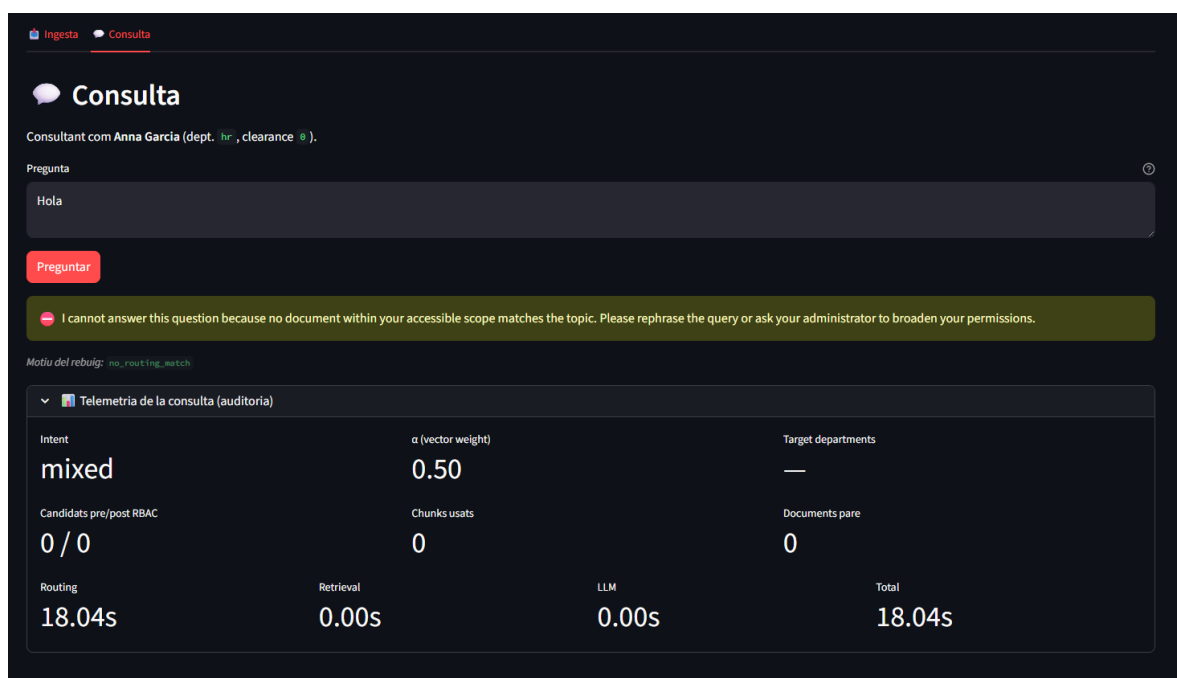


Figura 31 - Imatge solució web, query tab amb telemetria

En totes dues pestanyes es mostra l'avís legal d'ús d'IA, detallat al apartat de legislació.

En la de consulta es pot veure com s'ha intentat accedir amb el perfil d'usuari d'Anna Garcia i el sistema ha retornat un missatge d'error. Si ens fixem en la telemetria mostrada, és degut a que en la fase de Routing no s'ha trobat cap KSP coincident.

8. Avaluació

Aquest capítol recopila les proves *end-to-end* executades sobre el sistema per validar empíricament les decisions arquitectòniques del capítol d'Implementació. Tots els tests s'han executat sobre la *stack Docker* completa excepte algun que s'indica.

Àrea	Comprovació	Dades	Resultat
Chunking	Detecció de PII multi-tipus en una sola línia (IBAN + email + telèfon)	Ingestar un document amb la línia 'Laia Roca · laia@demo.com · +34 612345678 · ES91 2100...' i comprovar les metadades dels chunks emesos	3 patrons desapareixen sobre el mateix interval; merge produeix 1 chunk PII (no 3 duplicats), sensitivity_types=['email','iban','phone'], clearance_level=STRICT
Chunking	Escalada automàtica de clearance per PII	Doc CONFIDENTIAL amb un IBAN aïllat en un paràgraf altrament públic	Chunk amb IBAN promogut a STRICT (cl=3); chunks germans mantenen CONFIDENTIAL (cl=2)
Chunking	Ampliació de formats (9 extensions)	Ingestar .txt, .md, .csv, .pdf, .docx, .xlsx, .pptx, .html, .htm (1 fitxer per format)	9/9 parsejats sense errors; 1 cas .bogus → DocumentProcessingError esperat
Chunking	Rebuig de AllowedDepartment al subject (ChunkInput.__post_init)	Construir ChunkInput(parent_department=AllowedDepartment.ALL, ...)	MetadataValidationError amb missatge accionable
Cross-dept rules	Test unitari de la funció pura expand_allowed_departments	11 escenaris: no-trigger, single-rule fire, multi-rule fire, ALL absorbent, ES keyword, CA keyword, already-granted no-op, monotonía, additions sorted, baseline buit (TypeError), string entry (TypeError)	11/11 OK
Cross-dept rules	Aplicació regex per chunk sobre doc real (multi-tòpic legal)	Ingestar MSA amb seccions warranty/refund/invoice/employee; baseline [legal, ops]	6/9 chunks extended: legal_compliance 4×, finance_topics 3×, hr_topics 1×; KSP record manté baseline (no extensions)
Intent classifier	Classificació multilingüe (20 queries EN/ES/CA)	Sotmetre 20 queries cobrint PII / LEXICAL / CONCEPTUAL / MIXED	20/20 OK un cop aplicada la fix de word-boundary (bug 'cif-dins-specific' detectat i resolt)
RBAC filter (retrieval)	Test unitari apply_zero_trust_filter	4 escenaris: cl=2/dept=hr OK; cl=3/dept=engineering via wildcard 'all'; malformed → deny; JSON-encoded list parsejada	4/4 OK (fail-secure verificat)
RRF fusion	Test unitari calculate_dynamic_rrf	4 escenaris: $\alpha=0.5$ produeix winner correcte; $\alpha=1.0$ → top vector wins; $\alpha=0.0$ → top lexical wins; α fora [0,1] → PipelineError	4/4 OK
BM25 lifecycle	Fit + save + evict + lazy-load + search roundtrip	BM25Client.fit('test', corpus) + save_index + evict + nova instància que carrega lazy + search('warranty')	Score 2.542 al chunk rellevant; nom invàlid + corpus buit + índex absent → LexicalIndexError esperats
Embedding model guard	Stamp + verify del ChromaDBClient	6 escenaris: estampat at create; idempotent re-open; mismatch → ConfigurationError; legacy collection lazy-stamp; no arg preserva stamp; query funcional post-stamp	6/6 OK
KSP Router-Index	CRUD + RBAC al routing	6 escenaris: has_entries pre/post add; query top-K amb RBAC; HR user ve 0 KSPs de docs legal; validació top_k=0; delete_doc decrementa count	6/6 OK
HierarchicalRouter	Composició classifier + KSP	4 escenaris: conceptual + legal user → $\alpha=0.8$ + target=[legal]; PII + finance user → $\alpha=0.15$; user_dept string → PipelineError; KSP buit → is_empty=True short-circuit	4/4 OK
Embedder singleton	Sharing real entre router i retriever	router._ksp_index._embedder is retriever._embedder + mesura latència embed cached vs fresh	is-check TRUE; cached 0.004 ms vs fresh 13 ms — cache hit verificat
Retriever validation	Arguments invàlids	target_departments=[], query="", alpha=2.0	3/3 PipelineError esperats
Retriever empty corpus	Short-circuit is_empty=True	Query sobre chunks_engineering buit	RetrievalResult(is_empty=True, context=['[NO ACCESSIBLE CONTEXT]...'], candidates pre_rbac=0)

User layer	16 escenaris cobrint dataclass + JsonUserStore	User happy path + frozen + reject string dept + reject AllowedDepartment.ALL + reject int cl + reject empty user_id; store load 8 users + get('u-003') + unknown user → UserNotFoundError; fixture file missing/malformed/wrong-shape/bad-cl/bad-dept/wildcard-subject/missing-field/duplicate-id	16/16 OK
Ingestion E2E (sense HITL override)	Doc MSA legal amb PII, accept LLM proposal as-is	Propose → commit acceptant home=legal, allowed=[legal,ops]	9 chunks (4 PII), BM25 search 'warranty clause' → 3 hits score 0.72; KSP retrievable; HR PUBLIC ve 0/9 chunks (RBAC); re-commit idempotent (count = 9)
Ingestion E2E (HITL override)	LLM mis-classifica MSA com engineering; humà corregeix a legal	Propose retorna home=engineering; HITL substitueix per home=legal; commit	Stack final chunks legal amb 8 chunks, baseline [legal, operations] propagat; demostra valor del HITL gate
Query happy path	Laia (legal STRICT) demana per warranty	Query 'What is the warranty duration?'	Resposta amb cite (source: msa_warranty.txt); intent=conceptual α=0.8; chunks 5/1 parent; LLM latency 20.5 s
Query refusal no_routing_match	Anna (HR PUBLIC) demana per warranty	Mateixa query, X-User-Id=u-001	refused=True, refusal_reason='no_routing_match', llm_latency=0.0 s, total=70 ms
Query refusal no_routing_match (KSP buit)	Laia demana sobre KSP buida	Wipe KSP + query qualsevol	refused=True, refusal_reason='no_routing_match', retrieval_latency=0.0, llm_latency=0.0
Query validation	Inputs invàlids	query="" → PipelineError; user='u-005' (string) → PipelineError	2/2 esperats
Cross-dept differential	Pere (HR CONFIDENTIAL) demana sobre chunk hr-extended	Doc baseline [legal, ops]; chunk 5 amb cross-dept rule afegint hr	Pere és refused perquè el routing usa baseline KSP (no extensions); confirma decisió de separació granularitat router vs retriever
API /health	Liveness + dependency probe	GET sense auth	HTTP 200, status=ok, chroma=up, ollama=up, latència 5 ms
API /users	Llista + get individual + 404	GET /users, GET /users/u-005, GET /users/u-doesnotexist	8 users, 'Laia Roca' retornat, 404 amb UserNotFoundError
API ingestion validation	4 paths d'error	sense X-User-Id; X-User-Id desconegut; extension .bogus; uploader mismatch al commit	422 / 404 / 400 / 400 esperats
API ingestion happy path	Propose + commit via HTTP	multipart upload msa_warranty.txt amb X-User-Id=u-005 + commit amb HITL override	HTTP 200 als dos passos; report amb n_chunks_total=8, n_chunks_pii=3, citacions amb filename original preservat
API query	Happy path + refusal via HTTP	Laia warranty query + Anna mateixa query	200 amb cite msa_warranty.txt; 200 amb refused=true llm_latency=0; empty body → 422
API native ↔ Docker parity	Mateixos tests en mode hybrid dev i full compose	Re-execució completa de la suite e2e en cada mode	11/11 native + 11/11 docker — comportament idèntic
Service discovery container	DNS interna de compose	docker exec zero_trust_streamlit curl -fs http://api:8000/health + docker exec zero_trust_api curl -fs http://chromadb:8000/api/v1/heartbeat	Resolució via DNS Docker (127.0.0.11) confirmada
Frontend compliance	Avis legal d'IA visible	Script assert que LEGAL_DISCLAIMER_FULL i _SHORT esmenten 'intel·ligència', 'imprecisions', 'verifiqueu'	3/3 keywords presents a les dues variants
Frontend smoke	App carrega sense errors d'import	streamlit run natiu + docker exec al container per executar el script directament	HTTP 200 a / + /_score/health=ok; imports nets dins el container amb PYTHONPATH=/app
Frontend flow E2E	6 fluxos cobrint tots els botons via ApiClient	health → list_users → propose → commit amb HITL → query happy → query refusal	6/6 OK amb stack Docker complet
Stack startup (Full Compose)	Ordre garantit + healthchecks	docker compose up -d des de zero	4 containers healthy en ordre chromadb → ollama → api → streamlit; primera build ~10 min, següents arrencades ~30 s
Persistència cross-mode	Ingesta hybrid llegida des de compose	Ingestar doc amb API native + parar + arrancar mode compose + query	El doc indexat a hybrid és immediatament visible al mode compose (bind-mount compartit)
Settings override verification	Resolució de variables d'entorn	docker exec zero_trust_api python -c 'from core.config import get_settings; ...'	Dins el container: chroma_base_url='http://chromadb:8000',

			ollama_base_url='http://ollama:11434', api_host='0.0.0.0'; al host: localhost defaults
Bind-mount hot-reload	Canvi al codi frontend reflectit al container	Editar frontend/streamlit_app.py al host, refresh browser	Streamlit detecta el canvi; 'Rerun' disponible sense rebuild

Taula 32 - Taula de Tests e2e sobre el sistema

Categoria	Tests passats	Tests fallits	Notes
Chunking i PII	4/4	0	Bug del merge d'intervals overlapping de Fase 1 resolt
Cross-dept rules	17/17	0	11 unit + 6 e2e amb baseline propagat correctament
Intent classifier	20/20	0	Bug 'cif-in-specific' detectat i fixat amb word-boundary regex
RBAC filter	7/7	0	3 unitaris + 4 escenaris integrats
RRF fusion	4/4	0	α extrems + validació
BM25 lifecycle	6/6	0	Persistència pickle + lazy-load + validació noms
Embedding model guard	6/6	0	Stamp + verify + mismatch detection
Routing (KSP + Hierarchical)	14/14	0	CRUD + RBAC + composició + short-circuits
Retrieval E2E	5/5	0	Includes empty corpus + parent reconstruction
User layer	16/16	0	Validation strict cross-layer
Ingestion E2E	9/9	0	HITL value demostrat empíricament
Query E2E	8/8	0	Happy + 2 refusal paths + validació
API e2e (native + Docker)	22/22	0	Parity verificada
Frontend	10/10	0	Compliance disclaimer + flow E2E
Desplegament	4/4	0	Service discovery + persistència cross-mode + hot-reload
TOTAL	152/152	0	

Taula 33 - Resum tests i resultats

Mètrica	Valor	Context
Latència routing	~80 ms	Embed + KSP query + RBAC
Latència retrieval	~140 ms	2 branques $\times$ 2 depts + RBAC + RRF + parent reconstruction
Latència LLM (query)	~20-25 s	llama3.2 CPU-bound al container Ollama
Latència LLM (classifier)	~55-65 s	Excerpt 6000 chars + JSON format constraint
Latència refusal (no_routing_match)	< 100 ms	Sense invocar LLM ni retrieval
Embedding query cached	0,004 ms	Cache hit del FIFO 128 del Embedder
Embedding query fresh	~13 ms	Inferència multilingual MiniLM-L12 a CPU
Build time imatge api	3-5 min	Primera vegada (torch + transformers + chromadb)
Build time imatge streamlit	30-60 s	Només streamlit + httpx
Cold start ollama (model pull)	~3 min	Primera arrencada amb download de ~2 GB
Cold start ollama (model cached)	~5 s	Següents arrencades amb volum persistent
Mida imatge api	9.83 GB	Pendent optimització via torch CPU-only wheel
Mida imatge streamlit	797 MB	Justificació de Dockerfile separat
Footprint memòria embedder	~120 MB	Multilingual MiniLM-L12 carregat lazy

Taula 34 - Mètriques observades

Els components sobre els que s'ha executat estan definits a l'apartat 5.2.2.

9. Legislació

Respecte a l'aplicació del **Reglament General de Protecció de Dades (RGPD – UE) 2016/679 [8]** i la **Llei Orgànica 3/2018 (LOPDGDD – Estat Espanyol) [9]**, que regulen el tractament de dades personals, sobre aquest projecte, cal fer una distinció clara entre la fase de desenvolupament (actual) i la seva hipotètica implantació en un entorn corporatiu real.

- **Fase de desenvolupament i prova de concepte:** Durant l'execució del projecte, no s'han tractat dades personals reals. Tots els documents utilitzats per l'entrenament, ingesta i validació contenen dades completament sintètiques i fictícies. Els usuaris del sistema i la GUI (*frontend*) s'han creat amb finalitats de demostració. Per tant, les obligacions estrictes de l'RGPD no per aquesta fase del projecte.
- **Implantació productiva:** L'essència de l'arquitectura dissenyada (Custom RBAC, Zero-Trust, On-Premise) és una eina que facilita el compliment normatiu a les empreses. El sistema actua aplicant el principi de “Privacitat des del disseny”, mencionat al mateix document UE RGPD al Article 25 (*Privacy by Design* – Art. 25 RGPD [1]), garantint que l'accés a la informació estigui estrictament limitat al personal autoritzat segons el seu nivell de seguretat (`clearance_level` i `department`). En un entorn real, la responsabilitat legal d'actuar com a responsable del tractament, sobre dades d'empleats, documents interns i si la solució s'usa també per a usuaris externs, recau sobre l'empresa que desplegui el *software* internament, sent aquesta una solució per assegurar la confidencialitat i integritat de les dades.

En relació amb la **propietat intel·lectual i les llicències d'ús**, el desenvolupament de la solució s'ha dut a terme sota el paradigma del programari lliure i de codi obert (*Open Source*) juntament amb desenvolupament de *software* propietari original. L'anàlisi dels drets d'autor i llicències d'explotació es divideix en dues parts:

- **Components de tercers:** El projecte garanteix la compatibilitat legal corporativa ja que fa ús exclusivament de les següents eines amb llicències que permeten l'ús, modificació i distribució comercial privativa per a la solució final:
 - **Bases de dades i Frameworks:** ChromaDB, FastAPI, StramLit, LangChain, Pydantic operen sota llicències com Apache 2.0 [13] o MIT [14].
 - **Models d'IA LLMs:** El servidor local Ollama s'utilitza sota la llicència MIT. Pel que fa al motor de raonament, el model principal utilitzat en aquesta prova de concepte (Llama 3.2) s'opera sota la *Meta Llama 3 Community License* [12], la qual en permet l'ús en entorns de recerca i comercials. No obstant això, es vol destacar que l'arquitectura de programari del projecte s'ha dissenyat sota el patró d'Injecció de Dependències (*Strategy Pattern*). Això significa que, per a una implantació productiva corporativa real, el programari està completament preparat per substituir aquest model base per un LLM local molt més potent (o fins i tot models propietaris específics de l'empresa client) de forma transparent i sense haver d'alterar la lògica de negoci del sistema.

- **Models d'Embeddings:** Els models de vectorització semàntica utilitzats, com `all-MiniLM-L6-v2` o `paraphrase-multilingual-MiniLM-L12-v2`, operen sota llicència Apache 2.0.
- **Auditoria del projecte:** Tot i l'ús de les llibreries de tercers esmentades, el codi font original desenvolupat per a aquest Treball de Final de Grau –que inclou components amb la lògica arquitectònica *Zero-Trust* com l'algorisme Custom RBAC, el sistema d'enrutament semàntic multiagent, el recuperador asimètric RRF, etc– **es registra sota tots els drets reservats (Copyright © Arnau Faura i Ciré, 2026), però sent Source-Available Software (tothom pot veure com està fet).**

Les llicències MIT y Apache 2.0 de l'entorn de desenvolupament no exigeixen clàusules *copyleft*. Per tant, l'autor es reserva de manera exclusiva tots els drets de propietat intel·lectual, explotació, distribució i comercialització de tot el codi propi d'aquesta aplicació (no comprèn els relacionats amb la llicència de Meta [12]), sent aquest **una solució propietària amb tots els drets reservats**. Tot i que el codi font estigui disponible per a la seva visualització en un repositori públic, no s'atorga cap llicència d'ús, modificació o distribució a tercers. Permetent la futura viabilitat econòmica del projecte i la seva possible venda o llicenciamnt a empreses i corporacions que requereixin un sistema RAG privat i segur.

La Llei 34/2002, d'11 de juliol, de serveis de la societat de la informació i de comerç electrònic (LSSI-CE) [10], no és aplicable en el cas d'aquest projecte. La solució s'ha concebut exclusivament com una eina de programari corporatiu intern (Intranet) –a no ser que en un futur es vulgui obrir també a clients de l'empresa contractadora, actualment viable per disseny però no aplicable a la solució actual. En basar-se en una infraestructura *On-Premise* totalment aïllada (*Zero-Trust*), sense exposició a la xarxa pública d'internet ni provisió de serveis comercials digitals cap a consumidors o usuaris externs, queda fora de l'àmbit d'aplicació d'aquesta llei.

Amb la recent aprovació del **Reglament Europeu d'Intel·ligència Artificial (EU AI Act) [11]**, s'estableix un marc jurídic que classifica els sistemes d'IA segons el seu nivell de risc.

- **Classificació del risc:** El sistema RAG (Retrieval-Augmented Generation) desenvolupat es classifica dins la categoria de risc mínim o limitat, ja que el seu propòsit és la recuperació documental corporativa i no s'usa en sectors d'alt risc tipificats per la llei (com el triatge biomètric, l'aplicació de la llei o l'accés a serveis essencials).
- **Obligacions de transparència:** D'acord amb el *Títol IV* del Reglament, l'única i principal obligació legal exigible per a aquesta categoria de sistemes és la transparència. Els usuaris han de ser plenament conscients que estan interactuant amb una màquina i no amb un ésser humà, i que el contingut que consumeixen ha estat generat sintèticament.
 - **Implementació:** Per donar estricte compliment a aquesta normativa, s'ha implementat un mecanisme de transparència en la capa de presentació (*Frontend*). La interfície d'usuari desenvolupada amb `Streamlit` incorpora un avís legal clar i visible durant les interaccions que adverteix que s'està usant una IA per la resposta i que pot contenir imprecisions.

10. Implicacions Ètiques, d'Igualtat i Mediambientals

L'ètica en la IA es centra fonamentalment en la transparència, la veracitat i el respecte a la privadesa. Aquest projecte aborda aquests tres pilars des de la seva mateixa concepció arquitectònica:

- **Protecció i Confidencialitat:** L'arquitectura Zero-Trust i el CustomRBAC garanteixen que la informació sensible no es democratitzi erròniament.
- **Mitigació d'Al·lucinacions:** L'enfocament RAG utilitzat obliga el model a fonamentar les seves respostes exclusivament en els documents corporatius recuperats, minimitzant el risc de desinformació.
- **Transparència:** El sistema no pretén suplantar la identitat humana. S'informa explícitament als usuaris, mitjançant la interfície gràfica, que estan interactuant amb un agent.

Tot i tractar-se d'un projecte de caire marcadament tècnic (infraestructura backend), **s'han pres mesures per alinear-lo amb els principis d'igualtat i no discriminació:**

- **Reducció del biaix propi:** Els LLMs hereten biaixos cognitius, per a qualsevol que s'usi en un futur, i en aquest cas per Llama 3.2, en restringir la generació de respostes de l'LLM al context estricte dels documents interns de l'empresa, es neutralitza en gran mesura l'aparició d'aquests biaixos intrínsecs del model pre-entrenat.
- **Disseny Inclusiu:** Tant en el desenvolupament del codi font, els comentaris de la documentació i el disseny de la interfície d'usuari (UI) s'ha intentat usar un llenguatge neutre.
- **Accessibilitat Tecnològica:** La interfície desenvolupada pretén democratitzar l'accés a la informació corporativa complexa, permetent a empleats de qualsevol departament interactuar amb les dades mitjançant llenguatge natural, reduint la bretxa tecnològica interna.

L'entrenament i inferència de models d'Intel·ligència Artificial té una petjada de carboni massiva a causa del consum elèctric i les necessitats de refrigeració dels grans centres de dades (Data Centers). **Aquest projecte fa una aportació directa a la sostenibilitat tecnològica:**

- **Eficiència Computacional (*On-Premise*):** A diferència de les solucions comercials basades en el núvol aquesta arquitectura està dissenyada per executar-se localment.
- **Ús de Models Optimitzats:** La decisió d'utilitzar models locals enlloc de models comercials i més potents permet executar la inferència en *hardware* estàndard corporatiu de menor consum que en supercomputadors o sistemes molt més potents i que consumeixen més.

- Optimització de recursos: La decisió de crear l'Enrutador Jeràrquic evita saturar la finestra de context de l'LLM amb text innecessari.

11. Avaluació de Costos

11.1. Estimació de l'Esforç i la Durada

L'esforç dedicat al projecte es deriva directament de la planificació inicial del TFG presentada a la guia docent i que segueix la següent estructura:

Activitat	Hores	Detall
Mecanismes de coordinació i seguiment	42h	7h de sessions presencials i 35 de seguiment fora de classe (reunions...).
Elaboració	234h	Estudi previ, disseny, implementació, validació i redacció de la memòria.
TOTAL	276h	-

Taula 35 - Esforç de la guia docent

Aquestes 276 hores es reparteixen al llarg de 14 setmanes amb una intensitat mitja de ~20 h/setmana. La distribució per fases del projecte segueix la planificació del Gantt del apartat 3. Planificació.

Fase	Setmanes	Hores estimades	% del total
Estudi previ + anàlisi d'alternatives + setup	S1-S3	30h	8,5%
Fase 1 – Disseny Ingesta (notebooks ph1)	S4-S6	40h	11,42%
Fase 2 – Disseny Retrieval (notebooks ph2)	S7-S10	50h	14,28%
Fase 3 – Disseny Routing (notebooks ph1)	S11-S12	35h	10%
Implementació productiva	S11-S14	70h	20%
Validació i tests	S13-S15	25h	7,14%
Documentació de la memòria + revisions	S1-S15	100h	28,57
TOTAL	15 setmanes	350h	100%

Taula 36 - Esforç real x hores

Algunes fases com la documentació contenen discrepàncies amb el diagrama de Gantt degut a que també s'han dedicat hores a les setmanes prèvies però la majoria d'aquestes es contrauen en les setmanes detallades al diagrama. Cal destacar també que les hores son estimades, no reals ja que no s'ha calculat al mil·límetre.

11.2. Estimació del Cost Personal

Per al càlcul s'utilitzen tarifes representatives del sector tecnològic a Catalunya per a un perfil de desenvolupador júnior (similar al d'una pràctica remunerada en una empresa del sector) per a l'estudiant, i una tarifa de professor universitari per al tutor que ha realitzat el seguiment.

Rol	Hores	Tarifa	Subtotal
Estudiant – Elaboració del TFG	329	10 €/h	3290 €
Tutor – Supervisió i correccions	21h	20 €/h	210 €
Subtotal personal	350h		3500 €

Taula 37 - Estimació cost personal

*Les hores de coordinació i seguiment (42h) s'han repartit entre equitativament entre estudiant i tutor (21h) per fer-ho d'alguna manera tot i que cadascú ha fet les 42h.

11.3. Estimació del Cost Material

El sistema s'ha desenvolupat íntegrament al ordinador personal. El cost material es desglossa en amortització de l'equip, consum elèctric durant les 329 h efectives i la quota proporcional de connexió a internet.

Concepte	Detall del càlcul	Cost
Amortització ordinador	Equip ~1500 € amortitzat a 4 anys × 3,5 mesos	109,40 €
Consum elèctric	~80 W × 350h = 28 kWh × 0,20 €/kWh	5,6 €
Connexió a internet	35 €/mes × 3,5 mesos × 30% dedicat	36,75 €
Subtotal d'infraestructura		151,75 €

Taula 38 - Estimació cost material

11.4. Cost Llicències

Tot s'ha desenvolupat sobre components de codi obert sense costos de llicència ni subscripcions a serveis externs, conseqüència directa dels requisits RNF-01 i RNF-02.

Components	Llicències	Cost
ChromaDB, Sentence-Transformers i HuggingFace embeddings i BM25	Apache 2.0 + MIT	0 €
Ollama + Llama 3.2, MIT	MIT + Llama Community License	0 €
Docker Desktop	Subscripció gratuïta <250 empleats	0 €

Taula 39 - Cost llicències

11.5. Cost Total Estimad

Categoria	Import
Personal (estudiant + tutor)	3500 €
Infraestructura/Maquinari	151,75 €
Programari i Serveis	0 €
Costos indirectes	700 €
TOTAL ESTIMAT DEL PROJECTE	4351,75 €

Taula 40 - Cost total estimat

12. Conclusions i Reptes Futurs

Conclusions

Aquest treball ha demostrat que és possible aplicar el model de seguretat Zero-Trust de manera estructural a un sistema RAG empresarial sense renunciar a la usabilitat ni a la qualitat de les respostes. La hipòtesi inicial –que els assistents RAG basats en API *cloud* externes són incompatibles amb requisits seriosos de confidencialitat corporativa, i que filtrar al final del *pipeline* és insuficient per garantir-la– s'ha confirmat durant la fase d'exploració.

El resultat tangible és un sistema funcional, *dockeritzat* i aixecable amb una sola comanda, capaç d'oferir respostes substantivament diferents al mateix *prompt* segons qui el formula, refusant en menys de 100 mil·lisegons quan correspon i sense invocar mai el model de llenguatge contra un context buit. La separació en sis capes seguint *Clean Architecture* s'ha revelat valuosa no tant per disciplina acadèmica com sinó per la capacitat real d'iterar sobre el sistema durant el desenvolupament (desenvolupament iteratiu i incremental).

Pel que fa a continuïtats raonables, més enllà de tractar les limitacions descrites al apartat de reptes futurs, hi ha tres direccions que considero especialment importants. La primera és substituir el classificador d'intent *regex* per un model lleuger entrenat amb pocs exemples del corpus real, mantenint la baixa latència però guanyant cobertura semàntica en *queries* ambigües. La segona és explorar si els índexs BM25 poden actualitzar-se de manera incremental sense refer tot el corpus, perquè la solució actual no escala a milions de chunks tot i ser correcta per a desenes de milers. I la tercera és estudiar com l'audit log proposat podria realimentar el sistema: si l'historial mostra que un patró de query es rebutja sistemàticament per *no_routing_match*, potser indica que un departament necessita visibilitat sobre material existent però mal etiquetat, i això suggereix una eina d'anàlisi més que un canvi de codi.

Per concloure, aquest TFG ha sigut un exercici d'estudi empíric de components, viabilitat i millora per implementar una sistema final que no és un prototip sinó una base sobre la qual continuar treballant si en algun moment el context professional ho demana, amb una detallada documentació i tests que suporten aquest pensament.

Reptes Futurs

Els següents punts s'han identificat com a necessaris per a un deploy productiu real però queden fora del TFG. Es vol destacar que no haver-les implementat no afecten ni a la demostració de l'arquitectura dissenyada ni a la qualitat de la solució proposada.

- **Integració amb proveïdors d'identitat real (OAuth):** Actualment, la identitat i gestió d'usuaris es simula amb un fitxer JSON. Tot i això, l'arquitectura ha estat dissenyada per al possible *swap* (un sol punt de canvi a `api/dependencies.py:get_user_store()`).
- **Audit log persistent:** La telemetria per query existeix al `QueryResponse` però no es persisteix enlloc. Això permetria generar un sistema de auditoria de les preguntes realitzades pels usuaris.

- **Enduriment de polítiques de xarxa (CORS i *Rate Limiting*):** L'API exposada utilitza actualment un *middleware* permissiu acceptable per a un entorn de demostració local però per al desplegament professional s'hauria de crear polítiques CORS.
- **Optimització de la mida d'imatge Docker:** La imatge api actual és de 9.83 GB dominada per *torch full CPU*. S'hauria de buscar una manera de reduir-la tot i que no es un gran *must-do*.
- **Configurar el servei *Docker Ollama*** per a córrer sobre GPU, no sobre CPU.

13. Referències

- [1] Figura 1: Retool. (2024). *The state of AI in 2024* [L'estat de la IA el 2024]. Retool Reports. <https://retool.com/reports/state-of-ai-2024>
- [2] Figura 2: Chroma Core. (2026). *Chroma: The open-source embedding database* [Codi font]. GitHub. <https://github.com/chroma-core/chroma>
- [3] Figura 2: Milvus Project. (2026). *Milvus* [Codi font]. GitHub. <https://github.com/milvus-io/milvus>
- [4] Figura 2: Qdrant. (2026). *Qdrant* [Codi font]. GitHub. <https://github.com/qdrant/qdrant>
- [5] Figura 2: Weaviate. (2026). *Weaviate* [Codi font]. GitHub. <https://github.com/weaviate/weaviate>
- [6] G. V. Cormack, C. L. A. Clarke, Stefan Büttcher (2009): *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods*. Boston, MA, USA: ACM.
- [7] Dettmers, T., Pagnoni, A., Holtzman, A., i Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *Advances in Neural Information Processing Systems*, 36, 10088-10115.
- [8] Parlament Europeu i Consell de la Unió Europea. (2016). *Reglament (UE) 2016/679, de 27 d'abril de 2016, relatiu a la protecció de les persones físiques pel que fa al tractament de dades personals (RGPD)*. Diari Oficial de la Unió Europea, L 119, 4 de maig de 2016, pp. 1-88.
- [9] Jefatura del Estado. (2018). *Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales*. Boletín Oficial del Estado, núm. 294, de 6 de diciembre de 2018, pp. 119788 a 119857.
- [10] Jefatura del Estado. (2002). *Ley 34/2002, de 11 de julio, de servicios de la sociedad de la información y de comercio electrónico*. Boletín Oficial del Estado, núm. 166, de 12 de julio de 2002, pp. 25388 a 25403.
- [11] Parlament Europeu i Consell de la Unió Europea. (2024). *Reglament (UE) 2024/1689, de 13 de juny de 2024, pel qual s'estableixen normes harmonitzades en matèria d'intel·ligència artificial (Llei d'Intel·ligència Artificial)*. Diari Oficial de la Unió Europea, L, 12 de juliol de 2024.

- [12] Meta Platforms, Inc. (2024). *Meta Llama 3 Community License Agreement*. <https://llama.meta.com/llama3/license/>
- [13] Open Source Initiative (OSI). (2004). *Apache License, Version 2.0*. <https://opensource.org/licenses/Apache-2.0>
- [14] Open Source Initiative (OSI). (n.d.). *The MIT License*. <https://opensource.org/licenses/MIT>
- [15] Microgate. (s.d.). *Witty: Guía rápida Manual d'usuari*. Recuperat de <https://training.microgate.it/sites/default/files/manuali/Witty-QuickGuide-ES.pdf>

14. Recursos

Sarat Kiran. *Hybrid Retrieval-Augmented Generation (RAG) Systems*. International Journal of Scientific Research in Computer Science, Engineering and Information Technology.

Chroma Data Inc. (s.d.). *ChromaDB Official Documentation*. Recuperat de:

<https://docs.trychroma.com/docs/overview/getting-started>

Chroma Data Inc. (s.d.). *Usage Guide: Metadata Filtering*. Recuperat de:

<https://www.datacamp.com/es/tutorial/chromadb-tutorial-step-by-step-guide>

Preetam Joshi. (2025). *Vectorial DBs for RAG Comparison*. AIMon. Recuperat de:

https://www.aimon.ai/posts/comparison-of-vector-databases-for-retrieval-augmented-generation-rag-systems/?utm_source=chatgpt.com

Roie Schwaber-Cohen. 2023. *Vector Databases*. What is a Vector Database & How Does it Work? Use Cases + Examples. Recuperat de:

<https://www.pinecone.io/learn/vector-database/>

Alejandro AO. 2025. *ChromaDB Crash Course*. YouTube. Recuperat de: <https://www.youtube.com/watch?v=god8Pox1laE>

Robert C. Matrin (Uncle Bob). 2012. *Clean Architecture*. The Clean Code Blog. Recuperat de:

<https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>

Hugo Cubino Cubino. *Desarrollo e integración de un sistema basado en IA, para la consulta de información en el sector asegurador*. (Treball de Final de Grau). Universidad de Valladolid.

Hugging Face. (s.d.). *Hugging Face Model Hub*. Recuperat de: <https://huggingface.co/models>

LangChain. (s.d.). *EnsembleRetriever - LangChain Python API Reference*. Recuperat de: <https://reference.langchain.com/python/langchain-classic/retrievers/ensemble/EnsembleRetriever>

15. Annexos

15.1. Declaració d'ús d'IA.

Es declara que per al desenvolupament d'aquest TFG s'han utilitzat eines basades en Intel·ligència Artificial (IA), específicament Models de Llenguatge Extens (com Gemini o Claude), exclusivament com a instrument de suport estratègic, optimització i revisió. En cap cas aquestes eines han substituït el procés d'anàlisi, reflexió crítica ni la presa de decisions tècniques per part de l'autor.

L'ús d'aquestes eines s'ha delimitat a les següents finalitats i contribucions:

- Transició a l'Arquitectura de Producció (Refactorització de codi): Un cop finalitzada la fase d'experimentació i anàlisi empíric als *Jupyter Notebooks*, s'ha utilitzat la IA com a assistent de programació per migrar aquest codi propi cap a una Arquitectura Neta (*Clean Architecture*) i per la generació de comentaris *docstring*. Això inclou la implementació de patrons de disseny (com el *Strategy Pattern* per als adaptadors d'Ollama i ChromaDB) i suggeriments d'eficiència en algorismes concrets per solucionar reptes trobats a la fase de disseny, com ara la funció de fusió d'interval solapats (`_merge_overlapping_intervals`) dins del motor *Custom RBAC Chunker* que evita el problema de duplicació de *chunks* PII en fitxers *.csv*.
- Revisió i adequació del text acadèmic: S'ha utilitzat la IA com a suport per a la revisió ortogràfica, gramatical, i per millorar la claredat expositiva i l'adequació del to formal acadèmic d'aquesta memòria, així com per a la generació del codi base dels diagrames visuals estructurals (Mermaid).
- Cerca i formatació de referències legislatives i de llicències: La IA ha donat suport en la localització inicial i formatació estàndard (segons normes de citació) d'alguns textos legals i normatius. No obstant això, totes les referències i normatives proporcionades per la IA han estat contrastades, verificades i validades manualment a les fonts oficials abans de la seva inclusió i anàlisi definitiu al document.

15.2. Guia d'Instal·lació & Desplegament

Aquest annex conté una guia que descriu els requisits i procediment per a executar l'aplicació en mode *Full-Compose*. El procediment que s'explica està basat en Windows 11 amb *Docker Desktop*; els passos són equivalents per a altres S.O. canviant només la sintaxi del *Shell*.

15.2.1. Prerequisits

- **Git (qualsevol versió)**
- **Docker Desktop 4.x. amb el *daemon* corrent.**
- **~15 GB d'espai lliure al disc i connexió a Internet** durant el primer build per les descàrregues:
 - ~10 GB per la imatge api (PyTorch + transformers + ChromaDB client + sentence-transformers).
 - ~800 MB per la imatge streamlit, ~2 GB per al model Ollama (llama3.2) descarregat al primer arrencament, i la resta per a Chroma + caches.
- **Ports lliures al host:** 8000 (API), 8001 (ChromaDB), 8501 (Streamlit), 11434 (Ollama).
 - Si Windows té el client natiu d'Ollama instal·lat, cal aturar-lo abans perquè el port 11434 no col·lisió amb el container.

15.2.2. Procediment

1. Clonar el repositori:

```
git clone <URL-del-repositori>.git
cd ai-rag-context-auth-system
```

2. Crear el fitxer d'entorn (.env) a partir de la plantilla (tot i que es pot deixar tal com està):

```
cp .env.example .env
```

3. Personalitzar el fitxer d'entorn (.env) si escau.

4. Aixecar tot *l'stack* (la primera vegada triga 5-10 minuts per *build* d'imatges + *pull* model Llama 3.2):

```
docker compose up -d
```

Per llistar els processos que es van obrint: `docker compose ps`

Per veure el progrés d'ollama pull: `docker logs -f zero_trust_ollama`

5. Verificar que tot respon i esperar als healthchecks

```
curl http://localhost:8000/health
```

6. Obrir el frontend al navegador (<http://localhost:8501>)

15.2.3. Aturar / Reiniciar / Netejar

Per aturar els serveis (l'estat es conserva): `docker compose stop`

Per tornar a posar-ho en marxa: `docker compose start`

Per aturar i eliminar els containers: `docker compose down`

En cas de que es volgués fer un *reset* total de les dades indexades fins al moment aquests serien els passos:

- `docker compose down`
- `rm -rf data/chroma_db/* data/bm25_indexes/* data/ollama/*`
- `docker compose up -d`